Segundo parcial ICSW — Índice por tema

Cada archivo es una sección del resumen completo, con sus preguntas de repaso. El contenido sale del resumen completo, reorganizado por tema y completado con diapositivas de la cátedra donde faltaba (cada archivo lo indica).

Formato: todos los archivos están reorganizados para leerse **de corrido**. Cada uno arranca con la idea central del tema y las preguntas que ordenan el archivo. Las listas de la cátedra aparecen una sola vez dentro del relato, y al final hay una **referencia rápida** con las listas en limpio y preguntas de práctica tipo situación. Los avisos naranjas marcan confusiones típicas: dos frases que parecen iguales y no lo son, o errores del resumen del compañero.

Ideas que se repiten entre temas: tienen una nota propia en `conceptos/`, con una tabla de cómo la llama cada tema. Si dos frases de temas distintos te parecen "dos formas de verlo", fijate primero ahí:

- Testing Temprano: probar cuanto antes (ISTQB, modelo en V, "no es una fase", CI, inspecciones).
- Calidad Embebida: la calidad se construye, el testing solo la controla (principios de calidad, Manifiesto del Testing, Lean, CI).
- Error y Defecto: qué se encuentra y dónde (testing, revisiones, Lean, métricas).
- Verificación y Validación: "¿lo hicimos bien?" vs. "¿era lo que hacía falta?" (calidad, revisiones, testing, CMMI).
- Mejora Continua: mejorar de a poco y siempre (Lean, Kanban, retros, proceso basado en calidad, IDEAL, CMMI 5).
- Independencia: quien controla no es quien hizo (revisiones, testing, GAC, auditorías).
- Limitar el WIP: no empezar más de lo que se puede terminar (Lean, Kanban, métricas de flujo).

Orden de lectura sugerido: 06-calidad primero, porque su mapa ubica a toda la U4. Después, U2 (Nexus, Lean, Kanban, Métricas) y U3 (CI/CD) se leen en cualquier orden, pero Lean antes que Kanban.

Diseño de un tablero Kanban: hay una guía paso a paso aparte: guia-tablero-kanban.

Temas

Archivo	Tema	Secciones (§)	Unidad
01-nexus	Nexus	§1	U2
02-lean	Lean	§2	U2
03-kanban	Kanban	§3	U2
04-metricas	Métricas	§4	U2

Archivo	Tema	Secciones (§)	Unidad
05-practicas-continuas	Prácticas continuas (CI/CD)	§5	U3
06-calidad	Calidad	§6 y §7	U4
07-aseguramiento-de-calidad	Aseguramiento de calidad (SQA / PPQA)	§8	U4
08-modelos-y-estandares	Modelos y estándares de calidad (CMMI, SPICE, ISO)	§9	U4
09-auditorias	Auditorías de software	§10 y §11	U4
10-revisiones-tecnicas	Revisiones técnicas	§12	U4
11-testing	Testing	§13	U4
12-testing-agil-y-tdd	Testing ágil y TDD	§14	U4

Convención de fuentes

- `[Fuente: Apuntes]` = sale tal cual de la presentación de la cátedra.
- `[Fuente: Externa — X]` = investigado fuera de las presentaciones (la cátedra no lo desarrolla o lo hace solo con imágenes).
- **Texto en este color** = viene del resumen de un compañero (`Resumen ICW 2.docx`) y **no lo pude confirmar** contra una presentación ni contra una fuente oficial. Tratalo como complemento, no como apunte firme.

> **Ojo con el alcance**: **Nexus** y **Prácticas continuas** no tienen presentación propia en la carpeta de la cátedra. Para Nexus la base es el resumen del compañero, contrastado con la *Nexus Guide* oficial (scrum.org). Para prácticas continuas, la cátedra solo las nombra (en Requerimientos Ágiles y en SCM: blue-green/canary); el desarrollo es externo (Fowler/Humble).

Nota final sobre fuentes y alcance

Este documento cubre el temario oficial del 2do parcial (imagen del temario: U2 Nexus/Lean-Kanban/Métricas, U3 Prácticas continuas, U4 Aseguramiento de Calidad completo). Base: las presentaciones de la cátedra 02, 09 (×3), TDD, 10, 11, 12, 13 y 14, leídas completas — las que son solo imágenes (Agile Testing, tabla de desperdicios Lean, visiones y círculos de calidad, severidad/prioridad, ambientes) se leyeron renderizando las diapositivas. Se contrastó con el resumen de compañero `fuentes/resumenes_parcial2/Resumen ICW 2.docx`; lo que solo aparece ahí y no pude confirmar está en color naranja.

Temas sin presentación de la cátedra, resueltos con fuentes externas:

- **Nexus**: *Nexus Guide* (scrum.org) + resumen del compañero, que coincide con la guía.
- **Prácticas continuas (CI/CD)**: Fowler, Humble & Farley; la cátedra solo las nombra.
- **SPICE e ISO 9001/90003**: la cátedra nombra SPICE e ISO 25000; el detalle de niveles de SPICE y de la certificación ISO es externo.
- **Auditorías al Grupo de Calidad**: no hay material de la cátedra; queda como interpretación marcada en naranja.
- **SPI, CPI, effort variance y failure rate** (§4.5): no aparecen en las presentaciones; se tomaron de Earned Value Management.

Fuentes externas consultadas: Online Nexus Guide — scrum.org · Nexus Guide (mirror nkdagility) · ISO/IEC 15504 — Wikipedia · SPICE — TechTarget · ISO/IEC/IEEE 90003:2018 — SCC · CI/CD Tools Primer — GitLab Handbook · Continuous Delivery vs Deployment — BrowserStack · CI vs CD vs CDeployment (cita a Fowler) — itwriting. Libros de la bibliografía de la cátedra usados para definiciones: Myers (*The Art of Software Testing*), Sommerville (cap. 24), Pressman (cap. 14), Poppendieck, Kniberg & Skarin, Crispin & Gregory.

Nexus

> **U2 — Gestión Lean Agile de Productos de Software** · Sección §1 del resumen completo. Índice y convención de fuentes en 00-indice. Supone conocido Scrum (1er parcial: Scrum).

Cómo leer este archivo: se lee de corrido, como una sola explicación. Las listas aparecen una sola vez dentro del texto. Al final hay una **referencia rápida** con las listas en limpio y preguntas de repaso.

Ojo con la fuente: Nexus **no tiene presentación propia** en la carpeta de la cátedra. La base es el resumen del compañero, contrastado con la *Nexus Guide* oficial (scrum.org), y los dos coinciden.

Ideas que se repiten en otros temas y tienen nota propia:

- Mejora Continua: la Nexus Sprint Retrospective es la versión Nexus de la misma idea que aparece en Lean, Kanban, IDEAL y CMMI 5.
- Testing Temprano: el NIT empuja la integración continua para que los problemas de integración aparezcan cada día y no al final del Sprint (ver 05-practicas-continuas).

El ejemplo que recorre el archivo: Crovato crece y lo trabajan **tres equipos**: Tienda, Stock y Pagos. El ítem que los cruza a todos es "**sets con descuento en la oferta de la semana**".

La idea del tema: un solo problema nuevo, la integración

Scrum limita el tamaño del equipo (la Scrum Guide 2020 habla de "10 personas o menos"). Cuando un producto es demasiado grande para un solo equipo, aparecen varios equipos Scrum trabajando sobre **el mismo producto**. Con ellos aparece **un problema que Scrum no resuelve**: las **dependencias entre equipos** y la **integración** del trabajo de todos en un único incremento [Fuente: Externa — *The Scrum Guide* 2020; coincide con el resumen del compañero].

Frameworks que existen para esto: **Nexus**, **Scrum@Scale**, **LeSS** (Large-Scale Scrum) y **SAFe** (Scaled Agile Framework) [Fuente: resumen del compañero; confirmado como frameworks reconocidos — Externa].

Qué es Nexus. Es un framework creado por Ken Schwaber (scrum.org). Está compuesto por roles, eventos, artefactos y reglas que vinculan el trabajo de **aproximadamente 3 a 9 equipos Scrum**. Esos equipos trabajan sobre **un único Product Backlog**, con **un único Product Owner**, para construir **un Incremento Integrado** "Terminado" al menos una vez por Sprint [Fuente:

Externa — *Nexus Guide* 2021, scrum.org]. Es “Scrum de Scrums” con una pieza extra: un equipo dedicado a que la integración ocurra.

Todo Nexus sale de esa idea: es Scrum, más lo mínimo necesario para que varios equipos entreguen **una sola cosa integrada**. Por eso todo lo que agrega viene de a “uno”: un Product Backlog, un PO, un Incremento Integrado, una Review. El resto del archivo responde, en orden:

1. ¿De dónde salen las dependencias entre equipos? → requerimientos, dominio, artefactos.
2. ¿Quién se ocupa de la integración? → el Nexus Integration Team (NIT).
3. ¿Qué cambia en cada evento del Sprint? → los eventos Nexus.
4. ¿Dónde queda escrito el compromiso común? → artefactos y compromisos.

> **¿Por qué en Nexus hay un solo Product Owner y un solo Product Backlog, aunque haya 6 equipos?**
> Porque hay un solo producto. Si cada equipo tuviera su PO o su backlog, cada uno priorizaría distinto y el producto “tiraría para dos lados”. Lo que se multiplica son los equipos, no el producto.

---

1. De dónde salen las dependencias

Nexus existe para lidiar con la complejidad de varios equipos sobre un mismo Product Backlog [Fuente: resumen del compañero; coincide con la Nexus Guide]. Dos equipos se pisan cuando **comparten algo**, y pueden compartir tres cosas:

- **Lo que hay que hacer** → **requerimientos**: el alcance de dos ítems puede superponerse, y cómo implementa uno puede afectar al otro.
- **Lo que hay que saber** → **conocimiento del dominio**: conviene mapear partes del negocio o del sistema a equipos concretos, para minimizar las interrupciones entre ellos durante el Sprint.
- **Con qué se hace** → **artefactos de software y prueba**: dos equipos tocando el mismo código o los mismos tests. Este tercero no está en el resumen del compañero; lo agrega la Nexus Guide.

En Crovato llega el ítem “sets con descuento en la oferta de la semana”:

- **Requerimientos.** Tienda tiene que mostrar el precio rebajado y Pagos tiene que cobrarlo. Los dos tocan **el mismo requerimiento**, y si Tienda redondea distinto que Pagos, el cliente ve un precio y paga otro.
- **Dominio.** Solo el equipo de Stock entiende bien cómo se arma un set a partir de los perfumes sueltos. Sin ese **conocimiento**, Tienda tiene que interrumpirlos a cada rato.
- **Artefactos.** Al final los tres terminan editando `precios.ts`: **el mismo artefacto**, con conflictos de merge y tests que se rompen entre sí.

> **Dos equipos editan el mismo archivo de configuración del pipeline. ¿Qué fuente de dependencia es?**
> Artefactos de software y prueba: comparten la herramienta, no el requerimiento ni el conocimiento.

> **Cada vez que el equipo de Tienda toca algo de sets, tiene que preguntarle al equipo de Stock cómo funcionan. ¿Qué dependencia es y qué sugiere Nexus?**
> Conocimiento del dominio. Nexus sugiere mapear esa parte del negocio a un equipo concreto (asignar los ítems de sets a quien conoce el dominio) para minimizar las interrupciones.

---

2. Quién se ocupa de la integración: el Nexus Integration Team

Nexus suma un solo problema a Scrum, la integración, y crea un grupo responsable de él. El **Nexus Integration Team (NIT)** es responsable de que se produzca un **Incremento Integrado** al menos una vez por Sprint [Fuente: Externa — Nexus Guide; resumen del compañero].

Cada parte del problema la cubre alguien distinto, y los tres juntos forman el NIT. El NIT no es un rol extra, sino el grupo que los contiene:

- **¿Quién decide qué se integra y en qué orden? → el Product Owner.** Es uno solo para todo el Nexus, porque hay un solo producto. Maximiza el valor del producto integrado y gestiona el Product Backlog único.
- **¿Quién cuida cómo se trabaja en conjunto? → el Scrum Master del NIT.** Asegura que el framework Nexus se entienda y se aplique como dice la Nexus Guide. Puede ser también Scrum Master de uno o más equipos.
- **¿Quién se asegura de que la integración efectivamente pase? → uno o más miembros del NIT.** Suelen ser también miembros de los equipos Scrum.

Lo que el NIT no hace: no hace el trabajo de integración “por” los equipos. Los ayuda a adoptar herramientas y prácticas para integrar (por ejemplo, integración continua: ver 05-practicas-continuas) y aborda las restricciones técnicas y no técnicas que impiden entregar un incremento integrado.

En Crovato:

- **PO.** Vos seguís siendo el único PO. No puede haber un PO de Tienda y otro de Pagos: si uno prioriza “sets en oferta” y el otro “pagos con QR”, el producto tira para dos lados.
- **Scrum Master del NIT.** Elegís a una persona que conoce bien Scrum, para que la Nexus Daily no se convierta en una reunión de estado eterna.

- **Miembro del NIT.** Sumás a la desarrolladora que mejor conoce el pipeline. Ella no integra el código **por** los equipos: les arma la integración continua para que cada equipo integre solo.

Los tres forman el NIT. Si al final del Sprint no hay un incremento integrado que funcione, la responsabilidad es de ese grupo.

> **¿El NIT hace el trabajo de integración en lugar de los equipos?**
> No. Los ayuda a adoptar herramientas y prácticas (por ejemplo, CI) y resuelve restricciones, pero integrar lo hacen los equipos.

> **¿Por qué Nexus necesita un NIT si cada equipo ya tiene su Scrum Master?**
> Porque el problema que agrega escalar no está dentro de cada equipo, sino entre equipos. Cada SM ve solo su equipo; el NIT responde por el incremento integrado.

---

3. Qué cambia en cada evento

Nexus **agrega o extiende** los eventos de Scrum. Su duración se guía por la de los eventos equivalentes de la Scrum Guide [Fuente: Externa — Nexus Guide]. El compañero agrega que tienen un bloque de tiempo adicional al de sus eventos de Scrum correspondientes.

Es el mismo ciclo de Scrum. La pregunta que ordena los eventos es: en cada momento, ¿qué cambia cuando hay varios equipos?

Momento	Qué hace falta coordinar	Evento Nexus	Qué agrega respecto de Scrum
Todo el período	que todos trabajen en el mismo tiempo	**Nexus Sprint**	Todos los equipos trabajan en el mismo Sprint y producen **un** Incremento Integrado.
Antes de planificar	qué equipo hace qué y dónde se pisan	**Refinamiento entre equipos** (Cross-Team Refinement)	Es **continuo**. Tiene doble propósito: prever **qué equipo** hará cada ítem, e **identificar dependencias** entre equipos para eliminarlas o minimizarlas.

Momento	Qué hace falta coordinar	Evento Nexus	Qué agrega respecto de Scrum
Al empezar	un objetivo común	**Nexus Sprint Planning**	Representantes de cada equipo más el PO coordinan el Sprint. Salida: el **Nexus Sprint Goal**, un Sprint Goal por equipo, el **Nexus Sprint Backlog** y el Sprint Backlog de cada equipo.
Durante	lo que rompe la integración	**Nexus Daily Scrum**	Asisten **representantes** de cada equipo. Se inspeccionan los **problemas de integración** y el progreso hacia el Nexus Sprint Goal. Las Daily de cada equipo la complementan.
Al terminar	mostrar una sola cosa	**Nexus Sprint Review**	**Reemplaza** a las Reviews individuales: se muestra el Incremento Integrado a los stakeholders.
Después	mejorar el conjunto	**Nexus Sprint Retrospective**	Mejora de todo el Nexus. Las retros de cada equipo la complementan "de abajo hacia arriba". Es Mejora Continua.

Detalles del resumen del compañero (coinciden con la Nexus Guide):

- La Daily de cada equipo arma el plan del día centrado en los problemas de integración que surgieron en la Nexus Daily. Además, los equipos se comunican durante el día, no solo en esa reunión.
- La Nexus Retrospective inspecciona individuos, equipos, interacciones, procesos, herramientas y la Definición de Terminado.

En Crovato, durante una semana de Sprint:

1. **Lunes, antes del Sprint.** Un representante de Tienda, uno de Stock y uno de Pagos miran juntos el backlog. Descubren que "sets en oferta" necesita trabajo de los tres, lo reparten y marcan la dependencia (**refinamiento**).
2. **Lunes, al empezar.** Acuerdan "esta semana se puede comprar un set en oferta" como objetivo común, y cada equipo define su parte (**planning**).
3. **Miércoles.** En la reunión de representantes, Pagos avisa que el endpoint del precio cambió y rompió el build de Tienda (**Nexus Daily**).

4. **Viernes.** No hay tres demos: hay una sola, en la que un cliente compra un set en oferta de punta a punta (**review**).
5. **Después.** Hablan de por qué el build se rompió el miércoles (**retro**).

Todo pasó en la misma semana para los tres (**Nexus Sprint**). Para reconstruirlo, recorré la semana: antes, al empezar, durante, al terminar, después.

Otro ejemplo, fuera de Crovato: una app bancaria con 5 equipos (pagos, préstamos, onboarding, notificaciones, plataforma). Si “transferir con QR” necesita algo de pagos y algo de notificaciones, el refinamiento entre equipos lo detecta antes del Sprint y lo asigna para minimizar la dependencia. La Nexus Daily trae a la mesa “el endpoint de notificaciones rompió el build integrado”. Un equipo Scrum solo nunca necesita ese evento.

> **¿Qué evento de Scrum deja de existir por equipo en Nexus?**
> La Sprint Review: la reemplaza la Nexus Sprint Review, porque al stakeholder se le muestra el incremento integrado.

> **¿Quiénes van a la Nexus Daily Scrum?**
> Representantes de cada equipo. Se enfoca en los problemas de integración; cada equipo igual hace su Daily.

> **A mitad del Sprint, un representante avisa en la Nexus Daily que su equipo cambió una interfaz y otro equipo quedó bloqueado. ¿Qué se espera que pase?**
> Que los equipos involucrados ajusten sus planes: en su Daily, cada uno arma el plan del día centrado en ese problema de integración, y conversan entre ellos durante el día. El NIT facilita esa conversación; no resuelve la integración por ellos.

---

4. Dónde queda escrito el compromiso común: artefactos

Cada artefacto responde a un horizonte de tiempo, y su compromiso es la meta de ese horizonte [Fuente: Externa — Nexus Guide; resumen del compañero]:

- **Todo el producto, a largo plazo** → **Product Backlog** (uno solo). Su compromiso es el **Product Goal**.
- **Este Sprint, para todos los equipos** → **Nexus Sprint Backlog.** Junta el Nexus Sprint Goal y los ítems de los Sprint Backlogs de cada equipo. Sirve para **hacer visibles las dependencias** y el flujo durante el Sprint. Su compromiso es el **Nexus Sprint Goal**, que es la suma de los Sprint Goals de los equipos.
- **Lo que ya está hecho** → **Integrated Increment.** Es la suma de todo el trabajo integrado. Su compromiso es la **Definition of Done**. La DoD la define el NIT; cada equipo puede usar una

más estricta, nunca una más laxa.

El compañero agrega, de la Nexus Guide: el incremento está terminado solo cuando está **integrado, es de valor y es utilizable**; y puede entregarse antes de la Nexus Sprint Review, aunque ahí es donde se inspecciona.

En Crovato:

- **Product Backlog.** El Product Goal es “que la mayoría de las ventas sean online”, y el Product Backlog es la lista de todo lo que podría acercarte a eso.
- **Nexus Sprint Backlog.** Para esta semana, el Nexus Sprint Goal es “se puede comprar un set en oferta”. El Nexus Sprint Backlog junta los ítems de Tienda, Stock y Pagos en un solo lugar, justamente para que se vea que el ítem de Tienda espera al de Pagos.
- **Integrated Increment.** El viernes, lo que se integró cuenta como incremento solo si cumple la DoD que definió el NIT; por ejemplo, “pasa todos los tests de integración”. El equipo de Pagos puede exigirse más (“además, revisión de seguridad”), pero nunca menos.

> **Un equipo quiere una DoD más laxa porque “su parte es simple”. ¿Puede?**
> No. Puede ser más estricta que la del NIT, nunca más laxa.

> **Al final del Sprint, el incremento integrado no pasa las pruebas de integración. Alguien propone relajar la DoD para poder entregar lo que sí anda. ¿Es correcto según Nexus?**
> No. El incremento solo cuenta si cumple la DoD. Lo alineado con Nexus es reforzar la integración (CI y pruebas automatizadas) y llevar el problema a la Nexus Sprint Retrospective.

Referencia rápida (las listas en limpio)

No hay contenido nuevo acá: son las mismas listas de arriba, sin explicación.

- **Frameworks para escalar**: Nexus · Scrum@Scale · LeSS · SAFe.
- **Nexus**: 3 a 9 equipos · un Product Backlog · un PO · un Incremento Integrado por Sprint.
- **Fuentes de dependencia**: requerimientos · conocimiento del dominio · artefactos de software y prueba.
- **NIT**: Product Owner · Scrum Master · uno o más miembros del NIT.
- **Eventos**: Nexus Sprint · Refinamiento entre equipos · Nexus Sprint Planning · Nexus Daily Scrum · Nexus Sprint Review (reemplaza las Reviews) · Nexus Sprint Retrospective.
- **Artefactos → compromisos**: Product Backlog → Product Goal · Nexus Sprint Backlog → Nexus Sprint Goal · Integrated Increment → Definition of Done.

Preguntas de repaso

1. **¿Por qué Nexus necesita un Nexus Integration Team si cada equipo ya tiene su Scrum Master?** — Porque el problema que agrega escalar no está dentro de cada equipo sino *entre* equipos: dependencias e integración. El NIT es responsable de que haya un Incremento Integrado cada Sprint; los SM de cada equipo solo ven su equipo.
2. **¿Qué evento de Scrum desaparece en Nexus y por qué?** — Las Sprint Review individuales: las reemplaza la Nexus Sprint Review, porque lo que tiene valor para el stakeholder es el incremento integrado, no partes sueltas.
3. **¿Para qué sirve el refinamiento entre equipos?** — Para prever qué equipo hará cada ítem e identificar las dependencias entre equipos, para eliminarlas o minimizarlas antes del Sprint. Es continuo.

Lean

> **U2 — Gestión Lean Agile de Productos de Software** · Sección §2 del resumen completo. Índice y convención de fuentes en 00-indice. Kanban, que es la forma concreta de aplicar Lean, está en 03-kanban.

Cómo leer este archivo: se lee de corrido. Los 7 principios y los 7 + 1 desperdicios aparecen una sola vez, dentro del texto. Al final hay una referencia rápida con las listas en limpio y preguntas de práctica.

Las ideas que se repiten en otros temas tienen nota propia, y acá solo se nombran:

- Limitar el WIP: el desperdicio de cambio de tareas (multitasking) y el de trabajo a medias se atacan limitando el trabajo en curso. Es la práctica central de 03-kanban.
- Calidad Embebida: el principio "embeber la integridad conceptual" es la misma idea que "la calidad no se inyecta" de 06-calidad.
- Mejora Continua: Lean, Kanban, la Retrospectiva de Nexus, IDEAL y CMMI nivel 5 la usan con palabras distintas.
- Error y Defecto: el desperdicio "defectos" es la definición de defecto de la U4.

El ejemplo que recorre el archivo: agregar "pagar en cuotas" a la tienda de Crovato, gastando lo menos posible.

La idea del tema

Qué es. Lean es una filosofía de gestión **anterior al agilismo**, nacida en Japón con **Toyota** (Sistema de Producción Toyota, años '40-'50). Busca **maximizar el valor entregado al cliente con el mínimo consumo de recursos**. Todo esfuerzo que no agrega valor desde la perspectiva del cliente es **desperdicio** [Fuente: Apuntes — Gestión de Productos: "El pensamiento Lean define la creación de valor como proveer beneficios reales a los clientes. Cualquier otra cosa es desperdicio"; resumen del compañero].

Se apoya en **Just in Time**: producir o entregar lo justo, en la cantidad justa, justo antes de que se necesite ("analizá cuando lo necesites, no antes"). La traducción a software la hicieron **Mary y Tom Poppendieck** (*Lean Software Development: An Agile Toolkit*, 2003), que está en la bibliografía de la cátedra [Fuente: Externa — Poppendieck, en `fuentes/Lean y Kanban`].

Todo el tema sale de una sola pregunta: ¿esto le agrega valor al cliente? Si no, es desperdicio. El archivo la responde en orden:

1. ¿Cómo se trabaja para no desperdiciar? → los 7 principios.
2. ¿Cómo se reconoce el desperdicio? → los 7 + 1 desperdicios, que son el principio 1 en detalle.
3. ¿Todo desperdicio se puede eliminar? → Tipo 1 y Tipo 2.
4. ¿Qué tiene que ver con ágil? → la relación Lean–Ágil.

> **Tres frases para una misma idea: no decidir antes de tiempo**
>
> - Principio 4: **diferir compromisos** hasta el último momento responsable [Fuente: Apuntes].
> - "**Evitar decisiones prematuras**": el resumen del compañero lo pone como parte del principio 2 (amplificar el aprendizaje).
> - El desperdicio **características extra** "no solo tiene que ver con las funcionalidades que se desarrollan de más, sino también con **especificar anticipadamente**" (resumen del compañero).
>
> Las tres dicen lo mismo: si decidís o especificás antes de tener la información, lo más probable es que haya que rehacerlo. Por eso la "planificación detallada a largo plazo" **genera** desperdicio.

---

1. Cómo se trabaja para no desperdiciar: los 7 principios

Los **7 principios Lean** [Fuente: Apuntes — 10 Lean Kanban] resuelven cuatro cosas.

¿Qué no hay que hacer?

- (1) **Eliminar desperdicios**. Es el más importante: todo paso que no contribuye a generar valor es desperdicio. En software, el desperdicio típico es la **funcionalidad extra** y el **trabajo parcialmente hecho** (el equivalente al inventario). Se conecta con los principios ágiles "software funcionando es la medida de progreso" y "simplicidad: maximizar el trabajo no realizado" [resumen del compañero; coherente con Poppendieck]. Estos dos ejemplos son los dos primeros desperdicios de la sección 2: no son ideas aparte.

¿Cómo se hace bien lo que sí hay que hacer?

- (2) **Amplificar el aprendizaje**. Tratar el desarrollo como un proceso de descubrimiento: ciclos cortos con feedback rápido, compartir conocimiento (convertir lo implícito en explícito) y dejar que la solución evolucione. El compañero lo titula "Amplificar el aprendizaje – (Mejora continua)". Es Mejora Continua.
- (3) **Embeber la integridad conceptual**. Construir calidad **desde el principio**, no inspeccionarla después: "si buscás calidad, no inspecciones después de los hechos; si no es posible, inspeccioná después de pasos pequeños". Incluye pensar la arquitectura

(coherencia, consistencia, mantenibilidad) desde temprano. Es Calidad Embebida, y el "inspeccioná después de pasos pequeños" es Testing Temprano.

¿Cuándo se decide y cuándo se entrega? Estos dos son opuestos a propósito:

- (4) **Diferir compromisos** hasta el **último momento responsable**. Decidir cuando ya tenés la mayor información posible, pero antes de que sea tarde. Decidir antes es decidir con información parcial, y eso da más incertidumbre. En ágil se ve en el Product Backlog, que nunca está 100% definido.
- (7) **Entregar lo antes posible**. Ciclos cortos, incrementos pequeños de valor, llegar rápido al mercado. Tiene relación directa con el MVP y las entregas frecuentes.

¿Quién decide y con qué mirada?

- (5) **Dar poder al equipo**. Delegar las decisiones en quien hace el trabajo; respeto por las personas; formar líderes.
- (6) **Ver el todo**. Visión holística: optimizar el flujo completo de valor, no cada área por separado. Un área "eficiente" que genera cuellos de botella aguas abajo no ayuda.

En Crovato, tenés que agregar las cuotas a la tienda:

- **Qué no hacer.** Nadie pidió 12 cuotas, así que no las programás (**eliminar desperdicios**).
- **Cómo hacerlo bien.** No sabés bien cómo se comporta la API de MercadoPago, así que hacés una prueba chica primero y aprendés de lo que devuelve (**amplificar el aprendizaje**). Desde el primer día escribís el test de `precios.ts`, en vez de revisar todo al final (**integridad conceptual**).
- **Cuándo.** No decidís hoy si vas a ofrecer 18 cuotas el año que viene; lo decidís cuando tengas datos de ventas (**diferir compromisos**). Pero las cuotas de 1, 3 y 6 las publicás apenas funcionan, sin esperar a tener "todo" (**entregar lo antes posible**).
- **Quién y con qué mirada.** Si alguien te ayuda, decide él cómo implementar su parte (**dar poder al equipo**). Y no te sirve que la tienda cobre rápido si después `controlstock` tarda dos días en descontar el stock: mirás el flujo completo, del clic a la entrega (**ver el todo**).

> **"Decidir en el último momento responsable" y "entregar lo antes posible", ¿se contradicen?**
> No. Uno habla de **decisiones** (esperar a tener información) y el otro de **valor** (que llegue cuanto antes al cliente). Se complementan: no te atás a decisiones prematuras y aun así entregás seguido.

> **Un equipo revisa la calidad recién en una fase de QA al final del proyecto. ¿Qué principio viola?**
> Embeber la integridad conceptual: la calidad se construye desde el principio, no se inspecciona después.

> **El área de desarrollo duplicó su velocidad, pero las features se acumulan esperando a testing y el cliente no recibe nada antes. ¿Qué principio se descuidó?**
> Ver el todo: se optimizó una parte y se generó un cuello de botella aguas abajo. Lo que importa es el flujo completo hasta el cliente.

---

2. Cómo se reconoce el desperdicio: los 7 + 1

La cátedra traduce los desperdicios de manufactura (Ohno/Toyota) a servicios basados en conocimiento (Poppendieck), y suma un octavo: el **talento no utilizado** [Fuente: Apuntes — tabla de la cátedra, verificada visualmente].

Desperdicio en manufactura	Desperdicio en software (conocimiento)	Ejemplo concreto
Inventario	**Trabajo parcialmente terminado**	10 historias “en curso” y ninguna terminada; código sin integrar en una rama vieja
Sobreproducción	**Características extra**	Funcionalidades que nadie usa (la cátedra cita que el **45%** de las funcionalidades típicas **nunca** se usan)
Proceso extra	**Proceso extra**	Documentación o aprobaciones que nadie lee
Transporte	**Cambio de tareas / Transferencia de conocimiento**	Handoffs analista → dev → tester donde se pierde contexto
Esperas	**Esperas / Demoras**	Esperar la aprobación de un gerente o el ambiente de prueba
Movimiento	**Cambio de contexto (Seteo)**	Multitasking: cada cambio de tarea tiene un costo de “volver a entrar”
Defectos	**Defectos**	Bugs que pasan a etapas posteriores y generan retrabajo
— (+1)	**Talento no utilizado**	No aprovechar experiencia/creatividad de la gente

Para no memorizar la tabla, preguntate **de qué forma se desperdició el trabajo**. Hay cuatro:

- **Se hizo algo que no hacía falta** → **características extra** (producto que nadie usa) y **proceso extra** (pasos que nadie usa).
- **Se hizo pero no se terminó** → **trabajo parcialmente terminado**, el inventario del software.
- **Se perdió tiempo entre medio** → **esperas**, **transferencias de conocimiento** (pasar el trabajo de mano en mano) y **cambio de contexto** (saltar entre tareas).

- **Se hizo mal, o con la gente equivocada** → **defectos** (hay que rehacerlo; es la definición de defecto de Error y Defecto) y **talento no utilizado** (+1: la persona que sabía no fue escuchada).

Dos desperdicios que se atacan con lo mismo. El trabajo parcialmente terminado y el cambio de contexto (multitasking) tienen la misma cura: **limitar el trabajo en curso**. Si nadie puede empezar algo nuevo hasta terminar lo que tiene, no se acumula trabajo a medias ni se salta entre tareas. Es Limitar el WIP, la práctica central de 03-kanban. El resumen del compañero lo dice así: "en Kanban se hace uso de limitar el WIP, porque busca reducir el multitasking", y relaciona el trabajo a medias con el cambio de tareas.

Otra versión de la lista. La diapositiva "Gastos en producción Lean" de la presentación 10 nombra los siete de manufactura como producción en exceso, stock, pasos extra en el proceso, **búsqueda de información**, defectos, esperas y transportes. El resumen del compañero sigue esa versión y relaciona la búsqueda de información con la falta de SCM. No es otro desperdicio: es la misma tabla con otros nombres.

En Crovato, es la semana de lanzamiento de las cuotas:

- **Lo que no hacía falta.** Le pedís a alguien que agregue también "pago en 12 cuotas" por si acaso, aunque ningún cliente lo pidió (**característica extra**). Además, le exigís un documento de diseño de diez páginas que nadie va a leer (**proceso extra**).
- **Lo que no se termina.** La rama de "sets en oferta" está a medio hacer desde hace tres semanas, sin integrarse (**trabajo parcialmente terminado**).
- **El tiempo perdido entre medio.** El desarrollador espera dos días a que vos apruebes el texto del checkout (**espera**). Después se lo pasa a otra persona para que lo pruebe, y le explica todo de nuevo (**transferencia**). En el medio lo interrumpen cada hora con consultas de stock, así que nunca se concentra (**cambio de contexto**).
- **Lo mal hecho o mal aprovechado.** Cuando sale, cobra mal en 6 cuotas y hay que rehacerlo (**defecto**). Y tu amigo que trabajó en una fintech te había avisado cómo se calculaban las cuotas, pero nadie le preguntó (**talento no utilizado**).

> **Un equipo tiene 15 historias "en curso" y ninguna terminada. ¿Qué desperdicio es y con qué práctica se ataca?**
> Trabajo parcialmente terminado (el equivalente al inventario). Se ataca limitando el WIP.

> **¿Cuál es el desperdicio "+1" y por qué la cátedra lo agrega?**
> Talento no utilizado. No viene de los 7 de manufactura: en el trabajo de conocimiento, no aprovechar la experiencia y la creatividad de la gente es una pérdida directa.

> **Un desarrollador trabaja en cuatro historias a la vez y ninguna avanza. ¿Qué desperdicios hay?**
> Cambio de contexto (cada salto tiene un costo de volver a entrar) y trabajo parcialmente terminado. Los dos se atacan limitando el WIP.

---

3. ¿Todo desperdicio se puede eliminar? Tipo 1 y Tipo 2

No. La cátedra distingue **dos tipos de desperdicio** [Fuente: Apuntes]. La pregunta que los separa es: si mañana lo elimino, ¿pasa algo malo?

- **Sí, pasa algo** (una multa, un riesgo legal) → **Tipo 1, necesario**. No agrega valor, pero hoy no se puede evitar; por ejemplo, ciertos controles regulatorios. Se **minimiza**.
- **No pasa nada** → **Tipo 2, puro desperdicio**. Se **evita**.

En Crovato tenés que emitir factura por cada venta. Al cliente no le agrega valor, pero si dejás de hacerlo te multan: es **Tipo 1**, así que lo automatizás para que te lleve un minuto en vez de diez. En cambio, imprimir cada pedido para archivarlo en una carpeta que nadie abre no tiene ninguna consecuencia si lo dejás de hacer: es **Tipo 2**, así que directamente dejás de hacerlo.

> **Las pruebas de regresión no agregan valor visible al cliente. ¿Son desperdicio Tipo 2?**
> No. Si las eliminás, aparecen defectos (otro desperdicio). Como mínimo serían Tipo 1: se minimiza su costo, por ejemplo automatizándolas.

---

4. Qué tiene que ver con ágil

Relación Lean–Ágil: Lean es previo al Manifiesto Ágil, y varios principios ágiles heredan de él [resumen del compañero; coherente con Poppendieck].

Lo que comparten: los dos están orientados al cliente y al valor, valoran los equipos motivados y autoorganizados, la adaptación al cambio y la mejora continua.

Lo que es solo de Lean:

- **eliminar desperdicios** y **optimizar (ver) el todo** no están explícitos en el Manifiesto Ágil;
- **minimizar el WIP** es de Lean/Kanban, no del Manifiesto.

Las otras ideas de Lean sí tienen un equivalente en el Manifiesto:

- *entregar lo antes posible* ↔ entregas tempranas y frecuentes;
- *dar poder al equipo* ↔ equipos autoorganizados;

- *diferir compromisos* ↔ responder al cambio más que seguir un plan.

Esta correspondencia es una interpretación propia.

> **¿“Minimizar el trabajo en curso” es algo que comparten Lean y el Manifiesto Ágil?**
> No. Limitar el WIP es de Lean/Kanban; el Manifiesto no lo dice. Es la trampa de “correcta, pero de otro enfoque”.

Referencia rápida (las listas en limpio)

No hay contenido nuevo acá: son las mismas listas de arriba, sin explicación.

- **7 principios**: eliminar desperdicios · amplificar el aprendizaje · embeber la integridad conceptual · diferir compromisos · dar poder al equipo · ver el todo · entregar lo antes posible.
- **7 + 1 desperdicios**: trabajo parcialmente terminado · características extra · proceso extra · cambio de tareas / transferencia de conocimiento · esperas / demoras · cambio de contexto (seteo) · defectos · (+1) talento no utilizado.
- **Tipos**: Tipo 1, necesario → minimizar · Tipo 2, puro desperdicio → evitar.
- **Solo de Lean, no del Manifiesto**: eliminar desperdicios · optimizar el todo · minimizar el WIP.

Preguntas de repaso

1. **¿Qué es desperdicio para Lean?** — Todo lo que no agrega valor desde la perspectiva del cliente.
2. **¿Por qué limitar el WIP reduce desperdicio?** — Porque ataca dos desperdicios a la vez: el trabajo parcialmente terminado y el cambio de contexto (multitasking). Ver Limitar el WIP.
3. **¿Por qué la planificación detallada a largo plazo es desperdicio?** — Porque decide con información parcial (contradice diferir compromisos) y especifica anticipadamente cosas que probablemente cambien (características extra).

Kanban

> **U2 — Gestión Lean Agile de Productos de Software** · Sección §3 del resumen completo. Índice y convención de fuentes en 00-indice. Kanban es la forma más difundida de aplicar 02-lean en software; sus métricas (lead time, cycle time, touch time) están en 04-metricas.

Cómo leer este archivo: se lee de corrido. Las listas de la cátedra aparecen una sola vez dentro del texto. El **diseño de un tablero a partir de un contexto** tiene su propia guía, guia-tablero-kanban; la sección 6 la presenta. Al final hay una referencia rápida y preguntas de repaso.

Las ideas que se repiten en otros temas tienen nota propia, y acá solo se nombran:

- Limitar el WIP: “al 100% de capacidad, rendimiento mínimo”, evitar el multitasking (“dejar de empezar, empezar a terminar” es una frase conocida de la comunidad Kanban, no de la cátedra). Lean dice lo mismo con el desperdicio “cambio de tareas” y el “trabajo parcialmente hecho”.
- Mejora Continua: el cambio evolutivo de Kanban es la misma idea que la mejora continua de Lean, la retrospectiva de Scrum/Nexus, IDEAL y CMMI nivel 5 (con matices).

El ejemplo que recorre el archivo: el tablero de pedidos y de desarrollo de la tienda de Crovato.

La idea del tema

Kanban es una forma de **ordenar el flujo de trabajo que ya existe y mejorarlo de a poco**, haciéndolo visible y poniéndole límites. Todo el tema sale de ahí. Responde, en orden:

1. ¿Qué es Kanban y qué lo guía? → definición, origen, lo que no es, los 9 valores.
2. ¿Cómo se introduce en una organización? → los principios (gestión del cambio y entrega de servicios).
3. ¿Cómo se trabaja con el tablero? → las 6 prácticas, con el WIP y el pull explicados una sola vez.
4. ¿Todos los trabajos se tratan igual? → las clases de servicio.
5. ¿En qué se diferencia de Scrum?
6. ¿Cómo se diseña un tablero para un contexto dado? → en una guía aparte, guia-tablero-kanban.

---

1. Qué es Kanban y qué lo guía

Definición de la cátedra: “un método para **definir, gestionar y mejorar servicios** que entregan **trabajo del conocimiento**, tales como servicios profesionales, trabajos o actividades en las que interviene la creatividad y el diseño, tanto de productos de software como físicos” [Fuente: Apuntes].

De dónde viene. *Kan-ban* (看板) significa **tarjeta de señal**. A fines de los ’40, Toyota estudió cómo los **supermercados** reponían stock **solo cuando el cliente se llevaba algo** (Just in Time) [Fuente: Apuntes]. Esa es la idea del **pull** que se ve en la sección 3: no se produce porque sí, se repone cuando hay una señal.

Lo que Kanban NO es: no es un proceso de desarrollo de software ni una metodología de gestión de proyectos. Es un **enfoque de gestión del cambio** que se aplica **sobre el proceso que ya tenés** y lo mejora de forma gradual [resumen del compañero; coincide con Anderson, *Kanban*, y con el principio de la cátedra “comenzar con lo que haces ahora”, que se explica en la sección 2]. Es la forma más difundida de aplicar Lean en software.

Los 9 valores [Fuente: Apuntes] son transparencia, equilibrio, colaboración, foco en el cliente, flujo, liderazgo, entendimiento, acuerdo y respeto. Un tablero Kanban funciona si se cumplen tres cosas, y cada valor responde a una:

- **¿Todos ven y entienden lo mismo?** → **transparencia** (está a la vista) y **entendimiento** (se comprende cómo funciona el sistema).
- **¿El trabajo avanza bien y para quien corresponde?** → **flujo** (que se mueva), **equilibrio** (entre lo que entra y lo que se puede hacer) y **foco en el cliente** (para quién se hace).
- **¿Las personas trabajan bien juntas?** → **colaboración**, **acuerdo**, **respeto** y **liderazgo**.

En Crovato, pegás el tablero de pedidos en la pared del local:

- Cualquiera ve qué pedido está trabado (**transparencia**), y al mes todos entienden por qué siempre se atasca en “empaquetado” (**entendimiento**).
- Querés que los pedidos salgan sin frenarse (**flujo**), pero no podés aceptar 50 pedidos por día si empaquetás 20 (**equilibrio**). Lo que importa es que el cliente reciba su perfume, no que vos estés ocupado (**foco en el cliente**).
- Cuando empaquetado se llena, quien atiende el chat se pone a ayudar (**colaboración**). Las reglas del tablero las decidieron entre todos (**acuerdo**). Nadie culpa a quien se atrasó (**respeto**). Y fue el chico de empaquetado, no vos, quien propuso dividir la columna en dos (**liderazgo** en cualquier nivel).

> **Un equipo dice “dejamos Scrum y adoptamos la metodología Kanban, con su proceso de desarrollo”. ¿Qué está mal?**
> Kanban no es una metodología ni un proceso de desarrollo: es un enfoque de gestión del cambio que se monta sobre el proceso que ya existe.

---

2. Cómo se introduce en una organización: los principios

Los **principios** [Fuente: Apuntes] vienen en dos grupos, que responden a dos preguntas distintas.

¿Cómo cambio una organización sin que se resista? → principios de **gestión del cambio**:

- (1) **Comenzar con lo que hacés ahora**: entender el proceso actual tal como es. No se arranca de cero.
- (2) **Acordar la búsqueda de mejora a través del cambio evolutivo**: no se cambia todo de golpe (Mejora Continua).
- (3) **Fomentar actos de liderazgo en todos los niveles**: el cambio no tiene que venir solo de arriba.
- Además, **respetar los roles, responsabilidades y puestos actuales**: no se le saca el puesto a nadie.

¿Cómo le sirvo bien al cliente? → principios de **entrega de servicios**:

- (1) Comprender y enfocarse en las **necesidades y expectativas del cliente**.
- (2) **Gestionar el trabajo, no a las personas**: dejar que se autoorganicen en torno a él.
- (3) Revisar periódicamente la **red de servicios y sus políticas**.

En Crovato, querés ordenar cómo se preparan los pedidos:

- **El cambio.** No inventás un proceso nuevo: dibujás en el tablero lo que ya hacen hoy (**comenzar con lo que hacés**). No cambiás todo el lunes; cada semana probás una mejora chica (**cambio evolutivo**). La persona que empaqueta sigue empaquetando y nadie pierde su lugar (**respetar roles**). Cuando ella propone un cambio, se prueba (**liderazgo en todos los niveles**).
- **El cliente.** Sabés que lo que más le importa es recibir rápido (**necesidades del cliente**). No le decís a cada uno qué hacer: el tablero muestra qué falta y cada uno toma (**gestionar el trabajo**). Cada mes revisás si las reglas del tablero siguen teniendo sentido (**revisar la red de servicios**).

> **Un gerente quiere implantar Kanban reemplazando todos los roles actuales por roles nuevos. ¿Qué principio viola?**
> Respetar los roles, responsabilidades y puestos actuales (y también comenzar con lo que se hace ahora).

> **El líder asigna cada mañana a cada persona qué tarjeta hacer. ¿Qué principio no se está respetando?**
> Gestionar el trabajo, no a las personas: el equipo se autoorganiza en torno al trabajo, y cada uno tira la tarjeta siguiente cuando tiene capacidad (pull).

---

3. Cómo se trabaja con el tablero: las 6 prácticas

Las **prácticas generales** [Fuente: Apuntes] siguen el orden en que mejorás algo que no conocés: primero mirás, después controlás, después acordás, después revisás y al final cambiás. No todas se aplican con la misma frecuencia: algunas se **deciden de vez en cuando** (fijar los límites, acordar las políticas) y otras se **respetan cada vez que alguien mueve una tarjeta** (no pasarse del límite, tirar en vez de empujar).

Mirar → (1) **Visualizar**. El tablero permite procesar mucha información rápido; todos ven la misma imagen (cooperación), y las decisiones son más rápidas y colaborativas.

Controlar → (2) **Limitar el WIP** y (3) **gestionar el flujo**.

- **Limitar el WIP** (*Work In Progress*) es fijar un **máximo explícito de ítems por columna o estado**. La idea clave de la cátedra es "**al 100% de capacidad se tiene un rendimiento mínimo**", como una autopista en hora pico: si todos están 100% ocupados no hay holgura y el flujo se traba. Los sistemas eficaces se centran **en el flujo del trabajo, no en tener ocupados a los trabajadores**. Además, "limitar el WIP **provoca conversación y mejora**": cuando una columna se llena, el equipo tiene que ayudar a destrabar en vez de empezar algo nuevo. Ojo con las dos caras: el **número** se fija al armar el tablero y se ajusta cada tanto (por ejemplo, si una columna vive vacía o vive llena); lo que pasa **en el día a día** es **respetarlo**, porque antes de tirar una tarjeta mirás si la columna tiene lugar. La idea completa, y su relación con Lean, está en Limitar el WIP.
- **Pull, no push**: es la forma en que el WIP funciona en la práctica. Nadie asigna trabajo a una persona; cuando alguien libera capacidad (su columna está por debajo del límite), **tira** (pull) la siguiente tarjeta. La tarjeta es justamente la **señal** de que hay capacidad [Fuente: Apuntes "Pull, no push!!!"; resumen del compañero].
- **Gestionar el flujo** es terminar el trabajo de la forma más fluida y predecible posible, a ritmo sostenido, y medir el flujo para predecir y gestionar expectativas. El foco está en el flujo "fin a fin", centrado en el cliente. Las métricas para hacerlo están en 04-metricas.

Acordar → (4) **Hacer explícitas las políticas**. Las políticas son acordadas entre todos (incluido el cliente), visibles, **pocas, sencillas, bien definidas, aplicables siempre y modificables** por quienes prestan el servicio.

Revisar → (5) **Implementar ciclos de feedback**: cadencias de reuniones, revisiones y releases.

Cambiar → (6) **Mejorar colaborativamente, evolucionar experimentalmente**. Los cambios son **experimentos** basados en el método científico, en entornos donde fallar es seguro: si la hipótesis se confirma, el cambio se mantiene; si no, se vuelve atrás fácilmente (Mejora Continua).

En Crovato, los pedidos se atrasan y no sabés por qué:

1. Los ponés en un tablero y lo ves: hay 12 pedidos en “empaquetado” (**visualizar**).
2. Decidís que no puede haber más de 4 ahí a la vez (**limitar el WIP**). Quien termina un pedido toma el siguiente, nadie se los reparte (**pull**). Empezás a medir cuántos días tarda cada pedido, para que salgan a un ritmo parejo (**gestionar el flujo**).
3. Escribís en el tablero “un pedido pasa a envío solo si tiene la factura adjunta” (**políticas explícitas**).
4. Cada viernes miran juntos los números de la semana (**feedback**).
5. Alguien propone: “probemos dos semanas armar los sets el día anterior”. Si los tiempos bajan, queda; si no, se vuelve atrás (**mejorar experimentalmente**).

> **¿Por qué “limitar el WIP” puede aumentar lo que se entrega, si deja gente sin tarea nueva?**
> Porque al 100% de capacidad el rendimiento es mínimo (la autopista en hora pico). Limitar el WIP evita el multitasking y obliga a terminar antes de empezar; quien queda libre ayuda a destrabar.

> **Un equipo tiene 20 historias “en desarrollo” y 0 terminadas. ¿Qué desperdicio Lean es y qué práctica Kanban lo ataca?**
> Trabajo parcialmente terminado (inventario). Se ataca limitando el WIP.

> **La columna “Testing” tiene WIP 3 y está llena; un desarrollador terminó su tarea. ¿Qué hace?**
> No empieza una tarea nueva de desarrollo (no puede empujar a Testing, que está lleno). Ayuda a destrabar Testing. Eso es lo que la cátedra llama “limitar el WIP provoca conversación y mejora”.

---

4. Distintos trabajos, distintas reglas: las clases de servicio

No todo trabajo tiene la misma urgencia, así que cada **clase de servicio** tiene su propia política y un color de tarjeta. La tabla son los ejemplos de políticas de la cátedra [Fuente: Apuntes]. **Esta es la única vez que se explica la regla de cada clase**: los tableros de guia-tablero-kanban solo la nombran.

Clase	Color	Política
Expreso (urgente)	Blanco	WIP = 1; los demás trabajos se ponen en espera; **puede exceder el límite de WIP**; **la capacidad no se reserva** (no se deja a nadie "libre por si acaso": cuando llega, el resto se pausa); de ser necesario se hace una entrega especial a producción.
Fecha fija	Rosa	Queda en cola hasta que convenga; respeta el WIP; se entrega en entregas programadas cuidando la fecha; si la fecha está en riesgo puede **promoverse a Expreso**.
Estándar	Amarillo	Respeta el WIP; se prioriza por valor de negocio; FIFO si no hay expresos/fecha fija; entregas programadas.
Intangible (ej. refactor, deuda técnica)	Verde	Igual que estándar en mecánica; valor de negocio no inmediato pero importante a futuro.

Una pregunta para ordenarlas: ¿qué pasa si este trabajo espera? Es una forma propia de ordenarlas, no una formulación de la cátedra; las reglas de cada clase sí son las de la tabla.

- **Si espera, pierdo ya mismo → Expreso**. Por eso rompe el WIP, pausa todo lo demás y puede salir en una entrega especial.
- **Si espera más allá de una fecha, pierdo; antes no → Fecha fija**. Por eso puede quedarse en cola hasta que convenga, y si la fecha corre riesgo se promueve a Expreso.
- **Si espera, pierdo de a poco → Estándar**. Se ordena por valor de negocio, FIFO, respetando el WIP.
- **Si espera, hoy no pierdo nada, pero mañana sí → Intangible**: refactor, deuda técnica, actualizaciones.

Ejemplo real: en un tablero de soporte, un caído de producción entra como **Expreso** (todos dejan lo suyo), un cambio legal con vigencia el 1° de enero es **Fecha fija**, una feature nueva es **Estándar**, y actualizar la versión de Next.js es **Intangible**.

En Crovato (`crovatofragrances`):

- Un sábado, la `tienda` deja de cobrar: cada minuto es una venta perdida, así que todos dejan lo que estaban haciendo (**Expreso**).
- La promo del Día del Padre tiene que estar publicada antes del domingo: el martes puede esperar, pero si el viernes no está lista, pasa a ser urgente (**Fecha fija**).
- La página de "sets armados a medida" suma ventas; cada semana que se atrasa es una semana sin esas ventas, pero nada se rompe (**Estándar**).
- Actualizar Next.js y las dependencias no cambia nada hoy, pero si lo dejás un año, el día que tengas que tocar algo va a ser un problema (**Intangible**).

> **¿La clase Expreso reserva capacidad del equipo para cuando llegue?**
> No. La capacidad no se reserva: cuando llega un Expreso, el resto del trabajo se pone en espera. Puede exceder el límite de WIP.

> **¿Qué pasa con un ítem de Fecha fija cuya fecha está en riesgo?**
> Puede promoverse a Expreso.

---

5. Kanban vs. Scrum

Síntesis útil para comparar [Fuente: Externa — Kniberg & Skarin, *Kanban vs Scrum*, en `fuentes/Lean y Kanban`]. La diferencia de fondo es una sola: Scrum organiza el trabajo en **cajas de tiempo** (Sprints); Kanban lo organiza como un **flujo continuo**. Todas las demás salen de ahí:

- **¿Dónde se limita el trabajo?** Scrum limita el WIP **por iteración** (lo que entra al Sprint); Kanban lo limita **por estado del flujo** (WIP por columna).
- **¿Qué se mide?** Scrum mide **velocidad** (cuánto entra en una caja); Kanban mide **lead/cycle time** (cuánto tarda en fluir).
- **¿Se puede cambiar la prioridad?** En Scrum el **Sprint Goal se protege**; en Kanban, como no hay caja, se pueden **cambiar prioridades en cualquier momento**.
- **¿Roles y eventos?** Scrum los **prescribe** para que la caja funcione; Kanban **no prescribe roles** y se monta sobre lo que ya hay (sección 2).

En Crovato: con Scrum, el lunes elegirías qué entra en las dos semanas y lo protegerías aunque aparezca algo nuevo; a fin de Sprint medirías cuántos puntos entraron. Con Kanban, cada vez que alguien se libera toma la tarjeta de arriba, podés reordenar la cola cuando quieras, y lo que medís es cuántos días tarda una tarjeta en llegar a "Publicado".

> **¿La “velocidad” sirve para expresar la capacidad de un sistema Kanban?**
> No: la velocidad es una métrica de Scrum. En Kanban la capacidad se expresa con el límite de WIP.

---

6. Cómo diseñar un tablero para un contexto

El diseño de un tablero tiene su propia guía: guia-tablero-kanban. Ahí están:

- el procedimiento paso a paso (los 7 pasos de la cátedra, con una tabla de señales “si el contexto dice X → en el tablero va Y”);
- errores típicos al armarlo;
- una plantilla y un control final.

Lo que este archivo aporta al diseño son las piezas que se usan en el tablero: columnas, WIP y pull (sección 3), y las clases de servicio con su regla (sección 4).

> **¿La asignación “60% features / 40% bugs” cuenta como política?**
> No. Es parte de los tipos de trabajo (capacidad por carril). Una política dice cómo se trabaja (quién decide, con qué criterio), no cuánto lugar tiene cada tipo.

> **¿Los tipos de trabajo son las columnas del tablero?**
> No. Las columnas son las etapas del proceso. Los tipos de trabajo son clases de ítem (requerimientos, defectos, mantenimiento, solicitudes) y se ven como carriles horizontales. Es el error más común al armar un tablero.

> **El contexto dice “la documentación es parte del DoD” y vos lo mencionás en un párrafo aparte. ¿Alcanza?**
> No. Tiene que verse en el tablero: tiene que haber una columna “Documentación” o un criterio de salida escrito sobre el tablero.

Referencia rápida (las listas en limpio)

No hay contenido nuevo acá: son las mismas listas de arriba, sin explicación.

- **9 valores**: transparencia · equilibrio · colaboración · foco en el cliente · flujo · liderazgo · entendimiento · acuerdo · respeto.

- **Principios de gestión del cambio**: comenzar con lo que hacés ahora · cambio evolutivo · liderazgo en todos los niveles · respetar roles, responsabilidades y puestos actuales.
- **Principios de entrega de servicios**: necesidades y expectativas del cliente · gestionar el trabajo, no a las personas · revisar la red de servicios y sus políticas.
- **6 prácticas**: visualizar · limitar el WIP · gestionar el flujo · políticas explícitas · ciclos de feedback · mejorar colaborativamente, evolucionar experimentalmente.
- **7 pasos para aplicar Kanban**: proceso · trabajo · límites de WIP · políticas de calidad · cuellos de botella y flujo · clases de servicio · cadencia.
- **Tipos de trabajo**: requerimientos · defectos · mantenimiento · solicitudes.
- **Clases de servicio**: Expreso (blanco) · Fecha fija (rosa) · Estándar (amarillo) · Intangible (verde).
- **Temas de políticas**: reposición · criterios de tracción (“hecho” por actividad) · WIP · clases de servicio · cadencia · acuerdos de colaboración.

Preguntas de repaso

1. **¿Por qué limitar el WIP aumenta el throughput si “hay gente ociosa”?** — Porque con todos al 100% no hay holgura y el trabajo hace cola (autopista en hora pico); limitar el WIP reduce el multitasking (desperdicio de cambio de contexto) y obliga a terminar antes de empezar.
2. **Un equipo tiene 20 historias “en desarrollo” y 0 terminadas. ¿Qué desperdicio Lean es y qué práctica Kanban lo ataca?** — Trabajo parcialmente terminado (inventario); se ataca limitando el WIP.
3. **¿Qué diferencia hay entre una columna, un carril y una clase de servicio?** — La columna es una etapa del proceso; el carril es un tipo de trabajo (con su % de capacidad); la clase de servicio es la urgencia de una tarjeta (color) y define con qué regla se mueve.

Métricas

> **U2 — Gestión Lean Agile de Productos de Software** · Sección §4 del resumen completo. Índice y convención de fuentes en 00-indice. Las métricas de flujo se usan en 03-kanban; las de defectos conectan con 11-testing y 10-revisiones-tecnicas.

Cómo leer este archivo: se lee de corrido. Cada métrica aparece una sola vez, explicada por la pregunta que responde. Al final hay una referencia rápida y preguntas de práctica.

Ideas transversales que acá solo se nombran:

- Error y Defecto: los "defectos" son una de las 4 métricas básicas, y la densidad de defectos es una métrica de producto.
- Limitar el WIP: las métricas de flujo (lead, cycle, touch) muestran el efecto de limitar el WIP.

El ejemplo que recorre el archivo: el proyecto "pagar en cuotas" de la tienda de Crovato, medido por los tres enfoques.

La idea del tema

Cada enfoque mide distinto porque controla distinto [Fuente: Apuntes — 11 Métricas]:

- el **tradicional** controla el cumplimiento del plan (procesos definidos);
- el **ágil** mide el valor entregado (procesos empíricos);
- **Lean/Kanban** mide el **flujo**.

El cierre de la cátedra: "**comparar herramientas para entender, no para juzgar**".

Para cualquier pregunta de métricas, primero preguntate **qué le preocupa a ese enfoque**, y la métrica sale sola. El archivo sigue ese orden:

1. Al tradicional le preocupa cumplir el plan → esfuerzo, tiempo, costos, defectos.
2. A ágil le preocupa entregar valor en cada Sprint → velocidad, capacidad, RTF.
3. A Lean/Kanban le preocupa que el trabajo fluya sin esperas → lead, cycle y touch time, eficiencia; y el servicio al cliente.
4. Lo que vale para todos → métricas de producto.
5. Métricas de seguimiento que no están en las presentaciones → SPI, CPI, effort variance, failure rate.

---

1. Enfoque tradicional: cumplir el plan

El dominio de las métricas se divide en **métricas de proceso, de proyecto y de producto**. Las métricas de proyecto se **consolidan** para crear métricas de proceso públicas para toda la organización [Fuente: Apuntes].

Las **4 métricas básicas de un proyecto** son **tamaño del producto, esfuerzo, tiempo (calendario) y defectos** [Fuente: Apuntes]. Son los recursos que fija el plan, más lo que sale mal.

“El sueño del pibe” es lo que cada nivel querría medir [Fuente: Apuntes]. Cada nivel mide lo que alcanza a ver, y cuanto más arriba, más largo es el horizonte:

- **El desarrollador ve su tarea**:
  - esfuerzo;
  - esfuerzo y duración estimada vs. real de una tarea;
  - % de cobertura de unit test;
  - número y tipo de defectos en unit test;
  - defectos en revisión de pares.
- **El equipo ve el proyecto, de hito en hito**:
  - tamaño del producto;
  - duración estimada vs. real entre hitos;
  - staffing actual vs. estimado;
  - tareas planificadas vs. completadas;
  - distribución del esfuerzo;
  - status y **volatilidad de requerimientos**;
  - defectos en integración/prueba de sistema y en peer reviews;
  - status de distribución de defectos;
  - % de tests ejecutados.
- **La organización ve todos los proyectos y el resultado final**:
  - tiempo calendario;
  - performance real vs. planificada de esfuerzo y de presupuesto;
  - **precisión de las estimaciones** de schedule y esfuerzo;
  - defectos en release.

En los tres niveles hay alguna métrica de **defectos**, cada uno donde le toca verlos: en mi código, al integrar, en la release.

“Mantenélo simple”: *si estás a millas de tu destino, no tiene sentido medir en milímetros* [Fuente: Apuntes]. Antes de medir algo, hay que preguntarse:

- ¿nos da más información que la que ya tenemos?
- ¿tiene beneficio práctico?
- ¿nos dice lo que queremos saber?

Las métricas sirven para **balancear** necesidades y expectativas de calidad, alcance, restricciones y proceso.

En Crovato, programando el checkout:

- **A vos** te importa que la tarea “calcular cuotas” te llevó 6 horas en vez de 3, y que tus tests cubren el 80% de `precios.ts`.
- **Al equipo** le importa que el hito “cuotas en producción” se atrasó una semana, porque el dueño cambió tres veces cuántas cuotas ofrecer (volatilidad).
- **A la empresa**, mirando todo el año, le importa que todas las estimaciones se pasaron un 40%, y que la release salió con dos bugs que vieron los clientes.

> **Un gerente quiere comparar la productividad de dos equipos con LOC por persona-mes. ¿Es confiable?**
> No. LOC/persona-mes no es confiable para comparar equipos de distinta experiencia.

---

2. Enfoque ágil: entregar valor en cada Sprint

La **regla de oro**: “**la medición es una salida, no una actividad**”. Es una filosofía minimalista: medir lo necesario y nada más [Fuente: Apuntes].

Qué medir lo guían **dos principios ágiles** [Fuente: Apuntes]:

- “satisfacer al cliente mediante entregas tempranas y continuas de software valioso”;
- “**el software funcionando es la principal medida de progreso**”.

De ahí salen **las 3 métricas ágiles de la cátedra** [Fuente: Apuntes]:

- **Velocidad**: ¿cuánto terminamos? Son los story points de historias **terminadas** (Done) por Sprint. Es **del equipo**, y no sirve para comparar equipos. Ejemplo de la cátedra: sprints 1-7 con 30, 30, 45, 60, 58, 52, 60 puntos.
- **Capacidad**: ¿cuánto podemos comprometer? Son las horas disponibles del equipo por Sprint (ejemplo: 240, 240, 480, 480, 420, 420, 480). Se compara con la velocidad para planificar (Capacidad = horas/día × días disponibles; ver el resumen del 1er parcial).
- **Running Tested Features (RTF)**: ¿cuánto funciona de verdad? Es la cantidad de features **funcionando y que pasan sus pruebas** por iteración. Si se rompe algo, **baja**; por eso refleja valor real y no solo “esfuerzo gastado”. Es el principio “software funcionando” convertido en número.

> **Velocidad vs. capacidad**
>
> Lo que permite evaluar cuánto puede comprometer el equipo en un sprint son **las horas disponibles** (capacidad). La velocidad no, porque mide lo que ya se terminó y no lo que el equipo puede comprometer.

En Crovato, un equipo Scrum preguntaría:

- "¿cuántos puntos terminamos este Sprint?" (**velocidad**);
- "¿cuántas horas tenemos la semana que viene, con dos personas de vacaciones?" (**capacidad**);
- "¿cuántas features siguen andando y pasando sus tests?" (**RTF**).

> **Para la pregunta "¿qué permite evaluar la capacidad del equipo para comprometerse en un Sprint?", ¿sirve la velocidad?**
>
> No, lo que sirve son las horas disponibles. La velocidad mide lo que ya se terminó, no lo que se puede comprometer.

> **Un equipo "quemó" 50 story points, pero una regresión rompió dos features que ya andaban. ¿Qué métrica lo muestra y cuál no?**
>
> El RTF baja, porque solo cuenta features que funcionan y pasan sus tests. Los puntos quemados no lo muestran: miden esfuerzo, no valor.

---

3. Enfoque Lean/Kanban: que el trabajo fluya

[Fuente: Apuntes — 10 Lean Kanban y 11 Métricas]

Los tiempos

Los tres tiempos miden el mismo ítem desde puntos de partida distintos, y cada uno está contenido en el anterior: **lead ≥ cycle ≥ touch**.

- **Lead Time** (*tiempo de entrega*): la **vista del cliente**, desde que **se pide** el ítem hasta que **se entrega**. Es el "**ritmo de entrega**". Se mide en días.
- **Cycle Time** (*tiempo de ciclo*): la **vista interna**, desde que **se empieza a trabajar** el ítem hasta que termina el proceso. Es el "**ritmo de terminación**", y "**la medición más mecánica de la capacidad del proceso**".
- **Touch Time** (*tiempo de tocado*): los días en que el ítem fue **realmente trabajado** (estuvo en columnas de "en curso"), sin contar colas o buffers ni bloqueos.

Con ellos se calcula la **eficiencia del ciclo de proceso** = **Touch Time / Elapsed Time** (tiempo transcurrido). Se busca que se acerque al 100%, o sea, que el ítem casi no espere.

> **El resumen del compañero invierte lead y cycle**
>
> El resumen del compañero le asigna al **lead** time "ritmo de terminación" y "la medición más mecánica de la capacidad del proceso", y dice que el cycle time incluye el tiempo en cola. **La cátedra dice lo contrario** (verificado en las presentaciones 10 Lean Kanban y 11 Métricas):
>
> - **cycle** time = "medición más mecánica de la capacidad del proceso", "ritmo de terminación";
> - **lead** time = "ritmo de entrega", e incluye la cola.
>
> Truco: el **lead** es el que ve el cliente, que **espera** desde que pide. Por eso incluye la cola.

Ejemplo numérico: el cliente pide una feature el día 1. Queda en la cola hasta el día 6; el equipo la toma el día 6 y la entrega el día 16. Durante esos 10 días estuvo 4 días bloqueada, esperando una definición.

- **Lead time** = 15 días.
- **Cycle time** = 10 días.
- **Touch time** = 6 días.
- **Eficiencia** = 6/10 = **60%**, si el tiempo transcurrido se mide desde que se empezó; o 6/15 = **40%**, si se mide desde el pedido.

En cualquiera de los dos casos, el tiempo que el ítem pasó esperando es el desperdicio a atacar (las **esperas** de 02-lean).

En Crovato, un tablero Kanban preguntaría:

- "¿cuántos días pasaron desde que el cliente pidió las cuotas hasta que las usó?" (**lead**);
- "¿cuántos desde que empezamos a programarlas?" (**cycle**);
- "¿en cuántos de esos días alguien trabajó de verdad en eso?" (**touch**).

Las métricas orientadas a servicio

Son cuatro respuestas a "¿cuánto tarda?", dichas por distintas voces [Fuente: Apuntes]:

- **Expectativa de nivel de servicio**: lo que **el cliente quiere**.
- **Capacidad de nivel de servicio**: lo que el sistema **realmente puede** entregar.
- **Acuerdo de nivel de servicio (SLA)**: lo **acordado** con el cliente.
- **Umbral de adecuación del servicio**: el nivel por debajo del cual el servicio es **inaceptable** para el cliente; si no se cumple, el cliente se va.

Un buen SLA queda entre lo que el cliente quiere y lo que vos podés, y siempre por encima del umbral.

En Crovato:

- Un cliente quiere recibir su perfume al día siguiente (**expectativa**).
- Mirando tus números, la mayoría de los pedidos te llegan en 2 o 3 días (**capacidad**).
- En la web prometés “entrega en hasta 4 días hábiles”, con margen sobre lo que podés (**SLA**).
- Sabés que si tarda más de una semana, el cliente cancela y no vuelve a comprar (**umbral**).

> **Un cliente pide algo el día 1, el equipo empieza el día 3, lo trabaja 4 días y lo entrega el día 10. ¿Lead, cycle y touch?**
> Lead = 9 días (del 1 al 10), cycle = 7 días (del 3 al 10), touch = 4 días. Eficiencia sobre el cycle: 4 / 7 ≈ 57%.

> **¿Qué métrica mejora la previsibilidad de la entrega en Kanban?**
> El cycle time: es la medición más mecánica de la capacidad del proceso. El touch time mide eficiencia, no previsibilidad.

---

4. Lo que vale para todos: métricas de producto

Cuadro resumen de la cátedra:

Tradicional	Ágil	Lean
Esfuerzo	Velocidad	Lead Time
Tiempo	Capacidad	Cycle Time
Costos	Running Tested Features	Touch Time
Defectos		Eficiencia del proceso

Además, hay **métricas de producto** que aplican a cualquier enfoque [Fuente: Apuntes]:

- de **tamaño**: líneas de código, casos de uso, features, requerimientos, clases, puntos de historia;
- de **defectos**: defectos por severidad, **densidad de defectos** = defectos / tamaño;
- y **cobertura**.

Son las que miden **el producto**, no el avance. Por eso no sirven para responder preguntas sobre el avance contra el plan.

---

5. Métricas de seguimiento que no están en las presentaciones

Son métricas clásicas de seguimiento de proyectos que las presentaciones no desarrollan [Fuente: Externa — Earned Value Management (PMI); Insight — EVM; Nimble — Effort variance]. Cada una responde a una sola pregunta:

Pregunta	Métrica	Fórmula	Qué mide	Enfoque
¿Voy a tiempo?	**SPI** (*Schedule Performance Index*, índice de desempeño del cronograma)	EV / PV = valor del trabajo hecho / valor del trabajo que se planeaba tener hecho a esta fecha	**Avance contra el plan**: >1 adelantado, <1 atrasado	Tradicional
¿Voy dentro del presupuesto?	**CPI** (*Cost Performance Index*)	EV / AC = valor del trabajo hecho / costo real	**Costo** contra el plan: >1 bajo presupuesto, <1 excedido	Tradicional
¿Cuánto me desvié del esfuerzo estimado?	**Effort Variance** (variación del esfuerzo)	(esfuerzo real – estimado) / estimado × 100	Desvío del **esfuerzo** respecto de lo estimado (positivo = se gastó más); también refleja qué tan bien se estima	Tradicional
¿Cuánto esfuerzo puse?	**Horas-persona** consumidas (por etapa)	suma de horas trabajadas	**Esfuerzo** invertido	Tradicional
¿El producto aguanta en uso?	**Failure rate** (tasa de fallos)	fallas / tiempo de operación	**Confiabilidad del software en uso** (en producción, no durante el desarrollo)	Producto

Truco para SPI y CPI: arriba siempre va “lo que hice” (**EV**). Abajo va lo que mirás: el plan (**PV**) para el tiempo, el costo (**AC**) para la plata. Mayor que 1 es bueno; menor que 1, malo.

Ejemplo:

- Se planificaron 200 h para un módulo y se usaron 250 h → effort variance = (250 – 200)/200 = **+25%**.
- Si a la semana 6 se planeaba tener hecho el 50% (PV = $50.000) y está hecho el 40% (EV = $40.000) → **SPI = 0,8**: el proyecto está atrasado.

Para qué sirve cada una:

- “**Esfuerzo invertido**” → effort variance y horas-persona.
- “**Avance respecto de la planificación original**” → SPI y effort variance.

- **No** sirven para eso la cobertura de código, la complejidad ciclomática ni la tasa de defectos: miden el producto, no el avance (sección 4).

En Crovato, a la mitad del proyecto de cuotas:

- el plan decía que a esta fecha tendrías el 50% hecho, y tenés el 40%: **SPI = 0,8**, vas atrasado;
- planificaste 200 horas y ya llevás 250: **effort variance = +25%**;
- gastaste $45.000 en lo que valía $40.000 de trabajo: **CPI ≈ 0,89**, vas excedido;
- una vez en producción, contás cuántas veces se cae el checkout por semana: ese es el **failure rate**.

> **¿Qué métricas sirven para medir "el avance respecto de la planificación original"? ¿Cobertura de código?**
> No: la cobertura mide el producto. Para el avance contra el plan: SPI y effort variance.

> **Un proyecto tiene CPI = 1,2 y SPI = 0,7. ¿Qué está pasando?**
> Gasta menos de lo planificado para lo que lleva hecho (CPI > 1), pero va atrasado respecto del cronograma (SPI < 1).

Referencia rápida (las listas en limpio)

No hay contenido nuevo acá: son las mismas listas de arriba, sin explicación.

- **Dominio tradicional**: métricas de proceso · de proyecto · de producto.
- **4 métricas básicas**: tamaño · esfuerzo · tiempo · defectos.
- **Sueño del pibe**: desarrollador (su tarea) · equipo (el proyecto) · organización (todos los proyectos).
- **Ágil**: velocidad · capacidad · RTF.
- **Lean/Kanban**: lead time (ritmo de entrega) · cycle time (ritmo de terminación) · touch time · eficiencia = touch / transcurrido.
- **Orientadas a servicio**: expectativa · capacidad · SLA · umbral de adecuación.
- **Producto**: tamaño · defectos (densidad = defectos / tamaño) · cobertura.
- **Externas (seguimiento)**: SPI = EV/PV · CPI = EV/AC · effort variance · horas-persona · failure rate.

Preguntas de repaso

1. **Diferencia entre lead time y cycle time.** — Lead: desde el pedido del cliente (vista del cliente). Cycle: desde que el equipo empieza (vista interna). Lead − Cycle = tiempo en cola

antes de empezar.

2. **¿Por qué el RTF es mejor indicador que “story points quemados”?** — Porque solo cuenta features que funcionan y pasan sus tests; si una regresión rompe una, el RTF baja. Mide valor entregado, no esfuerzo.
3. **¿Por qué cada enfoque usa métricas distintas?** — Porque controla distinto: el tradicional, el cumplimiento del plan; el ágil, el valor entregado; Lean/Kanban, el flujo.

Prácticas continuas (CI/CD)

> **U3 — Prácticas continuas** · Sección §5 del resumen completo. Índice y convención de fuentes en 00-indice. Se apoya en 12-testing-agil-y-tdd (pirámide, automatización) y en SCM del 1er parcial.

Cómo leer este archivo: se lee de corrido, como una sola explicación. Las listas aparecen una sola vez dentro del texto. Al final hay una **referencia rápida** y preguntas de repaso.

Ojo con la fuente: el tema **no tiene presentación propia** en la carpeta de la cátedra. La cátedra lo nombra en tres lugares:

- en la presentación de **Requerimientos Ágiles**, entre los valores de los equipos ágiles: "integración continua", junto a "testing concurrente" y TDD/ATDD [Fuente: Apuntes — 03 Rqs Ágiles];
- en **SCM**, con la referencia a las estrategias de despliegue blue-green y canary (Harness.io) [Fuente: Apuntes — 02 SCM];
- en **SCM en Agile**, con ideas como "automatizar lo más posible" y "feedback continuo y visible sobre calidad, estabilidad e integridad" [Fuente: Apuntes — 02 SCM].

El resto del desarrollo es `[Fuente: Externa]` salvo indicación.

Ideas que se repiten en otros temas y tienen nota propia:

- Testing Temprano: integrar y probar cada día, en vez de en una "fase de integración" final.
- Calidad Embebida: las prácticas continuas son "aseguramiento de calidad embebido".
- Verificación y Validación: el pipeline verifica automáticamente cada cambio.

El ejemplo que recorre el archivo: tu proyecto real `tecnex` (que ya tiene pipeline) y la tienda de Crovato (que todavía no lo tiene).

La idea del tema: que integrar, probar y desplegar sea aburrido

Si el software es un **producto vivo** que se entrega en incrementos frecuentes (y no un proyecto que se entrega una vez), entonces **integrar, probar y desplegar** tienen que ser actividades **automáticas, frecuentes y de bajo riesgo**. Las "prácticas continuas" son la evolución práctica de dos cosas que ya viste:

- lo que en SCM se vio como builds privados, de integración y de release (Berczuk);
- lo que pide Lean: lotes chicos, feedback rápido, entregar lo antes posible.

Las tres prácticas son **la misma cinta transportadora**, y lo único que cambia es **hasta dónde llega sola**. El archivo responde, en orden:

1. ¿Hasta dónde llega sola la cinta? → integración, entrega y despliegue continuos.
2. ¿Por qué etapas pasa? → el pipeline.
3. ¿Cómo libero sin que un error afecte a todos? → canary y blue-green.
4. ¿Qué tiene que ver con la calidad? → aseguramiento embebido y SCM en ágil.

---

1. Hasta dónde llega sola la cinta: CI, entrega y despliegue continuos

Integración Continua (CI): llega sola hasta "el código integrado está probado"

Cada desarrollador integra su trabajo a la rama principal **al menos una vez por día**. Cada integración dispara un **build automático con tests automatizados**, que avisa en minutos si algo se rompió. El objetivo es detectar los errores de integración cuando son chicos y baratos, en vez de en una "fase de integración" final [Fuente: Externa — Fowler, "Continuous Integration", martinfowler.com; Humble & Farley, *Continuous Delivery*, 2010]. Es Testing Temprano aplicado a la integración.

Las **prácticas asociadas** responden a cómo lograr que la cinta no se trabe:

- **un único repositorio**: todos ponen las cosas en la misma cinta;
- **commits frecuentes a la rama principal**: cosas chicas y seguido;
- **build automatizado y autoverificable** (con tests): la cinta se controla sola;
- **build rápido**: avisa en minutos;
- **arreglar el build roto inmediatamente**, con prioridad máxima: si la cinta para, para todo el mundo.

Entrega Continua (Continuous Delivery): llega sola hasta "listo para producción"

El software está **siempre en estado desplegable**: cada cambio que pasa el pipeline queda listo para producción, y desplegar es una **decisión de negocio** que se ejecuta con un botón. Hay un **paso manual de aprobación** antes de producción [Fuente: Externa — Fowler; Humble & Farley; coinciden fuentes secundarias (GitLab handbook, BrowserStack)].

Despliegue Continuo (Continuous Deployment): llega sola hasta producción

Va un paso más: **todo cambio que pasa el pipeline se despliega automáticamente** a producción, sin intervención humana. Para hacer despliegue continuo **hay que** hacer entrega continua primero [Fuente: Externa — mismas fuentes].

Cada una **contiene** a la anterior: no hay despliegue continuo sin entrega continua, ni entrega continua sin CI.

	CI	Continuous Delivery	Continuous Deployment
Automatiza	build + tests al integrar	todo hasta "listo para producción"	todo, incluido el pase a producción
¿Pase a producción?	no aplica	**manual** (decisión de negocio)	**automático**
Requiere	tests unitarios/integración automatizados	+ tests de aceptación automatizados, ambientes reproducibles	+ altísima confianza en los tests y monitoreo

Ejemplo real, de tu proyecto `tecnex`:

- **CI.** En la API, cada push a cualquier rama dispara GitHub Actions, que levanta un Postgres limpio y corre RSpec y Brakeman.
- **Entrega continua.** Cuando el código llega a la rama `production`, el mismo pipeline despliega solo a Fly.io, pero solo si los tests y Brakeman pasaron. La decisión de llegar a esa rama la toma una persona al hacer el merge.
- **Casi despliegue continuo.** El frontend en Vercel se despliega con cada push a `main`, sin nadie en el medio. Se parece al despliegue continuo, pero no tiene tests automáticos, que es justamente lo que ese enfoque exige.

Para reconstruirlo: **probado, listo, publicado**. Lo único que cambia es si hay un humano antes de producción.

Otros ejemplos reales: GitHub Actions o GitLab CI corriendo `npm test` y `npm run build` en cada push. Amazon y Netflix despliegan a producción miles de veces por día (despliegue continuo con canary).

En Crovato: `crovatofragrances/tienda` tiene un solo test (`src/lib/setCombo.test.ts`) y no tiene carpeta `.github/workflows`, o sea que **hoy no hay CI**. El primer paso sería un workflow que corra lint, ese test y `next build` en cada push. Como la tienda se despliega con `wrangler` (Cloudflare), automatizar ese deploy después de que el pipeline pase sería entrega o despliegue continuo.

> **¿Cuál es la única diferencia entre entrega continua y despliegue continuo?**
> En la entrega continua el pase a producción es una decisión manual (de negocio); en el despliegue continuo es automático.

> **¿Por qué CI exige integrar al menos una vez por día?**
> Porque cuanto más tiempo vive un cambio sin integrar, más diverge, y más caros son el merge y el error. Además, el trabajo parcialmente terminado es desperdicio Lean.

> **Un equipo integra a la rama principal una vez por semana, pero tiene un servidor que corre los tests cada noche. ¿Hace CI?**
> No del todo. Tiene build automatizado, pero CI exige integrar al menos una vez por día. Con integraciones semanales, los cambios divergen y los errores de integración se acumulan.

---

2. Por qué etapas pasa la cinta: el pipeline

El pipeline típico (coincide con el resumen del 1er parcial, sección 25) tiene automatización creciente en cada etapa. Cada etapa prueba algo más grande y más parecido a la realidad que la anterior, y solo se pasa a la siguiente si la actual salió bien:

1. ¿**Compila**? → **construcción**.
2. ¿Cada **pieza** anda? → **pruebas unitarias**.
3. ¿Anda **instalado en un ambiente** parecido al real? → **despliegue a ambiente de prueba**.
4. ¿Hace **lo que el usuario pidió**? → **pruebas de aceptación**.
5. **Que lo usen** → **despliegue a producción**.

Se conecta directamente con la **pirámide de testing ágil** (12-testing-agil-y-tdd): sin una base grande de tests unitarios automatizados, no hay CI posible. Y las etapas 3 y 4 son los ambientes y niveles de 11-testing.

En Crovato: la nueva versión del checkout compila; `setCombo.test.ts` pasa; se instala en staging; ahí comprás un set en 6 cuotas con la tarjeta de prueba de MercadoPago y anda. Recién entonces sale a producción.

> **Un cambio pasa las pruebas unitarias pero falla las de aceptación en el ambiente de prueba. ¿Llega a producción?**
> No. En el pipeline solo se pasa a la siguiente etapa si la actual salió bien.

---

3. Cómo liberar sin que un error afecte a todos: estrategias de despliegue

La cátedra las referencia en SCM [Fuente: Apuntes — referencia Harness.io; desarrollo Externa]. Hay dos formas de bajar el riesgo de un despliegue:

- **Probándolo con pocos primero → canary deployment.** La versión nueva se libera primero a un **subconjunto pequeño de usuarios** (el “canario en la mina”: si algo anda mal, lo sufren pocos). Si las métricas están bien, se amplía gradualmente.

- **Teniendo la versión anterior lista para volver al instante → blue-green deployment.** Hay dos ambientes de producción idénticos: “blue” corre la versión actual y “green” la nueva. Se **conmuta el tráfico** a green; si algo falla, se vuelve a blue al instante (rollback rápido).

En Crovato: para no arriesgar todas las ventas del sábado, mostrás la versión nueva del checkout primero al 5% de los visitantes y mirás si fallan los pagos (**canary**). O dejás la versión vieja corriendo al lado: si la nueva falla, en un clic volvés todo el tráfico a la vieja (**blue-green**).

> **¿Qué ventaja tiene blue-green sobre un despliegue “in place”?**
> Permite volver atrás instantáneamente conmutando el tráfico, sin reinstalar la versión anterior.

> **Querés probar una versión nueva con usuarios reales, pero sin exponer a todos al riesgo. ¿Qué estrategia usás?**
> Canary: liberarla primero a un subconjunto pequeño y ampliarla gradualmente si las métricas están bien.

---

4. Qué tiene que ver con la calidad

Las prácticas continuas son “**aseguramiento de calidad embebido**” (Calidad Embebida): en vez de auditar o testear al final, cada cambio se verifica automáticamente al integrarse. Por eso, en ágil, buena parte del control formal (CCB, auditorías de configuración) se reemplaza por el pipeline automatizado (ver el resumen del 1er parcial, SCM en ágil).

La presentación de SCM lo dice con sus palabras, en “SCM en Agile” [Fuente: Apuntes — 02 SCM]:

- sirve a los practicantes, y no al revés;
- hace seguimiento y coordina, en lugar de controlar a los desarrolladores;
- responde a los cambios en lugar de tratar de evitarlos;
- busca ser transparente y “sin fricción”, **automatizando tanto como sea posible**, con coordinación y automatización frecuente y rápida;
- elimina el desperdicio y usa documentación Lean y trazabilidad;
- da **feedback continuo y visible sobre calidad, estabilidad e integridad**.

Sus “tips” son:

- es **responsabilidad de todo el equipo**;
- automatizar lo más posible;
- educar al equipo;
- embeber las tareas de SCM en las demás tareas del Sprint.

Y deja para debatir qué pasa con el Comité de Control de Cambios, los ítems de configuración, las auditorías y los reportes de estado.

Calidad antes y después del despliegue. El pipeline previene: no deja pasar lo que no supera los tests. Pero hay problemas que solo aparecen en producción, y para esos hacen falta prácticas que **detectan** después del despliegue: monitoreo continuo, métricas de uso y carga, y alertas automáticas.

> **Después de varios despliegues exitosos, el rendimiento en producción empieza a bajar. ¿Sirve agregar pruebas de carga antes del despliegue?**
> No para este problema: las pruebas de carga previas previenen, pero no detectan algo que ya está en producción. Lo que corresponde es monitoreo continuo de rendimiento, análisis de métricas de uso y carga, y alertas automáticas ante degradación.

> **En un equipo ágil con CI/CD, ¿quién se encarga de que el build no esté roto?**
> Todo el equipo: arreglar el build roto es prioridad máxima para cualquiera, y la cátedra dice que SCM en ágil es responsabilidad de todo el equipo.

Referencia rápida (las listas en limpio)

No hay contenido nuevo acá: son las mismas listas de arriba, sin explicación.

- **CI → Entrega continua → Despliegue continuo**: probado · listo (pase manual) · publicado (pase automático). Cada una contiene a la anterior.
- **Prácticas de CI**: único repositorio · commits frecuentes (al menos una vez por día) · build automatizado y autoverificable · build rápido · arreglar el build roto inmediatamente.
- **Pipeline**: construcción · pruebas unitarias · despliegue a ambiente de prueba · pruebas de aceptación · despliegue a producción.
- **Estrategias de despliegue**: canary (pocos usuarios primero) · blue-green (dos ambientes, conmutar el tráfico).
- **SCM en ágil (tips)**: responsabilidad de todo el equipo · automatizar lo más posible · educar al equipo · tareas de SCM embebidas en el Sprint.

Preguntas de repaso

1. **¿Diferencia entre Continuous Delivery y Continuous Deployment?** — En Delivery el software está siempre listo y el pase a producción es manual (decisión de negocio); en Deployment cada cambio que pasa el pipeline va a producción automáticamente.

2. **¿Por qué CI exige integrar al menos una vez por día?** — Porque cuanto más tiempo vive un cambio sin integrar, más diverge y más caros son el merge y el error (trabajo parcialmente terminado = desperdicio Lean).
3. **¿Qué ventaja tiene blue-green sobre un despliegue "in place"?** — Permite volver atrás instantáneamente conmutando el tráfico, sin reinstalar la versión anterior.

Calidad

> **U4 — Aseguramiento de Calidad de Proceso y de Producto** · Secciones §6 y §7 del resumen completo. Índice y convención de fuentes en 00-indice. Este archivo es la puerta de entrada a toda la U4: el **mapa** de la sección 4 ubica a los demás temas (07-aseguramiento-de-calidad, 08-modelos-y-estandares, 09-auditorias, 10-revisiones-tecnicas, 11-testing).

Cómo leer este archivo: se lee de corrido. Las listas de la cátedra aparecen una sola vez, dentro del texto. Al final hay una referencia rápida con las listas en limpio y preguntas de práctica.

Las ideas que se repiten en otros temas tienen nota propia, y acá solo se nombran:

- Calidad Embebida: “no se inyecta”, “es un esfuerzo de todos”, “más pruebas no es más calidad”, “el testing controla, no asegura”, “es más barato”. Son cinco frases para una sola idea.
- Verificación y Validación: aparece en los tres círculos de la calidad.

Este tema es la base para entender por qué existen el aseguramiento, las revisiones, las auditorías y el testing. Sus ideas reaparecen en los otros temas, por ejemplo “el testing controla, no asegura”.

El ejemplo que recorre el archivo: el mismo proyecto de 11-testing, “pagar en cuotas” en la tienda de Crovato, pero esta vez visto antes de que se haga bien.

La idea del tema

Los proyectos de software fallan de forma repetida y previsible, y la cátedra llama a eso **falta de calidad**. El tema responde, en orden:

1. ¿Cómo se ve la falta de calidad? → los síntomas.
2. ¿Qué es la calidad, si cada uno la ve distinto? → definición, a quién satisface, las 5 visiones.
3. ¿Qué busca gestionarla? → que coincidan la calidad necesaria, la programada y la realizada.
4. ¿Dónde y con qué se trabaja? → el mapa: proceso, proyecto y producto; modelos, aseguramiento y control.
5. ¿Con qué actitud? → los 10 principios.
6. ¿Vale la pena? → ventajas y desventajas.

---

1. Cómo se ve la falta de calidad

La cátedra arranca preguntando qué pasa frecuentemente en los proyectos [Fuente: Apuntes — 12 PPQA]. Los síntomas duelen en cuatro lugares:

- **En el plan**: atrasos en las entregas, costos excedidos, incumplimiento de compromisos.
- **En lo que se construye**: requerimientos poco claros, y software que no hace lo que tiene que hacer.
- **En la gente**: trabajo fuera de hora.
- **En el control**:
  - el **fenómeno del 90-90**: "está 90% hecho"… y falta otro 90%, porque nadie sabe cuánto falta de verdad;
  - "**¿dónde está ese componente?**": nadie sabe qué versión es cuál, por falta de SCM.

Las estadísticas que muestra la cátedra: ~90% de los proyectos retrasados, ~50% excedidos en presupuesto, ~50% no alcanzan los objetivos, ~30% cancelados.

En Crovato, el proyecto de cuotas iba a durar dos semanas y lleva seis (**atraso**). Pagaste horas extra de un freelancer (**costo**) y le prometiste a un cliente que el lunes estaba y no estuvo (**compromiso**). Nunca quedó claro si las 6 cuotas eran con o sin interés (**requerimientos**), así que cobra mal (**no hace lo que tiene que hacer**). Te quedás hasta las 2 de la mañana arreglándolo (**fuera de hora**). Cada día decís "ya está casi" y al otro día aparece otra cosa (**90-90**). Y cuando querés volver a la versión que andaba, no sabés cuál era (**¿dónde está ese componente?**).

---

2. Qué es la calidad

Definición de la cátedra: calidad son "**todos los aspectos y características de un producto o servicio que se relacionan con su habilidad de alcanzar las necesidades manifiestas o implícitas**" [Fuente: Apuntes — 12 PPQA]. Lo implícito importa: nadie pide "que no se cuelgue", pero se espera.

La calidad es relativa: depende de las personas, su edad, las circunstancias y el tiempo. "El tiempo varía las percepciones": lo que hoy es un buen celular, en 5 años no lo es [Fuente: Apuntes].

Como es relativa, la pregunta siguiente es **¿calidad para quién?** Un software de calidad satisface a varios interesados [Fuente: Apuntes]. Si te olvidás de alguno, pensá en quién se queja cuando el software anda mal:

- al **cliente**, que paga;
- al **usuario**, que lo usa;

- a la **gerencia**, que lo dirige;
- al **equipo de desarrollo y mantenimiento**, que lo construye y lo mantiene;
- y a otros interesados.

Cada uno juzga la calidad con una vara distinta. Esas son **las 5 visiones** [Fuente: Apuntes; definiciones de Garvin — Externa, Pressman cap. 14]:

- **Del usuario** (¿me sirve?): cumple lo que el usuario necesita (fácil de usar, rápido).
- **De manufactura** (¿se hizo como se especificó?): se ajusta a la especificación, se construyó como se dijo.
- **Del producto** (¿qué características medibles tiene?): la calidad está ligada a atributos internos medibles (funciones, características).
- **Basada en el valor** (¿cuánto pagaría por esto?): cuánto está dispuesto a pagar el cliente por el producto.
- **Trascendental** (¿para qué existe?): la calidad "se reconoce pero no se puede definir". En la cátedra está ligada a "¿para qué quiero construir este software?".

En Crovato, cinco personas opinan sobre la tienda:

- Un cliente: "Es buenísima, compro en dos clics" (**usuario**).
- Vos, con la ERS en la mano: "Hace exactamente lo que especificamos" (**manufactura**).
- Un amigo programador, después de correr Lighthouse: "Carga en 1,2 s y no tiene errores de accesibilidad" (**producto**).
- Un comprador mayorista: "Pagaría por tener algo así en mi negocio" (**valor**).
- Tu socio: "No sé explicarlo, pero se nota que está bien hecha; es lo que queríamos que fuera la marca" (**trascendental**).

> **"El sistema cumple al 100% la especificación." ¿Qué visión es? ¿Alcanza para decir que tiene calidad?**
> Es la visión de manufactura (conformidad con la especificación). No alcanza: si la especificación no recogió lo que el usuario necesita, falla la visión del usuario. Es la diferencia entre verificar y validar.

---

3. Qué busca gestionar la calidad: que coincidan tres círculos

El diagrama de 3 círculos de la cátedra [Fuente: Apuntes] compara tres versiones del mismo producto:

- **Necesaria**: lo que **hacía falta**, es decir, lo que el usuario realmente necesita.
- **Programada**: lo que **dijimos** que íbamos a hacer; la que se planificó o especificó alcanzar.
- **Realizada** (lograda): lo que **hicimos** de verdad; la que efectivamente tiene el producto.

La **gestión de la calidad** busca que los tres círculos **coincidan** lo más posible. "**Todo lo que esté fuera de dicha coincidencia será desperdicio o insatisfacción**." Lo que queda afuera se clasifica con una sola pregunta, **¿alguien lo necesitaba?**:

- **No, pero se programó o se hizo → desperdicio**.
- **Sí, pero no se programó o no se logró → insatisfacción**.

Para acercar los círculos se usan dos herramientas que aparecen en toda la unidad (Verificación y Validación):

- **verificar** acerca la realizada a la programada (¿lo hicimos como lo dijimos?);
- **validar** acerca la realizada a la necesaria (¿era lo que hacía falta?).

En Crovato:

- Tus clientes necesitan pagar en cuotas y saber si hay stock (**necesaria**).
- La ERS pide cuotas y, además, 12 cuotas "por las dudas" (**programada**).
- Lo que salió cobra 1, 3 y 12 cuotas, pero falla en 6 (**realizada**).

Las 12 cuotas se hicieron y nadie las usa: **desperdicio**. Ver el stock hacía falta y nadie lo especificó: **insatisfacción**. Las 6 cuotas se programaron pero no se lograron: **insatisfacción**.

> **Se programó y se construyó un reporte que ningún usuario pidió ni usa. ¿Qué es, según los tres círculos?**
> Desperdicio: está en la programada y en la realizada, pero fuera de la necesaria.

---

4. Dónde y con qué se trabaja: el mapa de la calidad

Este es el esquema que ordena **toda la unidad** [Fuente: Apuntes — 12 PPQA]. Cruza **qué se mira** (filas) con **qué se hace** (columnas):

- **Filas**, de lo general a lo particular:
  - **proceso**: cómo trabaja la organización;
  - **proyecto**: una instancia de ese trabajo;
  - **producto**: lo que sale.
- **Columnas**:
  - **modelos**: contra qué me comparo;
  - **aseguramiento**: cómo prevengo;
  - **control**: cómo detecto.

Ámbito	Modelos	Técnicas y herramientas para aseguramiento	Para control
Proceso	para crearlos, mejorarlos, evaluarlos (CMMI, SPICE, IDEAL, ISO) → 08-modelos-y-estandares	Revisiones técnicas, Auditorías	—
Proyecto	—	Revisiones técnicas, Auditorías (de proyecto)	—
Producto	para medirlos (ISO 25010, McCall, Barbacci); no entran en este temario	Revisiones técnicas, Auditorías (de configuración)	**Testing** → 11-testing

Cada celda sale razonando:

- Las **revisiones técnicas** (10-revisiones-tecnicas) están en las tres filas, porque se puede revisar cualquier cosa.
- Las **auditorías** (09-auditorias) cambian de nombre según la fila: de proyecto o de configuración.
- El **testing** está solo en producto y solo en control, porque necesita algo que ejecutar y **detecta, no previene**.
- Los **modelos** sirven, para el proceso, para crearlo, mejorarlo o evaluarlo; para el producto, para medirlo.

La idea clave sale de la última columna: **el testing controla la calidad del producto; no la asegura**. Asegurar es prevenir (durante el proceso); controlar es detectar (sobre el producto). Es Calidad Embebida.

Por qué hay una fila "proyecto" entre proceso y producto. El proceso es una definición de cómo trabajar; se vuelve real cuando se ejecuta en un proyecto. El resumen del compañero lo dice así: "el proceso se materializa en proyectos, los cuales son la unidad de trabajo que da vida a dicho proceso", y al iniciar un proyecto hay que incorporar actividades de aseguramiento como revisiones técnicas y auditorías. Por eso la auditoría de proyecto mira si el proyecto sigue el proceso.

En Crovato:

1. Definís que todo cambio pasa por rama, test y deploy, basándote en lo que pide CMMI (**proceso, modelo**).
2. Para la versión de cuotas (**proyecto**), alguien revisa a mitad de camino que se esté siguiendo ese proceso (**auditoría de proyecto**).
3. Antes de escribir código, revisan juntos la ERS (**revisión técnica**, prevenir).
4. Al final, la tienda (**producto**) se prueba comprando en 6 cuotas (**testing**, detectar), y se mide contra la lista de ISO 25010 para no olvidar seguridad ni rendimiento (**modelo de producto**).

> **¿Por qué el testing no aparece como técnica de aseguramiento?**
> Porque el testing detecta defectos que ya están en el producto (control). Asegurar es prevenir que se introduzcan, durante el proceso.

> **Un gerente propone "contratemos más testers para asegurar la calidad". ¿Qué le contestás con el mapa?**
> Que los testers controlan, no aseguran: detectan defectos sobre el producto ya hecho. Para asegurar hay que trabajar en el proceso y el proyecto (estándares, revisiones técnicas, auditorías). Los testers pueden ser necesarios igual, pero no reemplazan eso.

---

5. Con qué actitud: los 10 principios

Los **principios de la cátedra** [Fuente: Apuntes — 12 PPQA] responden a tres preguntas.

¿Dónde está la calidad? En el producto desde el principio; no se agrega al final. Esto es Calidad Embebida:

- (1) La calidad **no se "inyecta" ni se compra**: debe estar **embebida** desde el inicio.
- (9) **El aumento de las pruebas no aumenta la calidad**: la calidad se construye; el testing solo la revela.

¿Quién la hace?

- (2) Es un **esfuerzo de todos**.
- (3) Las **personas son la clave** para lograrla → **capacitación**.
- (4) Se necesita un **sponsor a nivel gerencial**… **pero se puede empezar por uno**.
- (5) Se debe **liderar con el ejemplo**.

¿Cómo se gestiona?

- (6) **No se puede controlar lo que no se mide**.
- (7) **Simplicidad**: empezar por lo básico.
- (8) El aseguramiento de calidad **debe planificarse**.
- (10) Debe ser **razonable para mi negocio**.

> **Ojo con el principio 6**
> El resumen del compañero lo escribe como "la calidad se mide mediante el testing". **La cátedra no dice eso**: dice "no se puede controlar lo que no se mide", que es sobre **medir** (métricas) para poder gestionar. La versión del compañero choca con el principio 9 y con "el testing controla, no asegura".

En Crovato:

- **Dónde está la calidad.** No la vas a arreglar al final probando el doble: los bugs de cuotas venían de una ERS ambigua, así que hay que trabajar bien desde el principio (**embebida**; **más pruebas no es más calidad**).
- **Quién la hace.** No lo podés hacer solo: el que diseña, el que programa y el que prueba tienen que estar en lo mismo (**esfuerzo de todos**), y les pagás un curso de testing (**personas, capacitación**). Lo ideal sería que el socio que maneja la plata lo apoye, pero mientras tanto empezás vos con tu propio código (**sponsor, pero se puede empezar por uno**), escribiendo tests aunque nadie te lo pida (**liderar con el ejemplo**).
- **Cómo se gestiona.** Anotás en el plan cuándo se revisa y cuándo se prueba (**planificarse**). Contás cuántos bugs llegan a producción por mes (**medir**). Arrancás solo con tests de `precios.ts` (**simplicidad**). Y no contratás una certificación ISO para una tienda de perfumes (**razonable para el negocio**).

> **“Para mejorar la calidad, dupliquemos los casos de prueba.” ¿Qué principio contradice?**
> El 9, el aumento de las pruebas no aumenta la calidad: el testing la revela, no la construye. La calidad tiene que estar embebida desde el inicio.

> **Un desarrollador quiere empezar a trabajar con calidad, pero la gerencia todavía no lo apoya. ¿Tiene sentido?**
> Sí. El principio 4 dice que hace falta un sponsor gerencial, pero que se puede empezar por uno; y el 5, que hay que liderar con el ejemplo.

---

6. ¿Vale la pena? Ventajas y desventajas

Trabajar con calidad es una **inversión**: las desventajas son lo que pagás al principio, y las ventajas, lo que cobrás después.

Lo que pagás al principio (desventajas o costos) [Fuente: Apuntes — se desprende de revisiones y auditorías; resumen del compañero]:

- **Costo inicial**: capacitación, plantillas, herramientas, programas de medición. El ahorro llega **recién cuando el equipo adquiere experiencia**.
- Requiere tiempo, y **parece ralentizar** el desarrollo al principio.
- Riesgo de **burocracia**: “no use prácticas inapropiadas simplemente porque se han establecido los estándares” [Fuente: Apuntes].
- Requiere **cambio y compromiso organizacional**: sin sponsor, se diluye.

Lo que cobrás después (ventajas):

- **Es más barato**: "**trabajar con calidad es más barato que hacerlo sin calidad**" [Fuente: Apuntes — Puntos claves PPQA]. El costo de reparar una falla crece ~**10x por etapa**: es barato en requerimientos y carísimo en producción [Fuente: Apuntes — 13 Revisiones Técnicas, datos de Gilb & Graham 1993].
- **Menos retrabajo**: el retrabajo evitable es el **40-50%** del desarrollo [Fuente: Apuntes — 13 Revisiones Técnicas].
- La mejora de procesos ha demostrado **retornos de inversión sustanciales** [Fuente: Apuntes].
- Predictibilidad (cumplir plazos y costos), clientes satisfechos, crecimiento del negocio y visibilidad para la gerencia.

Fijate que las ventajas son los **síntomas de la sección 1 dados vuelta**: atrasos y costos excedidos → predictibilidad; software que no hace lo que tiene que hacer → clientes satisfechos; trabajo fuera de hora → menos retrabajo.

En Crovato, decidís que toda ERS se revisa antes de programar. El primer mes es un fastidio: armás una plantilla, aprendés a revisar, y todo parece ir más lento (**costo inicial, parece lento**). Si encima obligás a revisar hasta el texto del footer, se vuelve burocracia (**riesgo**). Pero al tercer mes ves que el bug de las 6 cuotas, que en producción te costó un fin de semana, en la revisión de la ERS se habría resuelto en 10 minutos (**más barato**). Dejás de rehacer cosas (**menos retrabajo**) y empezás a cumplir las fechas que prometés (**predictibilidad**).

> **Un equipo adoptó revisiones y estándares hace un mes y dice que "van más lentos que antes". ¿Fracasó la iniciativa?**
> No necesariamente. El costo inicial y la sensación de lentitud son esperables: el ahorro llega cuando el equipo adquiere experiencia. Sí conviene revisar que no se haya vuelto burocracia (prácticas inapropiadas aplicadas solo porque están en el estándar).

Referencia rápida (las listas en limpio)

No hay contenido nuevo acá: son las mismas listas de arriba, sin explicación.

- **Síntomas**: atrasos · costos excedidos · incumplimiento de compromisos · requerimientos poco claros · no hace lo que tiene que hacer · trabajo fuera de hora · 90-90 · ¿dónde está ese componente?
- **A quién satisface**: cliente · usuario · gerencia · equipo de desarrollo y mantenimiento · otros interesados.
- **5 visiones**: usuario · manufactura · producto · valor · trascendental.

- **3 círculos**: necesaria · programada · realizada. Fuera de la intersección: desperdicio o insatisfacción.
- **Mapa**: filas proceso · proyecto · producto; columnas modelos · aseguramiento · control.
- **10 principios**: no se inyecta · esfuerzo de todos · personas/capacitación · sponsor (pero se puede empezar por uno) · liderar con el ejemplo · no se controla lo que no se mide · simplicidad · planificarse · más pruebas no es más calidad · razonable para el negocio.
- **Ventajas**: más barato (10x por etapa) · menos retrabajo (40-50%) · retorno de inversión · predictibilidad, clientes satisfechos, crecimiento, visibilidad.
- **Desventajas**: costo inicial · parece más lento · riesgo de burocracia · requiere cambio y compromiso.

Preguntas de repaso

1. **¿Qué significa que la calidad sea relativa?** — Que depende de quién la juzga (cliente, usuario, gerencia, equipo) y de cuándo: el tiempo varía las percepciones. Por eso hay 5 visiones.
2. **¿Qué busca la gestión de la calidad con los tres círculos?** — Que la calidad necesaria, la programada y la realizada coincidan. Lo que queda afuera es desperdicio (se hizo y no hacía falta) o insatisfacción (hacía falta y no se hizo).
3. **¿Asegurar y controlar son lo mismo?** — No. Asegurar es prevenir, durante el proceso (revisiones, auditorías). Controlar es detectar, sobre el producto (testing). Ver Calidad Embebida.

Aseguramiento de calidad (SQA / PPQA)

> **U4 — Aseguramiento de Calidad de Proceso y de Producto** · Sección §8 del resumen completo. Índice y convención de fuentes en 00-indice. Este archivo supone leído 06-calidad, sobre todo el **mapa** (sección 4).

Cómo leer este archivo: se lee de corrido. Las listas aparecen una sola vez dentro del texto; al final hay una referencia rápida y preguntas de práctica.

Ideas transversales que acá solo se nombran: Calidad Embebida ("cultura de calidad", "responsabilidad de todos", "el testing controla, no asegura") y Verificación y Validación.

La idea del tema

En 06-calidad quedó que la calidad no se agrega al final: hay que **asegurarla** mientras se trabaja. Este tema responde **quién se ocupa de eso y cómo**.

La respuesta de la cátedra cabe en una frase de Watts Humphrey: "**Lo que no está controlado no está hecho**" [Fuente: Apuntes — 12 PPQA]. La **administración de la calidad** se ocupa de asegurar que se alcancen los niveles requeridos de calidad para el producto. Para eso:

- **define estándares y procesos** de calidad apropiados;
- **asegura que se respeten**;
- debería ayudar a desarrollar una **"cultura de calidad"**, donde la calidad es vista como una responsabilidad de todos y cada uno (Calidad Embebida).

El resto del archivo desarrolla esa frase:

1. ¿Qué hace la administración de la calidad? → las 3 actividades, los objetivos y las funciones.
2. ¿Contra qué controla? → los estándares, y las revisiones de calidad para comprobarlos.
3. ¿Desde dónde, para ser creíble? → dónde reporta el Grupo de Aseguramiento de Calidad.
4. ¿Por qué se trabaja sobre el proceso? → el proceso basado en calidad, y proceso vs. producto.

> **Dos frases parecidas que no dicen lo mismo**
>
> - "**Lo que no está controlado no está hecho**" (Humphrey, este tema): un trabajo **no se da por terminado** hasta que alguien comprobó que se hizo como se debía.
> - "**No se puede controlar lo que no se mide**" (principio 6 de 06-calidad): para poder controlar, primero hay que **medir**.
>
> Juntas forman una cadena (la unión es propia): **medir → controlar → recién ahí está hecho**.

> **Ojo: "aseguramiento" y "control" tienen dos sentidos en la cátedra**
> Las dos palabras aparecen en dos diapositivas distintas de 12 PPQA, con alcances distintos. Verificado en la presentación:
>
> - **En el mapa de la calidad** (06-calidad, sección 4) son dos columnas. **Aseguramiento** = prevenir (revisiones técnicas, auditorías). **Control** = detectar sobre el producto (testing).
> - **En las 3 actividades de la administración de calidad** (sección 1 de este archivo):
>   - **Aseguramiento** es solo la primera actividad: establecer los estándares organizacionales.
>   - **Control** es la tercera: asegurar que el equipo respete esos estándares, con revisiones y mediciones. Mira **el proceso**, no solo el producto.
>
> Cuando una pregunta use estas palabras, fijate de cuál de los dos esquemas viene.

El ejemplo que recorre el archivo: la tienda de Crovato, después del proyecto de cuotas que salió mal. Decidís organizarte para que no vuelva a pasar.

---

1. Qué hace la administración de la calidad

Las 3 actividades: escribir las reglas, elegirlas y hacerlas cumplir

Son los tres verbos de cualquier regla: se **escribe**, se **elige cuál aplica** y se **hace cumplir** [Fuente: Apuntes — 12 PPQA; coincide con Sommerville cap. 24]:

1. **Aseguramiento de calidad** (escribir, para toda la organización): **establecer** estándares y procedimientos **organizacionales** de calidad.
2. **Planificación de calidad** (elegir, para este proyecto): **seleccionar** los procedimientos y estándares aplicables a un proyecto particular, y **modificarlos** si hace falta. El resultado es el **plan de calidad**, que:
   - define los productos de calidad deseados y **cómo serán evaluados**;
   - define los **atributos de calidad más significativos**;

    - debería definir el **proceso de evaluación** de la calidad;
    - define **qué estándares organizacionales** se aplican y si hace falta usar **estándares nuevos**.
3. **Control de calidad** (hacer cumplir, en el día a día): asegurar que el equipo de desarrollo **respete** esos procedimientos y estándares. Implica controlar el proceso de desarrollo. Tiene **dos enfoques**:
    - **revisiones de calidad**;
    - **evaluaciones de software automáticas y mediciones**.

Las tres van de lo general a lo particular: organización → proyecto → día a día.

Los objetivos de SQA son lo mismo visto como resultado [Fuente: Apuntes — 14 Auditorías]:

- realizar los **controles** apropiados del software y del proceso;
- asegurar el **cumplimiento** de estándares y procedimientos;
- asegurar que los defectos en producto, proceso o estándares **se informen a la gerencia** para que se solucionen.

En Crovato:

1. Escribís una regla para todo lo que hagas: “todo cambio de precios lleva test unitario” (**establecer**).
2. Para el próximo proyecto decidís que, además, como hay plata en juego, todo cambio en el checkout lo revisa otra persona; para la página “Nosotros” no aplicás esa regla (**planificar**).
3. Cada viernes alguien mira los PR de la semana para ver si se cumplió (**controlar**). Si encuentra uno sin test, te lo informa a vos, que sos quien puede corregirlo (**informar a la gerencia**).

Las funciones del SQA

Las **funciones** [Fuente: Apuntes] son las tareas concretas del grupo que hace todo eso. Son de dos tipos:

Preparar el terreno:

- desarrollar las herramientas, técnicas, métodos y estándares que se usan en las revisiones de aseguramiento de calidad;
- **adaptar** los procedimientos de aseguramiento de calidad a cada proyecto.

Evaluar cada etapa del ciclo de vida, en orden:

1. la **planificación del proyecto**;
2. los **requerimientos**;
3. el **proceso de diseño**;
4. las **prácticas de programación**;

5. el proceso de **integración y prueba**;
6. los procesos de **planificación y control** de proyectos, que acompañan todo.

La diapositiva "**Calidad de procesos en la práctica**" lo resume en cuatro consejos [Fuente: Apuntes — 12 PPQA]:

- **definir procesos estándar** (por ejemplo, cómo conducir revisiones o cómo hacer la administración de configuración);
- **monitorear** el proceso de desarrollo para asegurar que los estándares se respeten;
- **reportar** sobre el proceso a la administración de proyectos y al responsable del software;
- "**no use prácticas inapropiadas simplemente porque se han establecido los estándares**".

En Crovato:

- **Preparar el terreno.** Antes de arrancar el próximo proyecto, alguien arma un checklist para revisar requerimientos y otro para revisar código, y decide que para este proyecto también se revisa el plan de pruebas.
- **Evaluar cada etapa.** Después recorre el proyecto. ¿El plan tiene fechas realistas? ¿La ERS aclara si las cuotas tienen interés? ¿El diseño valida el precio en el servidor? ¿El código respeta el estándar? ¿Se corrió la regresión antes de integrar? ¿Se está siguiendo el plan?

> **Para un proyecto chico, el equipo decide no aplicar la revisión formal de diseño que exige el estándar de la organización. ¿Es una violación del SQA?**
> No necesariamente. La planificación de calidad consiste justamente en seleccionar y modificar los estándares para cada proyecto, y la cátedra advierte no usar prácticas inapropiadas solo porque están en el estándar. Lo que corresponde es que esa decisión quede en el plan de calidad.

---

2. Contra qué se controla: estándares y revisiones de calidad

Estándares

Los estándares son "**la clave para la administración de calidad efectiva**". Pueden ser internacionales, nacionales, organizacionales o de proyecto [Fuente: Apuntes]. Un estándar dice cómo tiene que ser **algo**, y la pregunta es si ese algo es una cosa o una forma de trabajar:

- **De producto**, para **una cosa que se entrega** (código, documento): definen las características que deben exhibir todos los componentes. Ejemplos: estilo de programación común, formato de documentos.
- **De proceso**, para **una forma de trabajar**: definen cómo deben implementarse los procesos de software. Ejemplos: cómo conducir una revisión, cómo hacer SCM.

Para qué sirven, en el orden en que se usan [resumen del compañero; coincide con Sommerville cap. 24]:

- te ahorran repetir errores, porque **capturan la experiencia** (los errores del pasado);
- te dan contra qué comparar: son una **base para evaluar** si se alcanzó la calidad;
- hacen que todos trabajen igual: **unifican** la forma de trabajo.

Revisiones de calidad

Son "**el principal método de validación de la calidad de un proceso o un producto**": un grupo examina parte de un proceso o producto, y su documentación, para encontrar potenciales problemas [Fuente: Apuntes — 12 PPQA]. Hay tipos distintos según qué se busca:

- **defectos, para sacarlos** → **inspecciones** para remoción de defectos (producto). Se desarrollan en 10-revisiones-tecnicas;
- **cuánto se avanzó** → **revisiones de progreso** (producto y proceso);
- **si se cumplen los estándares** → **revisiones de calidad** (producto y estándares).

En Crovato:

- **Los estándares.** Escribís "las funciones de precios tienen nombre en español y test unitario" (**producto**) y "todo cambio va en una rama y se revisa antes del merge" (**proceso**). Lo escribiste porque una vez una función sin test cobró mal (**captura experiencia**). Ahora podés revisar cualquier PR contra esa regla (**base para evaluar**), y quien se sume al proyecto trabaja igual que vos (**unifica**).
- **Las revisiones.** Cuando revisás `precios.ts` buscando bugs, es una **inspección**. Cuando mirás si las cuotas van a estar para el viernes, es de **progreso**. Cuando verificás que todo tenga su test, es de **calidad**.

> **Se revisa si los últimos 20 commits respetan la convención de nombres y tienen test. ¿Qué tipo de revisión es y qué estándar se controla?**
> Una revisión de calidad (producto contra estándares), y el estándar es de producto: dice cómo tiene que ser el código.

---

3. Desde dónde: dónde reporta el Grupo de Aseguramiento de Calidad

Para que el control sirva, quien controla tiene que poder decir lo que encontró. Por eso importa **a quién le reporta el GAC**: es la clave de su independencia [Fuente: Apuntes — 12 PPQA]. La pregunta que lo ordena es: si el GAC encuentra un problema grave, ¿quién podría querer taparlo?

- **El dueño del proyecto auditado.** Por eso el GAC **no debería reportar al gerente o líder de proyecto**: sería juez y parte.
- **Una cadena larga de jefes intermedios**, donde el mensaje se diluye. Por eso **no debería haber más de una posición** entre la **gerencia de primer nivel** y el GAC.
- **Alguien a quien la calidad no le importa.** Por eso, cuando sea posible, el GAC debería reportar a alguien **realmente interesado en la calidad** del software.

La misma idea de independencia reaparece en las auditorías (09-auditorias: el auditor es alguien independiente) y en testing (Myers: un programador debería evitar probar su propio código, ver 11-testing).

En Crovato: la persona que revisa la calidad descubre que se publicaron precios sin probar. Si le tuviera que informar al responsable del proyecto de la tienda, él tendría todos los incentivos para decir "no pasa nada, lo arreglamos después". Si tuviera que pasar por tres jefes antes de llegar a vos, el mensaje llegaría suavizado. Por eso te informa directamente a vos, el dueño, que sos quien pierde plata si se cobra mal.

> **En una empresa, el grupo de calidad depende del gerente del proyecto que evalúa. ¿Qué problema hay?**
> El GAC sería juez y parte: el gerente tiene incentivos para minimizar los problemas que encuentra. Debería reportar a alguien interesado en la calidad, a no más de una posición de la gerencia de primer nivel.

---

4. Por qué se trabaja sobre el proceso

El proceso basado en calidad

El diagrama de la cátedra es prueba y error aplicado a la forma de trabajar [Fuente: Apuntes — 12 PPQA]:

1. **Definir** el proceso.
2. **Desarrollar** el producto.
3. **Evaluar** la calidad del producto.
4. ¿Calidad OK?
    - **Sí** → **estandarizar** el proceso.
    - **No** → **mejorar** el proceso y repetir.

Se cambia el proceso, y no otra cosa, por el supuesto de fondo: "**el proceso es el único factor controlable** al mejorar la calidad del software y su rendimiento como organización" [Fuente: Apuntes]. En el diagrama, el **producto** está rodeado por el **proceso**, las **características del**

cliente y las **condiciones del negocio**. De esos, el único que la organización controla es el proceso. Por eso la misma diapositiva advierte que “lo que sirve a unos puede no servir a otros”.

La diapositiva “La administración de calidad y el desarrollo de software” pone, al lado del proceso de desarrollo, un **proceso de administración de calidad**, y entre los dos, tres documentos: **estándares y procedimientos**, **plan de calidad** e **informes de revisión de calidad**. Se corresponden con las tres actividades de la sección 1 (la correspondencia es propia).

Aseguramiento de proceso vs. de producto

Las dos filas de abajo del mapa de 06-calidad miran cosas distintas. La pregunta que las separa es **¿qué estás mirando?** [resumen del compañero; coherente con el mapa de la cátedra]:

	De proceso	**De producto**
Foco	Cómo se trabaja	El software resultante y sus artefactos
Pregunta	¿Seguimos el proceso acordado? ¿Es bueno?	¿El producto cumple los requerimientos?
Actividades	Definir estándares, auditorías de proyecto, revisiones de proceso, métricas de proceso, SCM	Revisiones técnicas/inspecciones, auditorías de configuración y, como control, **testing**

Procesos definidos vs. empíricos

Esto viene del resumen del compañero.

- El enfoque **definido** asume “**mejor proceso → mejor producto**”: si el proceso es de calidad y se respeta, el producto sale bien.
- El enfoque **empírico o ágil** pone el foco en **las personas** y en el feedback sobre el producto funcionando.

El supuesto definido tiene un límite. Que un proyecto siga el proceso no garantiza que cada paso se haya hecho bien, ni que el proceso sea bueno. Por eso se necesitan las dos filas, proceso y producto.

En Crovato:

- **El ciclo.** Definís “rama → test → deploy” y publicás las cuotas así. Medís: dos bugs en producción. No estás conforme, así que agregás “revisión de otra persona antes del merge” y repetís con la próxima feature. Si ahí ya no aparecen bugs, ese pasa a ser tu proceso estándar.
- **Proceso vs. producto.** Un día, alguien verifica que los últimos 10 cambios hayan pasado por revisión (**proceso**: ¿se hizo como se debía?). Otro día, alguien revisa el código de `precios.ts` buscando errores (**producto**: ¿lo que salió está bien?).

> **Un proyecto siguió el 100% del proceso definido. ¿Eso garantiza que el producto tenga calidad?**
> No. El aseguramiento de proceso verifica que se sigan los pasos, no que cada paso se haya hecho bien ni que el proceso sea bueno. Por eso también hace falta el aseguramiento de producto.

> **Los productos salen con defectos aunque el equipo cambió dos veces de tecnología. ¿Dónde conviene actuar, según la cátedra?**
> En el proceso: es el único factor controlable. El ciclo es evaluar la calidad del producto y, si no está OK, mejorar el proceso y repetir. Si sale bien, se estandariza.

Referencia rápida (las listas en limpio)

No hay contenido nuevo acá: son las mismas listas de arriba, sin explicación.

- **3 actividades**: aseguramiento (establecer estándares organizacionales) · planificación (seleccionar y adaptar para el proyecto → plan de calidad) · control (hacer respetar: revisiones de calidad + evaluaciones automáticas y mediciones).
- **Objetivos de SQA**: controlar software y proceso · asegurar el cumplimiento de estándares · informar los defectos a la gerencia.
- **Funciones del SQA**: desarrollar herramientas, técnicas, métodos y estándares · evaluar la planificación, los requerimientos, el diseño, la programación, la integración y prueba, y la planificación y control · adaptar los procedimientos a cada proyecto.
- **Reporte del GAC**: no al gerente de proyecto · no más de una posición hasta la gerencia de primer nivel · a alguien interesado en la calidad.
- **Estándares**: de producto · de proceso. Sirven para capturar experiencia, dar una base para evaluar y unificar.
- **Revisiones de calidad**: inspecciones (defectos) · de progreso · de calidad (estándares).
- **Proceso basado en calidad**: definir → desarrollar → evaluar → estandarizar o mejorar.

Preguntas de repaso

1. **¿Por qué “testing” no es lo mismo que “aseguramiento de calidad”?** — Porque el testing *controla* la calidad del producto (detecta, sobre lo ya construido) y el aseguramiento la *previene y la embebe* en el proceso (estándares, revisiones, auditorías). “El aumento de las pruebas no aumenta la calidad”. Ver Calidad Embebida.
2. **¿A quién no debe reportar el Grupo de Aseguramiento de Calidad, y por qué?** — Al gerente de proyecto: sería juez y parte. Además, no debe haber más de una posición entre el GAC y la gerencia de primer nivel.

3. **¿Qué diferencia hay entre aseguramiento, planificación y control de calidad?** — Aseguramiento escribe los estándares de la organización; planificación elige y adapta cuáles aplican a un proyecto (plan de calidad); control verifica que el equipo los respete.

Modelos y estándares de calidad (CMMI, SPICE, ISO)

> **U4 — Aseguramiento de Calidad de Proceso y de Producto** · Sección §9 del resumen completo. Índice y convención de fuentes en 00-indice. Este archivo es la columna **"Modelos"** del mapa de 06-calidad (sección 4): conviene tenerlo leído.

Cómo leer este archivo: se lee de corrido. Las listas y tablas de la cátedra aparecen una sola vez, dentro del texto. Al final hay una referencia rápida con las listas en limpio y preguntas de repaso.

Ideas transversales que acá solo se nombran:

- Verificación y Validación: en CMMI son dos áreas de proceso separadas del nivel 3, **VER** y **VAL**.
- Mejora Continua: aparece en el nivel 5 de CMMI y en el ciclo IDEAL. Tiene parientes en Lean, Kanban y la retrospectiva de Nexus, pero no son idénticas: ver la sección 3.
- **Evaluar no es auditar**: un appraisal de CMMI y una auditoría (09-auditorias) se parecen, pero tienen objetivos distintos. Está en la sección 4.

El objetivo del tema es "valorar los modelos de calidad para elegir el más adecuado". Por eso las preguntas de práctica son del tipo "una empresa quiere X, ¿qué modelo le sirve?".

> **Fuera del temario**
> La cátedra avisó que en esta instancia **no se toman los modelos de calidad de producto** (ISO 25010, McCall, Barbacci) ni la **comparación CMMI vs. Ágil**. Por eso esas dos partes se sacaron de este archivo. Lo que queda es todo de modelos de proceso.

El ejemplo que recorre el archivo: la tienda de Crovato, y las agencias que le ofrecen hacerla o mejorarla.

La idea del tema

Un modelo de calidad es **contra qué te comparás**: alguien escribió, a partir de experiencia, cómo debería ser un buen producto o un buen proceso, y vos te medís contra eso. La cátedra separa los modelos según **qué** miran [Fuente: Apuntes — 12 PPQA]:

- **Producto** → modelos **para medirlo** (ISO 25010, McCall, Barbacci). No entran en este temario; alcanza con saber que existen y que son de producto, no de proceso.
- **Proceso** → modelos **para crearlos, para mejorarlos y para evaluarlos** (CMMI, SPICE, IDEAL, ISO). Son el contenido de este archivo.

El archivo responde, en orden:

1. ¿Qué tipos de modelos de **proceso** hay? → referencia, mejora y evaluación.
2. ¿Cómo debería ser un buen proceso? → CMMI (el modelo de referencia más usado).
3. ¿Cómo hago el cambio? → los modelos de mejora: IDEAL y SPICE.
4. ¿Cómo demuestro cuánto cumplo? → evaluación y certificación: CMMI, SPICE e ISO 9001 comparados.

---

1. Los modelos de proceso: referencia, mejora y evaluación

Para el proceso, la cátedra habla de "**modelos para crearlos, para mejorarlos y para evaluarlos**" [Fuente: Apuntes — 12 PPQA]. Cada tipo responde una pregunta distinta:

- **Referencia** (para crearlos): **¿cómo debería ser** un buen proceso? → CMMI, ISO 9001.
- **Mejora** (para mejorarlos): **¿cómo hago el cambio** desde donde estoy? → IDEAL, SPICE.
- **Evaluación** (para evaluarlos): **¿cuánto cumplo?** → por ejemplo, SCAMPI para CMMI, o la auditoría de certificación ISO 9001.

[Fuente: resumen del compañero `Resumen ICW 2` y Resumen_Pedro para la clasificación; Apuntes para SPICE e IDEAL como modelos de mejora]

> **SPICE está en "mejora", aunque evalúa**
>
> La diapositiva "Mejora de Procesos" de la cátedra pone a **SPICE e IDEAL como modelos de mejora** (verificado en 12 PPQA). Al mismo tiempo, el estándar SPICE (ISO/IEC 15504) es, oficialmente, de **evaluación** de procesos (sección 3). No se contradicen: SPICE mide la capacidad de cada proceso **para saber qué mejorar**. Si en una pregunta aparece como modelo de mejora, es por la clasificación de la cátedra.

En Crovato: tres agencias te ofrecen mejorar cómo trabajás.

- La primera te muestra cómo debería ser un buen proceso de desarrollo (**referencia**).
- La segunda te propone un plan paso a paso para pasar de cómo trabajás hoy a cómo deberías trabajar (**mejora**).
- La tercera te ofrece medir tu proceso y darte una nota (**evaluación**).

> **Una empresa no sabe por dónde empezar a ordenar su forma de trabajar y quiere un plan de cambio por etapas. ¿Qué tipo de modelo necesita, y cuál de la cátedra?**
> Un modelo de mejora. IDEAL: iniciar, diagnosticar, establecer, actuar y aprender.

---

2. Cómo debería ser un buen proceso: CMMI

Qué es

[Fuente: Apuntes — 12 PPQA]

- **CMMI** (Capability Maturity Model Integration) es la evolución del **SW-CMM**.
- Lo emitió el **SEI** (Software Engineering Institute), fundado en **1984** por el **DoD de EE.UU.** junto con la universidad **Carnegie Mellon**.
- La versión 1.3 es de noviembre de 2010. Desde **2020**, el CMMI Institute pertenece a **ISACA** y la versión vigente es la **2.0**.
- Es uno de los modelos más implementados del mundo.
- **No es una norma y no se "certifica"**: se **evalúa** (appraisal) por profesionales reconocidos por el SEI, los **Lead Appraisers**.

Constelaciones: qué hace la organización

Las **constelaciones** dependen de qué hace la organización:

- **desarrolla** software → **CMMI-DEV**;
- **presta un servicio** → **CMMI-SVC**;
- **compra** software a otros → **CMMI-ACQ**.

Las tres comparten **16 áreas de proceso fundacionales**.

Representaciones: una nota para la organización o una nota por área

La pregunta que las separa es si le ponés **una nota a toda la organización** o **una nota a cada área** [Fuente: Apuntes]:

Por etapas (staged)	**Continua**
5 niveles (1 a 5) de **madurez organizacional**	**6 niveles (0 a 5)** de **capacidad** de cada **área de proceso**
Cada nivel = un conjunto de áreas de proceso	Cada área se mide por separado
Una sola clasificación → **facilita comparar organizaciones**	Permite comparar área por área

Por etapas (staged)	Continua
Secuencia probada de mejoras	**Elegís el orden** de las mejoras
Similar a SW-CMM	Similar a EIA/IS-731

Pista para no confundir: **madurez** es de la **organización**; **capacidad** es de un **proceso**. La continua arranca en **0** porque un área puede directamente no hacerse.

En Crovato: una consultora que les desarrolla tiendas online a marcas como Crovato quiere sumar clientes grandes, y los clientes grandes piden “nivel 3”. Necesita una sola nota para mostrar: **por etapas**. Otra empresa sabe que su problema es solo la gestión de configuración (pierde versiones) y quiere mejorar eso primero, sin importar el resto: le sirve la **continua**.

Los 5 niveles de madurez y sus áreas de proceso

Los niveles cuentan **cómo crece una empresa**. En cada nivel, preguntate qué le falta para dejar de sufrir el problema del nivel anterior: las áreas de proceso son la respuesta [Fuente: Apuntes — CMMI-DEV v1.3]:

Nivel	Características	Áreas de proceso
1 Inicial	Procesos impredecibles, pobremente controlados, reactivos	— (sin definir o improvisados)
2 Administrado	Procesos caracterizados **por proyecto**; a menudo reactivos	REQM (Adm. de Requerimientos), PP (Planificación de Proyectos), PMC (Monitoreo y Control), SAM (Acuerdo con Proveedores), MA (Medición y Análisis), **PPQA** (Aseg. de Calidad de Proceso y Producto), **CM** (Adm. de Configuración)
3 Definido	Procesos caracterizados **por la organización**; proactivos	RD (Desarrollo de Req.), TS (Solución Técnica), PI (Integración de Producto), **VER** (Verificación), **VAL** (Validación), OPF (Foco en el Proceso Org.), OPD (Definición del Proceso Org.), OT (Capacitación Org.), IPM (Adm. Integrada de Proyectos), RSKM (Riesgos), DAR (Análisis y Resolución de Decisiones)
4 Cuantitativamente administrado	Procesos **medidos y controlados**	QPM (Gestión Cuantitativa del Proyecto), OPP (Performance del Proceso Org.)
5 Optimizado	Foco en la **mejora continua** de procesos	CAR (Análisis Causal y Resolución), OPM (Adm. de Performance Org.)

Cómo se lee cada nivel:

- **Nivel 1, Inicial.** Todo depende de héroes, y cada proyecto es un caos distinto.
- **Nivel 2, Administrado: que cada proyecto no se descontrole.**
  - ¿Qué hay que hacer? → **REQM** (saber qué pidieron).
  - ¿Cuándo y cuánto? → **PP** (planificar) y **PMC** (seguir el plan).
  - ¿Con quién de afuera? → **SAM** (acuerdos con proveedores).
  - ¿Con qué números? → **MA** (medición).
  - ¿Cómo cuido lo que sale? → **PPQA** (calidad) y **CM** (versiones).
- **Nivel 3, Definido: que todos los proyectos trabajen igual.**
  - La organización define *su* proceso → **OPF** y **OPD**, y enseña a usarlo → **OT**.
  - La ingeniería se hace en serio, en el orden en que se construye: requerimientos → **RD**; solución → **TS**; integración → **PI**; ¿lo hice bien? → **VER**; ¿hice lo correcto? → **VAL** (Verificación y Validación).
  - La gestión madura → **IPM** (proyectos integrados al proceso de la organización), **RSKM** (riesgos) y **DAR** (decisiones con criterio).
- **Nivel 4, Cuantitativamente administrado: que se pueda predecir con números.** → **QPM** (el proyecto) y **OPP** (la organización).
- **Nivel 5, Optimizado: que se mejore solo.** ¿Por qué pasan los problemas? → **CAR** (análisis causal). ¿Cómo mejoramos el rendimiento? → **OPM**.

A medida que se sube de nivel aumentan la **productividad y la calidad**, y baja el **riesgo**. Truco para recordar: nivel 2 = “cada proyecto se gestiona”, 3 = “la organización tiene un proceso estándar”, 4 = “se mide con números”, 5 = “se mejora continuamente”.

Las cuatro categorías de áreas de proceso. La tabla de la cátedra para CMMI-DEV v1.3 ordena además las áreas en cuatro columnas [Fuente: Apuntes — 12 PPQA]:

- **Administración de proyectos**: REQM, PP, PMC, SAM (nivel 2); RSKM, IPM (3); QPM (4).
- **Soporte**: PPQA, CM, MA (2); DAR (3); CAR (5).
- **Administración de procesos**: OPD, OPF, OT (3); OPP (4); OPM (5).
- **Ingeniería**: RD, TS, PI, VER, VAL (3).

(En la diapositiva, TS aparece abreviada “TD” y OPM aparece como “OID”. Son errores de tipeo de la diapositiva o la sigla de una versión anterior; las siglas de v1.3 son TS y OPM.)

Roles o grupos: según el tamaño de la organización, cada responsabilidad puede ser un rol o un grupo. “Lo importante es que exista alguien responsable de cubrir las actividades de cada rol” [Fuente: Apuntes].

Conexión con la materia: **PPQA** (el título de esta unidad, ver 07-aseguramiento-de-calidad) y **CM** (SCM, 1er parcial) son áreas de proceso del **nivel 2**. Son lo primero que CMMI le pide a una organización.

En Crovato: empezás a hacer tiendas para otras marcas.

1. Al principio cada proyecto sale distinto y depende de que vos estés encima (**nivel 1**).
2. Te pasa que un cliente dice “yo no pedí eso”, te atrasás sin darte cuenta, un proveedor no entrega, no sabés cuánto tardás y perdés la versión que andaba. Entonces anotás requerimientos, planificás y seguís el plan, firmás acuerdos con proveedores, medís, controlás la calidad y usás Git (**nivel 2**).
3. Con cinco proyectos a la vez, cada uno trabaja a su manera. Escribís el proceso de la empresa y capacitás a todos; formalizás cómo se relevan requerimientos, se diseña, se integra, se verifica y se valida; y empezás a gestionar riesgos y a decidir con criterio (**nivel 3**).
4. Después ya podés decir “con este proceso, una tienda nos lleva 6 semanas ± 1” (**nivel 4**).
5. Finalmente, cada bug grave dispara un análisis de por qué pasó, y se cambia el proceso para que no vuelva a pasar (**nivel 5**).

Para reconstruirlo: héroes → cada proyecto controlado → todos iguales → con números → mejora sola.

> **¿En qué nivel están VER y VAL? ¿Y PPQA?**
> VER y VAL en el nivel 3 (Definido). PPQA en el nivel 2 (Administrado), junto con CM.

> **¿Qué distingue al nivel 2 del 3?**
> En el 2 los procesos se caracterizan por proyecto: cada uno se gestiona, a menudo de forma reactiva. En el 3 se caracterizan por la organización: hay un proceso estándar que todos adaptan, y son proactivos.

> **Una empresa ya gestiona bien cada proyecto, pero cada equipo trabaja a su manera y los resultados no se pueden comparar. ¿En qué nivel está y qué le falta?**
> Está en el nivel 2. Le falta el 3: definir un proceso de la organización (OPF, OPD), capacitar en él (OT) e integrar los proyectos a ese proceso (IPM).

> **Una empresa quiere mejorar solo su gestión de configuración, sin ocuparse por ahora del resto. ¿Qué representación de CMMI le conviene?**
> La continua: mide la capacidad de cada área por separado y permite elegir el orden de mejora.

---

3. Cómo se hace el cambio: modelos de mejora

La diapositiva “Mejora de Procesos” nombra dos modelos de mejora [Fuente: Apuntes — 12 PPQA]: **SPICE** e **IDEAL**.

IDEAL

IDEAL (SEI) es un ciclo de mejora en 5 fases. Son los cinco pasos de cualquier cambio serio, y el nombre ya es la lista [Fuente: Apuntes]:

- **I — Initiating** (¿por qué cambiar y quién me respalda?): el estímulo para mejorar, establecer el contexto y el **sponsorship**, la infraestructura.
- **D — Diagnosing** (¿dónde estoy y dónde quiero estar?): evaluar y caracterizar la práctica actual frente a la deseada, y hacer recomendaciones.
- **E — Establishing** (¿qué ataco primero y cómo?): prioridades y estrategias, planificar las acciones, armar los equipos de acción.
- **A — Acting** (¿funciona?): crear la solución, probarla en **pilotos**, refinarla e implementarla.
- **L — Leveraging** (¿qué aprendí?): analizar y documentar las lecciones aprendidas, y revisar el enfoque organizacional.

Es **circular**: al terminar, se vuelve a iniciar.

En Crovato: perdés ventas un sábado porque la tienda publicó un catálogo roto.

- **I.** Decidís que vas a mejorar cómo publicás, y te reservás tiempo para eso.
- **D.** Mirás cómo publicás hoy: sin tests del feed y sin CI.
- **E.** Decidís empezar por un test que valide `catalogo.json` y dejar el resto para después.
- **A.** Lo probás dos semanas en tu forma de publicar, lo ajustás, y queda como estándar.
- **L.** Anotás que el test atrapó dos errores, y volvés al principio con el próximo problema.

SPICE

La cátedra solo lo nombra; en la diapositiva figura como “Software Process Improvment Capability Evaluation” [Fuente: Apuntes]. El nombre oficial es *Software Process Improvement and Capability dEtermination*.

El detalle es externo [Fuente: Externa — ISO/IEC 15504 (Wikipedia, TechTarget); verificado en 2 fuentes]:

- Es el estándar internacional **ISO/IEC 15504** de **evaluación de procesos**, hoy reemplazado (2015) por la familia **ISO/IEC 33000**.
- Evalúa la **capacidad de cada proceso** en una escala de **6 niveles, de 0 a 5**. Es como un proceso que va madurando:
  - **0 Incompleto**: no se hace completo;
  - **1 Realizado**: se hace;
  - **2 Gestionado**: se hace y se gestiona;
  - **3 Establecido**: se hace igual en todos lados;
  - **4 Predecible**: se puede predecir;
  - **5 Optimizado**: se mejora.

- Los niveles se miden con **9 atributos de proceso**, calificados en una escala de 4 puntos (de “no alcanzado” a “totalmente alcanzado”).
- Se parece a la **representación continua** de CMMI.
- Una variante muy usada es **Automotive SPICE**, de la industria automotriz.

Qué dice la cátedra sobre mejorar

Los “**Puntos claves**” de la presentación [Fuente: Apuntes — 12 PPQA]:

- la mejora de procesos exitosa requiere **compromiso organizacional y cambio organizacional**;
- existen varios modelos disponibles para dar soporte a los esfuerzos de mejora;
- la mejora de procesos ha demostrado **retornos de inversión sustanciales**.

> **Mejora continua: misma familia, distintas escalas**
> El nivel 5 de CMMI (“foco en la mejora continua”) y el ciclo circular de IDEAL son mejora continua **del proceso de la organización**. Lean (amplificar el aprendizaje), Kanban (mejorar en forma colaborativa) y la retrospectiva de Nexus o Scrum también mejoran continuamente, pero **el flujo o el trabajo del equipo**, con ciclos más cortos. La idea de fondo es la misma; lo que cambia es qué se mejora y cada cuánto. Ver Mejora Continua. (La comparación es propia.)

> **En IDEAL, ¿en qué fase se prueba la solución en un piloto? ¿Y en cuál se consigue el sponsor?**
> El piloto es en Acting. El sponsor, en Initiating.

> **Una empresa ya mejoró su proceso de testing y quiere arrancar con el de requerimientos. Según IDEAL, ¿qué hace antes de volver a empezar?**
> Leveraging: analizar y documentar las lecciones aprendidas, y revisar el enfoque. Después el ciclo vuelve a Initiating.

---

4. Cómo se demuestra cuánto se cumple: evaluación y certificación

ISO para procesos

La cátedra solo nombra la familia ISO/IEC 25000 (producto). Para el lado de **proceso o sistema de gestión** [Fuente: Externa — ISO/IEC/IEEE 90003:2018; ISO 9001:2015]:

- **ISO 9001**: norma de **sistemas de gestión de calidad** para **cualquier tipo de organización** (no es específica de software). **Sí se certifica**: un organismo certificador externo audita a la organización contra ISO 9001.
- **ISO/IEC/IEEE 90003:2018**: **guía para aplicar ISO 9001 al software** (adquisición, suministro, desarrollo, operación y mantenimiento). **No agrega requisitos** ni es una norma de certificación: se certifica contra ISO 9001.
- **ISO/IEC 15504 / 33000**: el SPICE de la sección 3.

CMMI, SPICE e ISO 9001 comparados

Si una empresa te dice “tenemos calidad”, ¿qué te muestra para probarlo? (Síntesis propia.)

	CMMI	**SPICE (ISO/IEC 15504/33000)**	**ISO 9001**
Qué es	Modelo de madurez/capacidad (SEI/ISACA)	Estándar de evaluación de procesos	Norma de sistema de gestión de calidad
Específico de software	Sí (DEV)	Sí en origen (hoy general)	No (guía 90003 para software)
Escala	1-5 madurez / 0-5 capacidad	0-5 capacidad por proceso	Cumple / no cumple
¿Se certifica?	**No**, se evalúa (Lead Appraiser)	Se evalúa	**Sí**, certificación
Lo que te muestran	“Estamos en nivel 3”	“Nuestro proceso de testing está en capacidad 4”	“Estamos certificados ISO 9001”

En Crovato, tres agencias le ofrecen a Crovato hacerle la tienda:

- La primera muestra “CMMI nivel 3”: un evaluador revisó sus proyectos y les dio esa nota.
- La segunda dice: “nuestro proceso de testing está en capacidad 4 según ISO 15504”. Es una nota solo de ese proceso.
- La tercera muestra un certificado ISO 9001 colgado en la pared, el mismo que podría tener una fábrica de galletitas.

Evaluar no es auditar

Las dos palabras se cruzan: la definición IEEE de auditoría empieza con “evaluación independiente” (09-auditorias), y la certificación ISO 9001 se hace con una auditoría. La diferencia está en **para qué** se hace cada una (la distinción es propia, a partir de las definiciones de los dos temas):

- **Evaluación de un modelo** (appraisal de CMMI, evaluación SPICE): compara los procesos de la organización contra el modelo y **da un nivel** de madurez o capacidad.

- **Auditoría**: verifica, con un criterio objetivo escrito de antemano, que un proyecto o una configuración **cumple** con los estándares, procedimientos y especificaciones acordados. Da hallazgos (cumple o no cumple), no un nivel.

> **¿Cuál de los tres se certifica?**
> Solo ISO 9001. CMMI y SPICE se evalúan.

> **Una empresa de software quiere venderle a una multinacional que exige "un certificado de calidad reconocido internacionalmente". ¿Qué le sirve?**
> ISO 9001 (aplicada al software con la guía 90003): es la única que se certifica. Un nivel de CMMI también es reconocido, pero es una evaluación, no un certificado.

> **Una consultora quiere demostrar a sus clientes que tiene un proceso de desarrollo maduro y poder compararse con otras consultoras de software. ¿Qué le conviene?**
> CMMI-DEV por etapas: da una sola nota de madurez de la organización, pensada para software, que facilita comparar organizaciones.

Referencia rápida (las listas en limpio)

No hay contenido nuevo acá: son las mismas listas de arriba, sin explicación.

- **Modelos según qué miran**: producto (medirlo: ISO 25010, McCall, Barbacci) · proceso (crearlo, mejorarlo, evaluarlo: CMMI, SPICE, IDEAL, ISO).
- **Modelos de proceso**: referencia (CMMI, ISO 9001) · mejora (IDEAL, SPICE) · evaluación (SCAMPI, auditoría de certificación ISO 9001).
- **CMMI, constelaciones**: DEV · SVC · ACQ (16 áreas fundacionales comunes).
- **CMMI, representaciones**: por etapas (madurez 1-5) · continua (capacidad 0-5).
- **CMMI, niveles**: 1 Inicial · 2 Administrado · 3 Definido · 4 Cuantitativamente administrado · 5 Optimizado.
- **CMMI, categorías de áreas**: administración de proyectos · soporte · administración de procesos · ingeniería.
- **IDEAL**: Initiating · Diagnosing · Establishing · Acting · Leveraging.
- **SPICE, niveles**: 0 Incompleto · 1 Realizado · 2 Gestionado · 3 Establecido · 4 Predecible · 5 Optimizado.
- **¿Se certifica?**: ISO 9001 sí · CMMI y SPICE no (se evalúan).

Preguntas de repaso

1. **CMMI por etapas vs. continua.** — Etapas: 5 niveles (1-5) de madurez de la organización, secuencia de mejora fija, compara organizaciones. Continua: 6 niveles (0-5) de capacidad por área de proceso, elegís el orden.
2. **¿En qué nivel de CMMI están PPQA y Gestión de Configuración?** — Nivel 2, Administrado.
3. **¿CMMI se certifica? ¿ISO 9001?** — CMMI no: se evalúa con un Lead Appraiser. ISO 9001 sí se certifica.
4. **¿Qué diferencia hay entre un modelo de referencia, uno de mejora y uno de evaluación?** — Referencia: cómo debería ser un buen proceso (CMMI, ISO 9001). Mejora: cómo hacer el cambio (IDEAL, SPICE). Evaluación: cuánto cumplís (SCAMPI, auditoría de certificación ISO).

Auditorías de software

> **U4 — Aseguramiento de Calidad de Proceso y de Producto** · Secciones §10 y §11 del resumen completo. Índice y convención de fuentes en 00-indice. En el mapa de 06-calidad, las auditorías están en la columna de **aseguramiento**: de proyecto (fila proyecto) y de configuración (fila producto). Son un instrumento del SQA de 07-aseguramiento-de-calidad.

Cómo leer este archivo: se lee de corrido. Las listas de la cátedra aparecen una sola vez, dentro del texto. Al final hay una referencia rápida con las listas en limpio y preguntas de práctica.

Las ideas que se repiten en otros temas tienen nota propia, y acá solo se nombran:

- Independencia: el auditor es de afuera del proyecto, por la misma razón que el GAC no reporta al líder de proyecto y que un programador no debería probar su propio código.
- Verificación y Validación: la auditoría de proyecto revisa que cada requerimiento tenga cómo validarse en el Plan de V&V.

El ejemplo que recorre el archivo: una agencia le hace la tienda a Crovato, y Juan, de la misma agencia pero de afuera del proyecto, la audita.

La idea del tema

Una organización define cómo hay que trabajar (proceso), qué hay que construir (requerimientos) y cómo se documenta. La auditoría es la forma de **comprobar que eso se cumplió**, con alguien **independiente** que compara contra un **criterio escrito de antemano**.

El tema responde, en orden:

1. ¿Qué es una auditoría, para qué sirve y en qué se diferencia de una revisión técnica?
2. ¿Qué se compara contra qué? → los 3 tipos: de proyecto, funcional (FCA) y física (PCA). Y la auditoría al grupo de calidad.
3. ¿Quién hace qué? → gerente de SQA, auditor y auditado.
4. ¿Cómo se hace? → las 4 etapas, el checklist, la conducta del auditor y las técnicas.
5. ¿Qué sale de una auditoría? → buenas prácticas, desviaciones y observaciones; el reporte y las métricas.

---

1. Qué es una auditoría y para qué sirve

Definición (IEEE Std 1028): "**evaluación independiente** de los productos o procesos de software para asegurar el cumplimiento con estándares, lineamientos, especificaciones y procedimientos, **basada en un criterio objetivo**, incluyendo documentación que especifique: (1) la forma o contenido de los productos a desarrollar, (2) el proceso por el cual se desarrollan, (3) cómo debería medirse el cumplimiento" [Fuente: Apuntes].

Leída con calma, la definición dice qué tiene que existir **antes** de auditar. Para juzgar "objetivamente" hace falta tener escrito:

- **qué** se tenía que producir (la forma o contenido de los productos);
- **cómo** se tenía que producir (el proceso);
- **cómo se mide** si se cumplió.

¿Por qué auditar? [Fuente: Apuntes]. Se ordena por a quién le sirve:

- **A todos**, porque es una mirada de afuera: da una **opinión objetiva e independiente**.
- **Al cliente**: identifica áreas de **insatisfacción potencial del cliente**, y permite **asegurarle** que se cumplen sus expectativas.
- **A la organización**: identifica **oportunidades de mejora**.

Beneficios [Fuente: Apuntes]. Son esas mismas razones vistas desde la empresa:

- evaluar el cumplimiento del proceso de desarrollo;
- determinar la implementación efectiva del **proceso organizacional**, del **proceso del proyecto** y de las **actividades de soporte**;
- dar **visibilidad a la gerencia**.

El resultado es una cadena: mejores productos → clientes satisfechos → crecimiento del negocio.

En Crovato: a mitad del proyecto llega Juan, que no es del equipo (**independiente**), con el proceso escrito de la agencia, la ERS y una lista de qué revisar (**criterio objetivo, escrito de antes**). Encuentra que el pago en 6 cuotas no tiene casos de prueba (**insatisfacción potencial**). Te muestra a vos el informe para que veas que el resto está bien (**asegurar al cliente**). Y la agencia aprende que necesita una plantilla de casos de prueba para pagos (**mejora**).

Auditoría vs. revisión técnica

Esta es la única vez que se explica la diferencia; 10-revisiones-tecnicas remite acá.

	Revisión técnica	**Auditoría**
Quién	**Pares** (compañeros, otros roles del proyecto)	Alguien **independiente**, externo al proyecto

	Revisión técnica	Auditoría
Qué busca	**Defectos en un artefacto**	**Cumplimiento** (de proceso, de requerimientos, de documentación)
Contra qué	El conocimiento de los revisores, checklists	Un **criterio objetivo** escrito de antemano

La independencia es la de Independencia: quien revisa su propio trabajo, o depende de quien es auditado, tiene incentivos para no ver los problemas.

> **Dos programadores leen juntos el código del checkout buscando bugs. ¿Es una auditoría?**
> No. Es una revisión técnica: la hacen pares y buscan defectos en un artefacto. Una auditoría la hace alguien independiente y busca cumplimiento contra un criterio objetivo.

---

2. Qué se compara contra qué: los tipos de auditoría

La cátedra da **3 tipos** [Fuente: Apuntes]. Lo que los distingue es **qué se compara contra qué**.

Auditoría de proyecto: cómo se trabajó contra el proceso definido

Valida el cumplimiento del proceso de desarrollo: "¿hicimos lo que dijimos que íbamos a hacer?". Se hace **durante** el proyecto, según lo establecido en el **PACS** (*Plan de Aseguramiento de Calidad de Software*), que indica quién la hace. Incluye inspecciones y revisiones de la documentación de diseño y de prueba.

Su objetivo es verificar la **consistencia del producto a medida que evoluciona**. Las verificaciones siguen **la cadena de documentos**, en el orden en que se producen:

- los requerimientos funcionales de la **ERS** (Especificación de Requerimientos de Software) se validan en el **Plan de V&V** (¿cada requerimiento tiene cómo validarse?);
- el diseño (**DDS**, Documento de Diseño de Software) satisface los requerimientos de la ERS;
- el **código es consistente con el DDS**;
- y además, las interfaces HW/SW son consistentes con los requerimientos de diseño de la ERS.

Auditoría de Configuración Funcional (FCA): lo que hace contra lo que se pidió

Valida que el producto cumpla con sus requerimientos. Compara el software construido (ejecutables + documentación) contra la **ERS**. Su propósito es que el código implemente **sólo y completamente** los requerimientos: ¿hace lo que se pidió, ni más ni menos? QA verifica que la **matriz de rastreabilidad** esté actualizada.

Auditoría de Configuración Física (PCA): lo que se entrega contra lo que está documentado

Valida que el ítem de configuración, tal como está construido, cumpla con la documentación técnica que lo describe. Compara el **código con la documentación de soporte**, para que lo que se entrega esté bien documentado: ¿lo que entrego es lo que dice la documentación? El software **solo puede entregarse cuando se arreglaron las desviaciones**.

El diagrama de SCM que resume las dos de configuración [Fuente: Apuntes — 02 SCM]: **Requerimientos** —(FCA: consistente con)→ **Producto construido** —(PCA: consistente con)→ **Documentación**.

En Crovato, la agencia hace las tres:

- A mitad del proyecto, alguien revisa si se hicieron las revisiones de código que el proceso exige y si el diseño del checkout cubre lo que dice la ERS (**proyecto**).
- Antes de entregar, otro toma la ERS y la matriz de rastreabilidad, y verifica que estén las cuotas de 1, 3 y 6, y que nadie haya agregado 12 cuotas que no se pidieron (**FCA**).
- Por último, alguien compara la versión que se va a entregar con el documento técnico. Si el documento dice que el Worker valida el precio contra el feed, el código tiene que hacerlo (**PCA**).

Auditorías al Grupo de Calidad

El temario las nombra, pero **ninguna presentación las desarrolla**. Interpretación razonable (no confirmada contra material de la cátedra): es auditar al propio GAC/SQA — verificar que el grupo de calidad cumple *su* proceso (que el plan de auditorías exista y se ejecute, que los reportes se emitan, que las no conformidades se sigan hasta cerrarse, que sus auditores sean independientes y estén capacitados). Responde a "¿quién controla al que controla?". En CMMI esto aparece como la práctica genérica de que **también el proceso de PPQA sea evaluado objetivamente** y revisado por la alta gerencia; en ISO 9001 equivale a la auditoría interna del propio sistema de gestión de calidad. Si en clase se dio una definición, pasámela y lo ajusto.

> **¿Qué auditoría usa la matriz de rastreabilidad, y para qué?**
> La FCA: para verificar que el producto implemente solo y completamente los requerimientos de la ERS.

> **Antes de entregar, se descubre que el manual técnico describe una validación de precios que el código no hace. ¿Qué auditoría lo detecta y qué pasa con la entrega?**
> La PCA (configuración física): compara el código con la documentación técnica. El software solo puede entregarse cuando se arreglan las desviaciones.

> **El producto cumple todos los requerimientos, pero se saltearon las revisiones de código que exige el proceso. ¿Qué auditoría lo encuentra?**
> La auditoría de proyecto: valida el cumplimiento del proceso ("¿hicimos lo que dijimos que íbamos a hacer?"). La FCA pasaría, porque el producto cumple la ERS.

---

3. Quién hace qué: roles y responsabilidades

[Fuente: Apuntes] Una auditoría tiene un **organizador**, alguien que **mira** y alguien a quien **miran**:

Gerente de SQA (organiza)	**Auditor (mira)**	**Auditado (lo miran; suele ser el líder de proyecto)**
Prepara el **plan de auditorías**	Acuerda la fecha	Acuerda la fecha
Calcula el costo	**Comunica el alcance**	Participa de la auditoría
Asigna los recursos	**Recolecta y analiza evidencia objetiva** relevante y suficiente	**Proporciona evidencia**
Responsable de **resolver las no-conformidades**	Realiza la auditoría	Contesta el reporte
	Prepara el reporte	**Propone el plan de acción** para las deficiencias
	Seguimiento de los planes de acción	Comunica el cumplimiento del plan de acción

Lo que hace cada uno sale de su papel:

- **El que organiza** (gerente de SQA) hace lo que una actividad necesita para existir: plan, costo y recursos. Como es el responsable final, también resuelve las no conformidades.
- **El que mira** (auditor) hace todo lo que es **observar y contar**: comunicar el alcance, recolectar y analizar evidencia, hacer el reporte y seguir los planes de acción.
- **Al que miran** (auditado) le toca todo lo que es **mostrar y responder**: dar evidencia, contestar el reporte, proponer el plan de acción y contar que lo cumplió.
- Auditor y auditado **acuerdan la fecha**: la auditoría no es una sorpresa.

El auditor debe ser **externo al proyecto** (Independencia; se desprende de "evaluación independiente" en la definición).

En Crovato: la gerente de calidad de la agencia arma el calendario de auditorías del año, calcula cuánto cuesta y asigna a Juan para auditar el proyecto (**organiza**). Juan acuerda la fecha con la líder del proyecto, le explica qué va a revisar, pide los PR y la ERS, y después escribe el informe (**mira**). La líder le muestra los PR, responde, y cuando aparece una desviación propone ella cómo arreglarla y avisa cuando está hecho (**la miran**). Juan revisa que el arreglo se haya hecho de verdad.

> **¿Quién propone el plan de acción ante una desviación: el auditor o el auditado?**
> El auditado lo propone; el auditor lo revisa, lo aprueba y hace el seguimiento.

> **La líder de un proyecto propone auditarlo ella misma "porque es quien mejor lo conoce". ¿Qué está mal?**
> Que sería juez y parte: el auditor tiene que ser externo al proyecto. Ella es la auditada.

---

4. Cómo se hace: el proceso de auditoría

Las 4 etapas

[Fuente: Apuntes] Son como ir al médico a hacerte un chequeo: **sacás turno**, **te revisa**, **te da el resultado** y **volvés a control**.

1. Preparación y planificación (sacar turno). Es conjunta entre auditor y auditado: **no hay sorpresas**.

- El auditado **solicita la auditoría** y propone asistentes.
- El auditor solicita y revisa la documentación preliminar.
- Juntos **acuerdan lugar, fecha, alcance, objetivo y participantes**.
- El auditado prepara la documentación requerida.

2. Ejecución (te revisa).

- **Reunión de apertura**: el auditor presenta a los participantes y al equipo auditor, informa el proceso y el carácter o formalismos de la auditoría, pide cooperación y aclara dudas. Se revisan el cronograma, el alcance y el objetivo.
- **Ejecución propiamente dicha**: el auditor **pregunta, solicita evidencia y completa el checklist**; el auditado **contesta y muestra evidencia**. El auditor informa la fecha de envío del informe.

3. Análisis y reporte del resultado (te da el resultado).

- **Evaluación de resultados**: el auditor revisa el checklist, la documentación y sus anotaciones, hace una **verificación cruzada de las desviaciones contra el proceso**, y arma el **informe preliminar**.
- **Reunión de cierre**: presenta el reporte preliminar y explica los resultados. **Se acuerdan** los resultados (el auditado puede pedir aclaraciones) y se define la fecha del reporte final.
- **Entrega del reporte final**: se actualiza con lo acordado, se guarda una copia y se envía al auditado.

4. Seguimiento (volvés a control).

- El auditado analiza las desviaciones, **prepara el plan de acción** (con fechas) y lo envía.
- El auditor lo **revisa y aprueba**.
- El auditado lo ejecuta e informa.
- El auditor **verifica el cumplimiento y cierra las desviaciones**.

En Crovato:

1. La líder del proyecto pide la auditoría y acuerda con Juan el martes a las 10, para revisar el checkout (**turno**).
2. El martes, Juan explica cómo va a ser, pide los PR y la ERS, y va marcando su checklist (**revisión**).
3. El jueves le muestra un borrador: “3 PR sin revisión”. Ella aclara que uno sí tuvo revisión, pero por chat. Acuerdan que son 2 y sale el informe final (**resultado**).
4. Ella propone “proteger la rama main” con fecha el viernes, Juan lo aprueba, y la semana siguiente verifica que esté hecho y cierra (**control**).

Las herramientas del auditor

El checklist garantiza que, sin importar quién audite, se controle lo mismo [Fuente: Apuntes]. Tiene un **encabezado** (fecha, lista de auditados con su rol, auditor, proyecto, fase actual), el **objetivo y alcance**, y la **lista de preguntas**. Ejemplos de preguntas:

- *Planificación*: ¿existe un plan de proyecto?, ¿está actualizado?, ¿hay un responsable por actividad?, ¿se asignaron recursos a actividades de soporte?, ¿están disponibles para todos? El auditor verifica además que los planes se basen en requerimientos, que los cambios estén aprobados y documentados y que los **riesgos** estén identificados y comunicados.
- *Requerimientos*: ¿existe la ERS?, ¿cada requerimiento está identificado unívocamente y descripto?, ¿fueron revisados y aprobados?, ¿los cambios siguieron el proceso de cambios y actualizaron los planes?

Las técnicas y herramientas responden a cómo revisar mucho en poco tiempo [Fuente: Apuntes]:

- con una lista: **checklist**;
- mirando una muestra y no todo: **muestreo**;
- revisando lo que quedó escrito: **revisión de registros**;
- dejando que una herramienta cuente: **herramientas automatizadas**.

La conducta del auditor apunta a entender bien lo que le dicen [Fuente: Apuntes]:

- **escuchando**: sin interrumpir, y observando el lenguaje corporal (ajeno y propio);
- **registrando**: tomando notas;
- **confirmando**: preguntando, y **repitiendo lo escuchado**.

La **técnica de cuestionario** es un embudo. Se empieza con preguntas **abiertas** (quién, cuándo, cómo, qué, dónde), se sigue con preguntas cortas y puntuales, y se termina con preguntas **cerradas** para clarificar.

Las reacciones comunes del auditado son las de alguien que se siente evaluado [Fuente: Apuntes]:

- se **pone nervioso**: ansiedad, sentirse examinado, enojo o nervios;
- **quiere quedar bien**: querer impresionar, **dar demasiada información**;
- **aprovecha para descargarse**: usar la auditoría para quejarse de la empresa.

En Crovato: Juan llega con su checklist. La líder, nerviosa, le cuenta todo lo que hicieron en el año (**dar demasiada información**) y aprovecha para quejarse de que nunca le dan tiempo (**descargarse**). Juan la escucha sin cortarla, anota, y le pregunta: "¿Cómo deciden qué PR se revisan?" (**abierta**). Al final confirma: "Entonces, ¿todos pasan por revisión antes del merge?" (**cerrada**). No revisa los 200 PR: elige 10 (**muestreo**), mira el historial de Git (**registros**) y corre un script que lista los merges sin aprobación (**herramienta**).

> **El auditor arranca la entrevista con "¿Todos los PR pasan por revisión, sí o no?". ¿Qué está haciendo mal?**
> Empieza por una pregunta cerrada. La técnica de cuestionario va de abiertas (cómo, quién, qué) a cerradas, que se usan al final para clarificar.

---

5. Qué sale de una auditoría

Los 3 tipos de resultado

[Fuente: Apuntes] La pregunta que los ordena es: comparado con lo esperado, ¿cómo está?

- **Mucho mejor → buena práctica**: algo hecho **mucho mejor de lo esperado**. Se reserva para cuando el auditado estableció un sistema efectivo, logró un alto grado de conocimiento o cooperación interna, o adoptó una práctica superior a cualquier otra vista. En el informe suelen ir primero.
- **Peor, y ya es un problema → desviación** (no conformidad): **requiere plan de acción**. Es cualquier desvío que haga que el producto no cumpla sus requerimientos, la falta de control para satisfacerlos, o un desvío del proceso definido que genere incertidumbre sobre la calidad.
- **Todavía no es un problema, pero puede serlo → observación**: **no requiere plan de acción**. Es una opinión sobre una condición incumplida, una práctica a mejorar, o una condición que **puede resultar en una futura desviación**.

El reporte y las métricas

El **reporte de auditoría** tiene el mismo encabezado que el checklist (identificación, fecha, auditor, auditados, proyecto, fase), pero su cuerpo trae lo que se **encontró**: la **lista de resultados**, comentarios y la **solicitud de planes de acción** [Fuente: Apuntes]. El checklist es el antes; el reporte, el después.

Métricas de auditoría (ejemplos) [Fuente: Apuntes]:

- esfuerzo por auditoría;
- cantidad de desviaciones;
- duración;
- desviaciones clasificadas por área de proceso de CMMI.

Puntos claves de la cátedra:

- las auditorías al proceso son tres: de proyecto, de configuración física y de configuración funcional;
- implican esfuerzo y costo, **pero sus beneficios son superiores**;
- son un instrumento del aseguramiento de calidad.

Ejemplo para fijar: una empresa dice en su proceso que “todo cambio a producción pasa por code review”.

- En la auditoría de proyecto, el auditor **muestrea** 10 pull requests y encuentra 3 mergeados sin aprobación → **desviación** (plan de acción: proteger la rama main).
- Ve que los tests se corren en local pero no en CI → **observación** (riesgo futuro).
- Ve que el equipo documenta las decisiones de arquitectura en ADRs mejor que cualquier otro → **buena práctica**.

> **El auditor ve una práctica que no incumple nada, pero que podría generar una desviación en el futuro. ¿Qué resultado es y qué requiere?**
> Una observación. No requiere plan de acción.

> **Se encontró que un requerimiento de la ERS no está implementado. ¿Qué resultado es y qué pasa después?**
> Una desviación: el producto no cumple sus requerimientos. Requiere plan de acción: el auditado lo propone con fechas, el auditor lo aprueba, verifica que se cumpla y cierra la desviación.

Referencia rápida (las listas en limpio)

No hay contenido nuevo acá: son las mismas listas de arriba, sin explicación.

- **Lo que la definición exige tener escrito**: forma o contenido de los productos · proceso · cómo medir el cumplimiento.
- **Por qué auditar**: opinión objetiva e independiente · insatisfacción potencial del cliente · asegurar al cliente · oportunidades de mejora.
- **Beneficios**: cumplimiento del proceso · implementación del proceso organizacional, del proyecto y de soporte · visibilidad a la gerencia.
- **Tipos**: de proyecto (proceso) · FCA (producto vs. ERS) · PCA (código vs. documentación). Más la auditoría al grupo de calidad (sin desarrollo de la cátedra).
- **Roles**: gerente de SQA · auditor · auditado.
- **Etapas**: preparación y planificación · ejecución (apertura + ejecución) · análisis y reporte (evaluación + cierre + reporte final) · seguimiento.
- **Técnicas**: checklist · muestreo · revisión de registros · herramientas automatizadas.
- **Conducta del auditor**: escuchar sin interrumpir · lenguaje corporal · tomar notas · preguntar · repetir lo escuchado. Preguntas abiertas → cerradas.
- **Reacciones del auditado**: impresionar · ansiedad · sentirse examinado · quejarse de la empresa · dar demasiada información · enojo o nervios.
- **Resultados**: buenas prácticas · desviaciones (con plan de acción) · observaciones (sin plan de acción).
- **Métricas**: esfuerzo por auditoría · cantidad de desviaciones · duración · desviaciones por área de proceso CMMI.

Preguntas de repaso

1. **Diferencia entre auditoría funcional y física de configuración.** — Funcional: producto vs. requerimientos (ERS, matriz de rastreabilidad), “hace sólo y completamente lo pedido”. Física: código vs. documentación técnica, “lo entregado está bien descripto”.
2. **¿Qué tipo de resultado de auditoría requiere plan de acción?** — Solo las desviaciones. Las observaciones no; las buenas prácticas se destacan.
3. **¿En qué se diferencia una auditoría de una revisión técnica?** — La revisión la hacen pares y busca defectos en un artefacto. La auditoría la hace alguien independiente y busca cumplimiento contra un criterio objetivo.

Revisiones técnicas

> **U4 — Aseguramiento de Calidad de Proceso y de Producto** · Sección §12 del resumen completo. Índice y convención de fuentes en 00-indice. En el mapa de 06-calidad, las revisiones técnicas están en las tres filas (proceso, proyecto y producto), en la columna de **aseguramiento**. La diferencia con una auditoría está explicada una sola vez, en 09-auditorias.

Cómo leer este archivo: se lee de corrido. Las listas de la cátedra aparecen una sola vez, dentro del texto. Al final hay una referencia rápida con las listas en limpio y preguntas de práctica.

Las ideas que se repiten en otros temas tienen nota propia, y acá solo se nombran:

- Verificación y Validación: las revisiones son la mitad **estática** de V&V; el testing es la mitad dinámica.
- Error y Defecto: qué encuentra una revisión según dónde nació la falla.
- Calidad Embebida: el costo de corregir crece con cada etapa; prevenir es mejor que curar.
- Testing Temprano: detectar los defectos lo más cerca posible de donde se generan.
- Independencia: revisan los pares, no el autor solo.

El ejemplo que recorre el archivo: el proyecto “pagar en cuotas” de la tienda de Crovato, pero esta vez revisando los documentos y el código **antes** de ejecutar nada.

La idea del tema

Las personas se equivocan, y cada equivocación cuesta más cuanto más tarde se encuentra. Las revisiones técnicas son la forma de encontrar esas fallas **sin ejecutar nada**: alguien que no es el autor lee el producto de trabajo (ERS, diseño, código, plan) y busca problemas. Por eso se pueden hacer desde el primer documento.

El tema responde, en orden:

1. ¿Por qué hay fallas y por qué conviene encontrarlas temprano? → V&V, causas de las fallas, gravedad, costo y principios.
2. ¿Qué es una revisión técnica, y cuánto cuesta? → definición, ventajas, desventajas y costos.
3. ¿Qué tan formal tiene que ser? → walkthrough vs. inspección, y quién revisa qué.
4. ¿Cómo se hace una inspección? → roles, etapas, tasas, métricas y puntos clave.

---

1. Por qué hay fallas y por qué encontrarlas temprano

Dónde encaja: la mitad estática de V&V

[Fuente: Apuntes — 13 Revisiones Técnicas] V&V compara el producto contra lo especificado (**verificación**: ¿lo construimos **correctamente**?) y contra la necesidad real del usuario (**validación**: ¿construimos **el producto correcto**?). El detalle está en Verificación y Validación.

Lo que agrega este tema:

- V&V es un proceso de **ciclo de vida completo**: empieza con revisiones de requerimientos, sigue con revisiones de diseño e inspecciones de código, y termina en la prueba. **Debe planificarse**, es caro, y conviene **comenzar temprano**.
- Tiene **dos aproximaciones complementarias**: las **revisiones técnicas** (estáticas, no ejecutan) y las **pruebas de software** (dinámicas, ejecutan: 11-testing) [Fuente: Apuntes].
- **Falla** es un error en un **producto de trabajo**, es decir, en la salida de cualquier actividad del ciclo de vida: ERS, diseño, código, plan de prueba… Por eso todo producto de trabajo se puede revisar.

Por qué existen las fallas

La cátedra da tres causas [Fuente: Apuntes]. Se ordenan por **dónde se pierde la información** en el camino entre lo que alguien quiere y lo que termina en el código:

- **Entre personas → ruido de comunicación**. Los caminos crecen como n(n−1)/2: 3 personas = 3 caminos, 5 = 10, 10 = 45 (el mismo argumento de Brooks del 1er parcial).
- **Dentro de una cabeza → limitaciones de memoria**: la memoria de corto plazo maneja **7 ± 2** elementos.
- **En el problema mismo** → “las fallas más persistentes están relacionadas con la **complejidad inherente** al producto” (Robert Glass).

En Crovato: le explicás las cuotas a la diseñadora, ella al programador y él a la tester, y en cada pasada se pierde un detalle, como que las 6 cuotas no tienen interés (**comunicación**). El programador tiene que acordarse a la vez de las cuotas, los sets en oferta, el envío, el stock y los redondeos, y se le escapa uno (**memoria**). Y aunque nadie se equivocara, combinar descuentos con cuotas y envío es difícil de por sí (**complejidad**). Si la tienda cobra lo que dice la ERS pero tus clientes querían pagar por transferencia, está **verificada pero no validada**.

Qué tan grave es lo que se encuentra

Si la falla nació en el producto que se está revisando, es un **error**. Si nació en un producto anterior y se escapó hasta acá, es un **defecto**. Por ejemplo, un bug en el código causado por un requerimiento ambiguo es un defecto cuando aparece en el código [Fuente: Apuntes]. La definición completa está en Error y Defecto.

En revisiones, cada falla se clasifica por **gravedad**, en tres niveles [Fuente: Apuntes]. La pregunta que los ordena es: si nadie lo arregla, ¿qué pasa cuando el sistema funciona?

- **Mayor**: puede fallar o dar un resultado inesperado. Es decir, puede causar una **falla operacional**. Ejemplos: en código, un error lógico; en requerimientos, una expresión ambigua o información faltante; en un plan de prueba, una condición que haría que no se detecten fallas.
- **Menor**: no falla, pero no está bien. Es no deseable pero **no causa falla operacional**. Ejemplos: violación de estándares de codificación (comentarios), requerimiento **no testeable**, información redundante en casos de prueba.
- **Cosmética**: no pasa nada, es de forma. Ejemplos: tipeo, ortografía, gramática, plantilla vieja, historia de revisiones desactualizada, copyright, sugerencia alternativa.

Ojo: no es la misma escala que la **severidad** de 11-testing, que tiene 5 niveles (bloqueante, crítico, mayor, menor, cosmético) y se complementa con la prioridad. Son dos clasificaciones de la cátedra para dos técnicas distintas.

En Crovato: revisando la ERS encontrás que RF-07 no dice si las cuotas tienen interés. Lo encontraste en la ERS, que es donde nació, así que es un **error**; y como puede hacer que el sistema cobre mal, es **mayor**. Si nadie lo ve y aparece cuando revisan el código, ese mismo problema ahora es un **defecto**. En el mismo documento hay un requerimiento que dice “la tienda debe ser rápida”: no hace fallar nada, pero no se puede probar, así que es **menor**. Y “cuoatas” mal escrito es **cosmético**.

Cuánto cuesta encontrarla tarde

Reparar una falla cuesta **1** en requerimientos y crece hasta **~100** en release (escala logarítmica, Gilb & Graham 1993). El **retrabajo evitable es el 40-50%** del desarrollo [Fuente: Apuntes]. Es el argumento de Calidad Embebida y de Testing Temprano.

De ahí salen los **6 principios de V&V** [Fuente: Apuntes], que responden tres preguntas:

- **¿Cuándo actuar?** Antes de que la falla exista, o lo antes posible:
  - la **prevención es mejor que la cura**;
  - **evitar es más efectivo que eliminar**.
- **¿Dónde poner el esfuerzo?** No en todo por igual:
  - **priorizar lo rentable**;
  - olvidarse de la perfección.
- **¿Cómo lograr que no se repita?** Que la gente aprenda:
  - la retroalimentación enseña efectivamente;
  - **enseñar a pescar** en lugar de dar el pescado.

En Crovato: en vez de esperar a que un cliente se queje de un cobro mal hecho, revisás la ERS antes de programar (**prevenir**), porque arreglarlo en requerimientos cuesta 1 y en producción

cerca de 100 (**evitar antes que eliminar**). No revisás con la misma lupa el texto de “Quiénes somos” que el checkout (**priorizar lo rentable**), y aceptás que algún bug cosmético va a llegar a producción (**olvidar la perfección**). Cuando encontrás un error en el código de un compañero, se lo mostrás para que entienda por qué estaba mal (**retroalimentación**), y le pasás el checklist de revisión para que la próxima vez lo detecte él mismo (**enseñar a pescar**).

> **¿Cuántos caminos de comunicación hay en un equipo de 6 personas?**
> 6 × 5 / 2 = 15.

> **En la revisión del código aparece un bug que viene de un requerimiento ambiguo. ¿Es un error o un defecto? ¿Qué gravedad tiene?**
> Es un defecto, porque nació en un producto anterior (la ERS) y se escapó hasta el código. Si puede hacer que el sistema dé un resultado inesperado, es mayor.

---

2. Qué es una revisión técnica y cuánto cuesta

[Fuente: Apuntes]

- Es un proceso de **V&V estático**: no ejecuta el programa. Su objetivo es **detectar defectos y corregirlos en etapas tempranas**.
- **Origen**: en **1976, Michael Fagan** (IBM) introdujo las **inspecciones**, basadas en la experiencia de hardware: detectar los defectos lo más cerca posible de donde se generan (Testing Temprano).
- Se puede inspeccionar **cualquier representación legible** del software: requerimientos, diseño, código, planes, casos de prueba, estimaciones…
- **Por qué revisar**: “el trabajo técnico necesita ser revisado por la misma razón que los lápices necesitan gomas: **errar es humano**”. Algunos errores se le pasan más fácil **al autor** que a otros (Independencia). Además, saber que lo van a revisar **motiva a hacer un mejor trabajo**.

Ventajas: lo que una revisión puede y el testing no.

- Se descubren muchos errores.
- Se pueden inspeccionar **versiones incompletas**, porque no hace falta ejecutar.
- Se consideran **otros atributos de calidad** (estándares, mantenibilidad), no solo el funcionamiento.

Desventajas: por qué cuesta adoptarlas. Al principio solo se ven los costos [Fuente: Apuntes]:

- es difícil introducir inspecciones formales;
- **sobrecargan los costos al inicio**, y el ahorro llega recién con la experiencia;
- requieren tiempo para organizarse y **parecen ralentizar** el desarrollo.

Son las mismas desventajas de trabajar con calidad que aparecen en 06-calidad.

Costos [Fuente: Apuntes], ordenados por **cuándo se pagan**:

- **De infraestructura**, una vez, para poder empezar: entrenamiento, plantillas e informes, guías de lectura, programa de medición, herramientas.
- **Operacionales**, cada vez que se revisa: tiempo individual y grupal, completar informes.
- **Adicionales**, lo que rodea a cada revisión: preparar el material, armar el calendario, recolectar datos, mejora de calidad.

En Crovato: decidís revisar el código de la tienda antes de cada release. El primer mes armás un checklist y aprendés a usarlo (**infraestructura**). Cada revisión te lleva una hora a vos y otra a quien te ayuda (**operacional**), más el rato de juntar los archivos y coordinar el horario (**adicional**). Al principio sentís que vas más lento (**desventaja**). Pero en la segunda revisión, antes de terminar el checkout (**versión incompleta**), encontrás un redondeo mal hecho y una función imposible de mantener que ningún test habría marcado (**ventajas**).

> **El checkout todavía no compila, pero querés encontrar problemas ya. ¿Testing o revisión?**
> Revisión: es estática, no necesita ejecutar, así que sirve con versiones incompletas. Además puede marcar problemas de mantenibilidad o de estándares que el testing no ve.

---

3. Qué tan formal: walkthrough vs. inspección

La pregunta que separa los dos tipos es **para qué hacés la revisión** [Fuente: Apuntes]:

	Walkthrough (recorrida)	**Inspección**
Para qué	**Mínima sobrecarga**, **capacitar** desarrolladores, **retorno rápido**	**Detectar y remover todos los defectos** eficiente y efectivamente
Proceso	**Informal**, poca o ninguna preparación	**Formal**, con roles, checklists, preparación previa
Mediciones	**No** hay	**Sí**
¿Es FTR (revisión técnica formal)?	**No**	Sí — fase de verificación

Walkthrough: “técnica de análisis estático en la que un diseñador o programador **dirige** a miembros del equipo y otros interesados **a través de** un producto de software, y los participantes formulan preguntas y comentarios sobre posibles errores, violación de estándares y otros problemas”. El autor guía. Es la más usada en ágil: por ejemplo, un code review informal, o un dev senior revisando con un junior un patrón de diseño [Fuente: Apuntes; ejemplo del resumen del compañero].

Qué son y qué NO son las inspecciones [Fuente: Apuntes]. La clave es que una inspección sirve para **encontrar**, no para **decidir**:

- **Son** todo lo que sale de encontrar defectos en grupo:
  - la forma **más barata y efectiva** de encontrar fallas;
  - una forma de proveer **métricas**;
  - **conocimiento cruzado**;
  - promueven el trabajo en grupo;
  - un método probado para mejorar la calidad.
- **No son** nada que implique decidir algo:
  - para **encontrar soluciones** a las fallas (cómo arreglarlo);
  - para obtener la **aprobación** de un producto;
  - para **evaluar el desempeño de las personas**.

Quién revisa qué (revisores sugeridos) [Fuente: Apuntes]. Para cada documento, revisa quien lo **usa como entrada** y quien lo **va a probar**. Por eso los **testers** están en casi todas las listas, el **diseñador** revisa la ERS (la va a usar para diseñar), el **programador** revisa el diseño detallado (lo va a programar), y **ventas/marketing** revisa la ERS y los planes (conocen al cliente y los compromisos):

- **arquitectura** → arquitecto, analista de requerimientos, diseñador, líder, testers;
- **diseño detallado** → diseñador, arquitecto, programadores, testers;
- **planes de proyecto** → líder, stakeholders, ventas/marketing, líder técnico, calidad;
- **ERS** → analista, líder, arquitecto, diseñador, testers, ventas/marketing;
- **código** → programador, diseñador, testers, analista;
- **plan de testing** → tester, programador, arquitecto, diseñador, calidad, analista.

El balance de la cátedra [Fuente: Apuntes]:

- **mejorar solo las pruebas** es caro y no remueve errores temprano;
- los **walkthroughs** dan buenos resultados, pero pocas métricas y sin control del proceso;
- las **inspecciones** dan **mejores resultados**, un proceso controlado y métricas útiles en todo el ciclo de vida.

En Crovato: un desarrollador senior se sienta con uno junior y le muestra cómo armó el cálculo de cuotas, mientras el junior le pregunta (**walkthrough**). Para el checkout, en cambio, arman una inspección formal con checklist, porque ahí no puede escaparse nada. En la reunión, alguien dice “el redondeo está mal” y otro empieza a proponer cómo arreglarlo: el moderador lo frena, porque eso lo resuelve el autor después (**no son para encontrar soluciones**). Y la ERS de las cuotas la revisan el diseñador, que la va a usar, y la tester, que la va a probar.

> **¿Una inspección sirve para aprobar un producto de trabajo?**
> No. Las inspecciones no son para obtener la aprobación del producto, ni para buscar soluciones ni para evaluar a las personas.

> **Un equipo ágil quiere feedback rápido sobre un patrón de diseño y, de paso, que el junior aprenda. No necesita métricas. ¿Qué tipo de revisión le conviene?**
> Un walkthrough: informal, con mínima sobrecarga, sirve para capacitar y da retorno rápido. No es una revisión técnica formal.

---

4. Cómo se hace una inspección

Roles

[Fuente: Apuntes] Una persona puede tener más de un rol. En una reunión para encontrar defectos hace falta alguien que **traiga** el producto, alguien que **organice**, alguien que **busque** de antemano, alguien que lo **recorra** en voz alta y alguien que **anote**:

- **Autor** (trae): creador o encargado del producto. **Inicia** el proceso asignando un **moderador**, elige con él los demás roles y le entrega el producto. Después reporta el tiempo de retrabajo y el número de defectos.
- **Moderador** (organiza y mantiene el orden): **planifica y lidera** la revisión. Entrega el producto a los inspectores **con 48 h de anticipación**, coordina la reunión evitando conductas inapropiadas y hace el **seguimiento** de los defectos.
- **Inspector** (busca): **examina el producto antes** de la reunión buscando defectos, y registra su tiempo de preparación. Todos pueden ser inspectores.
- **Lector** (recorre): **lee** el producto durante la reunión, para que no se disperse. No es el autor, para que se lea lo que está escrito y no lo que el autor quiso escribir.
- **Anotador** (anota): **registra** los hallazgos. También se lo llama “escriba”.

Las 6 etapas

[Fuente: Apuntes] Son las del **proceso convencional** de inspección. Se ordenan en antes, durante y después de la reunión.

Antes:

1. **Planificación**: elegir el equipo, preparar el material y el calendario.
2. **Visión general** (opcional): presentar el proceso y el producto.
3. **Preparación**: **análisis individual** de cada inspector, para encontrar potenciales defectos.

Durante:

4. **Reunión de inspección**: análisis del equipo para **recolectar** los defectos encontrados y **filtrar los falsos positivos**. Termina con una **decisión**:
   - **aceptar** sin modificaciones;
   - **rechazar**;
   - **aceptar provisoriamente**.

   El **informe** responde: ¿qué se revisó?, ¿quién lo revisó?, ¿qué se descubrió?, y las conclusiones.

Después:

5. **Corrección**: el autor corrige.
6. **Seguimiento**: verificar las correcciones y recolectar datos. Si los defectos son graves, hay **reinspección**.

En Crovato, inspeccionan tu `precios.ts`:

1. Le pasás el archivo a una compañera que va a moderar, y entre los dos eligen quién inspecciona (**autor**, **moderador**, **planificación**).
2. En 10 minutos contás qué hace `rebajar` (**visión general**).
3. Los inspectores lo leen solos durante dos días (**preparación**).
4. En la reunión, otra persona lo lee línea por línea (**lector**), un inspector marca “redondea antes de aplicar el descuento” y alguien lo registra (**anotador**). Deciden aceptarlo provisoriamente (**reunión**).
5. Vos corregís (**corrección**).
6. La moderadora verifica el arreglo y anota “3 defectos, 1 hora de retrabajo” (**seguimiento**).

Tasas recomendadas

[Fuente: Apuntes] La reunión dura **2 horas como máximo**:

Operación	**Código**	**Documentos**
Planificación	15 min	30 min
Vista previa	500 LOC/h	500 líneas/h
Preparación	100 LOC/h	140 líneas/h
Inspección	125 LOC/h	140 líneas/h
Mejora del proceso	30 min	45 min
Tamaño máximo por inspección	**250 LOC**	**280 líneas**

Métricas

[Fuente: Apuntes] De una inspección querés saber cuánto **encontraste**, cuánto te **costó** y si **funcionó**:

- **Cuánto encontré**:
  - **total de defectos** = mayores + menores;
  - **densidad de defectos** = total de defectos / tamaño (para comparar productos de distinto tamaño).
- **Cuánto me costó**:
  - **esfuerzo de inspección** = planificación + preparación + reunión + retrabajo;
  - **esfuerzo por defecto** = esfuerzo de inspección / total de defectos.
- **Si funcionó**:
  - **% de reinspecciones** = reinspecciones / inspecciones.

En Crovato: en la inspección de `precios.ts` encontraron 3 defectos mayores y 2 menores, 5 en total, en un archivo de 250 líneas. Son 2 por cada 100 líneas (**densidad**). Entre planificar, prepararse, la reunión y tu arreglo se fueron 10 horas: 2 horas por defecto (**esfuerzo**). Como no hizo falta volver a inspeccionarlo, cuenta como una inspección sin reinspección.

Puntos clave de una buena revisión

[Fuente: Apuntes] Una revisión sale mal por cuatro motivos, y cada punto clave evita uno:

- **Se vuelve personal → revisar el producto, no al productor**.
- **Se va de tema o se alarga →**
  - fijar una agenda y cumplirla;
  - limitar el debate;
  - **enunciar los problemas pero no intentar resolverlos** en la reunión;
  - limitar los participantes.
- **Llegan sin haber mirado nada →**
  - **insistir en la preparación previa**;
  - usar una lista de revisión (checklist);
  - disponer recursos (tiempo y lugar).
- **No se aprende de una a otra →**
  - tomar notas;
  - entrenamiento;
  - repasar revisiones anteriores.

En Crovato: en la revisión del checkout, alguien dice “esto lo hizo mal Juan”. La moderadora lo corta: se habla del código, no de Juan (**producto, no productor**). Dos personas empiezan a discutir cómo arreglar el redondeo; ella anota el problema y sigue con la agenda (**no resolver, limitar el debate**). Funcionó porque eran cuatro personas y no diez, y todos habían leído el

código con el checklist antes de llegar (**preparación**). Y al empezar, repasaron qué se había encontrado en la revisión anterior para ver si se repetía (**aprender de una a otra**).

> **¿En qué etapa no participa el autor?**
> En la preparación, que es el análisis individual de los inspectores. En la reunión está presente, pero no lee ni defiende el producto.

> **Vas a ser inspector en una revisión formal de código. ¿Qué te toca hacer antes de la reunión?**
> Leer el producto por tu cuenta y anotar los posibles defectos (preparación individual), con el checklist si lo hay, y registrar tu tiempo de preparación. Verificar que el producto esté listo para inspeccionar le corresponde al moderador.

> **Terminó una inspección y no se encontró ningún defecto. ¿Se puede concluir que no hacía falta hacerla?**
> No. Una revisión exitosa se juzga por cómo se hizo (preparación, agenda, participación, hallazgos registrados con responsables), no solo por la cantidad de defectos. Que no haya hallazgos también es información.

Referencia rápida (las listas en limpio)

No hay contenido nuevo acá: son las mismas listas de arriba, sin explicación.

- **Por qué existen las fallas**: ruido de comunicación · limitaciones de memoria · complejidad inherente.
- **Gravedad (revisiones)**: mayor · menor · cosmética.
- **Principios de V&V**: prevenir es mejor que curar · evitar es más efectivo que eliminar · la retroalimentación enseña · priorizar lo rentable · olvidarse de la perfección · enseñar a pescar.
- **Ventajas de las revisiones**: muchos errores · versiones incompletas · otros atributos de calidad.
- **Desventajas**: difícil de introducir · costo al inicio · parecen ralentizar.
- **Costos**: infraestructura · operacionales · adicionales.
- **Tipos**: walkthrough (informal, sin métricas, no FTR) · inspección (formal, métricas, FTR).
- **Las inspecciones no son para**: encontrar soluciones · aprobar el producto · evaluar a las personas.
- **Roles**: autor · moderador · lector · anotador (escriba) · inspector.
- **Etapas**: planificación · visión general (opcional) · preparación · reunión · corrección · seguimiento.

- **Decisiones de la reunión**: aceptar · rechazar · aceptar provisoriamente.
- **Métricas**: densidad de defectos · total de defectos · esfuerzo de inspección · esfuerzo por defecto · % de reinspecciones.
- **Puntos clave**: producto, no productor · agenda · limitar debate · no resolver · notas · limitar participantes · preparación previa · checklist · recursos · entrenamiento · repasar revisiones anteriores.

Preguntas de repaso

1. **Walkthrough vs. inspección.** — Walkthrough: informal, sin preparación ni métricas, para capacitar y tener feedback rápido. Inspección: formal, con roles (autor, moderador, lector, anotador, inspector), checklist, preparación individual y métricas.
2. **¿Qué NO se hace en una reunión de inspección?** — Buscar soluciones, aprobar el producto o evaluar a las personas. Se enuncian los problemas, no se resuelven.
3. **¿Revisión técnica o testing?** — Son las dos mitades de V&V: la revisión es estática (no ejecuta, sirve desde el primer documento), el testing es dinámico (ejecuta el producto). Se complementan.

Testing

> **U4 — Aseguramiento de Calidad de Proceso y de Producto** · Sección §13 del resumen completo. Índice y convención de fuentes en 00-indice. La versión ágil del tema está en 12-testing-agil-y-tdd.

Cómo leer este archivo: está escrito para leerse de corrido, como una sola explicación. Las listas de la cátedra aparecen dentro del texto, cada una una sola vez, explicada por su propósito. Al final hay una **referencia rápida** con las listas en limpio y preguntas de práctica.

Las ideas que se repiten en varios temas tienen su propia nota, y acá solo se nombran:

- Testing Temprano: probar cuanto antes. Es la misma idea que el principio 3 del ISTQB, el modelo en V y "el testing no es una fase" del testing ágil.
- Calidad Embebida: el testing controla, no asegura. Es la misma idea que "más pruebas no es más calidad" y "prevenir sobre encontrar".
- Error y Defecto: qué es lo que el testing encuentra.
- Verificación y Validación: el testing es V&V dinámico; las revisiones son el estático.

El ejemplo que recorre todo el archivo: en la tienda de Crovato vas a lanzar **"pagar en cuotas"** con MercadoPago. Cada sección lo usa para mostrar una parte del testing.

La idea del tema en dos hechos

Hecho 1: el testing encuentra defectos, pero no da calidad. Para la cátedra, el testing es un **proceso destructivo**: se trata de encontrar defectos **cuya presencia se asume**, yendo con actitud negativa, a demostrar que algo es incorrecto. Por eso **un test exitoso es el que encuentra defectos** [Fuente: Apuntes — 09 Testing de Software]. Pero encontrarlos no es lo mismo que evitarlos: el testing está **dentro** del aseguramiento de calidad, junto con el control de calidad de proceso y de producto y SCM, y lo que hace es **controlar** la calidad del producto, **no asegurarla**: "el testing NO puede asegurar ni calidad en el software ni software de calidad" [Fuente: Apuntes]. Esto es Calidad Embebida visto desde el testing.

Hecho 2: probar todo es imposible. El ejemplo de la cátedra: un sistema de 20 pantallas × 3 menús × 4 opciones × 10 campos × 100 valores da **240.000** pruebas. A 1 segundo cada una son 67 horas (8,5 días); a 1 minuto, **1,4 años**; a 10 minutos, **13,7 años** [Fuente: Apuntes]. Y el testing ya es caro: representa el **30-50% del costo** de un software confiable [Fuente: Apuntes].

Todo lo demás del tema sale de juntar esos dos hechos: hay que encontrar la mayor cantidad de defectos posible, sabiendo que no se puede probar todo. Por eso el resto del archivo responde, en orden:

1. ¿Qué buscamos y qué tan grave es? → defectos, severidad y prioridad.
2. ¿A qué escala y en qué ambiente probamos? → niveles y ambientes.
3. ¿Con qué probamos? → el caso de prueba.
4. ¿Cómo elegimos **pocos** casos que cubran **mucho**? → caja negra y caja blanca.
5. ¿Qué aspectos del sistema probamos? → tipos de prueba.
6. ¿En qué orden trabajamos y cuándo paramos? → proceso, regresión y criterios de aceptación.

La respuesta de fondo a casi todas es la misma: **el riesgo decide**. Los riesgos deciden **qué testear primero, a qué dedicarle más esfuerzo y qué no testear (por ahora)** [Fuente: Apuntes].

Definiciones formales (por si preguntan "qué es el testing" textual) [Fuente: Apuntes]:

- **ISTQB**: "todas las actividades del ciclo de vida, estáticas y dinámicas, de planificación, preparación y evaluación… para determinar que satisfacen los requerimientos, demostrar que son aptos y detectar defectos".
- **IEEE 610**: operar un sistema bajo condiciones especificadas, observar y registrar los resultados, y evaluar.
- **IEEE 829**: analizar un ítem para detectar diferencias entre las condiciones existentes y las requeridas (bugs).

Lo que el testing es y no es: mitos y principios

Los **mitos** que la cátedra pide romper son las ideas equivocadas que salen de no entender esos dos hechos. Cada uno se equivoca sobre una cosa distinta [Fuente: Apuntes]:

- **Cuándo**: "el testing es una etapa que empieza al terminar de codificar". Falso: empieza con los requerimientos (Testing Temprano).
- **Para qué**: "el testing es probar que el software funciona". Falso: es encontrar defectos (hecho 1).
- **Qué abarca**: "testing = calidad de producto" y "testing = calidad de proceso". Falso: el testing es una parte del control, no toda la calidad (hecho 1).
- **Con quién**: "el tester es enemigo del programador". Falso: trabajan para lo mismo.

Los **7 principios del testing** (son los del ISTQB) [Fuente: Apuntes] dicen lo mismo en positivo. Se entienden mejor si se los agrupa por la pregunta que responden:

- **¿Qué puede y qué no puede el testing?** El testing **muestra la presencia de defectos, no su ausencia** (1); el **testing exhaustivo es imposible** (2, hecho 2); y la **falacia de la ausencia de errores** (7): que no se encuentren defectos no significa que el sistema sirva, porque puede

no cumplir lo que el usuario necesita (es la diferencia entre verificar y validar: Verificación y Validación).

- **¿Dónde y cuándo conviene buscar? Testing temprano** (3, Testing Temprano) y **agrupamiento de defectos** (4): la mayoría se concentra en pocos módulos (~80/20), así que ahí conviene poner más pruebas.
- **¿Cómo no perder efectividad con el tiempo?** La **paradoja del pesticida** (5): si repetís siempre los mismos casos, dejan de encontrar defectos nuevos, así que hay que renovarlos. Y el testing **depende del contexto** (6): no se prueba igual un home banking que un juego.

En Crovato: probás la tienda y no encontrás errores, pero eso no quiere decir que no los haya (**presencia, no ausencia**), y probar todas las combinaciones de perfumes, cuotas y envíos llevaría años (**exhaustivo imposible**). Aunque no haya bugs, si tus clientes querían pagar por transferencia, la tienda igual no sirve (**falacia**). Pensás los casos cuando escribís la ERS (**temprano**) y, como todos los bugs del último mes estuvieron en el checkout, ahí ponés más pruebas (**agrupamiento**). Pasados unos meses, tus tests de siempre ya no encuentran nada, así que agregás casos nuevos (**pesticida**). Y como hay plata de por medio, probás los pagos con más rigor que una página de contenido (**contexto**).

Myers, citado por la cátedra, agrega consejos que salen de la psicología del testing [Fuente: Apuntes]. Buscar fallas se percibe como una crítica al producto o al autor, y construir y testear requieren mentalidades distintas. Por eso:

- un programador debería **evitar probar su propio código** (salvo las unitarias), y una unidad de programación no debería probar sus propios desarrollos;
- probar que el sistema **no hace lo que no debería** es la mitad de la batalla;
- **no hay que planificar** el testing suponiendo que no habrá defectos.

> **Un sistema pasa todas las pruebas y no tiene defectos conocidos, pero el usuario no lo usa porque no resuelve su problema. ¿Qué principio es?**
> La falacia de la ausencia de errores. Se verificó, pero no se validó.

> **Los mismos casos de regresión hace un año que no encuentran nada. ¿Qué principio describe esto y qué se hace?**
> La paradoja del pesticida: hay que revisar y renovar los casos.

> **“Para mejorar la calidad, dupliquemos los casos de prueba.” ¿Qué está mal?**
> Que el testing controla la calidad, no la construye. Más pruebas encuentran más defectos, pero no evitan que se introduzcan. Ver Calidad Embebida.

---

1. Qué buscamos y qué tan grave es

Lo que el testing encuentra son **defectos**: errores que no se detectaron en la etapa en que se introdujeron y pasaron a una etapa posterior. La diferencia con "error" está en Error y Defecto [Fuente: Apuntes].

Cuando aparece uno, hay que decidir qué hacer con él, y para eso se le hacen **dos preguntas distintas, que responden dos personas distintas** [Fuente: Apuntes]:

- **Severidad**: el que testea pregunta **"¿qué tan roto está?"**. Es la gravedad técnica, en 5 niveles: **1 Bloqueante** (no permite continuar) · **2 Crítico** · **3 Mayor** · **4 Menor** · **5 Cosmético**.
- **Prioridad**: el negocio (el cliente) pregunta **"¿para cuándo lo necesito arreglado?"**. Es la urgencia de negocio, en 4 niveles: **1 Urgencia** · **2 Alta** · **3 Media** · **4 Baja**.

Son **independientes**. Un error de ortografía en el logo de la home es **cosmético** pero de **prioridad alta** (marketing). Un crash en un reporte anual que se usa una vez por año es **crítico**, pero quizás hoy sea de **prioridad baja**.

En Crovato: el botón de pagar no responde y no se puede terminar ninguna compra → **bloqueante** y **urgente**. El nombre de un perfume está mal escrito en la página de inicio justo antes del Día de la Madre → **cosmético**, pero **prioridad alta**. El reporte de ventas anual se cuelga con más de 5 años de datos → **crítico**, pero como lo vas a usar recién en diciembre, **prioridad baja**.

> **Un tester marca un defecto como "prioridad urgente". ¿Qué está mal?**
> La prioridad no la define el tester sino el negocio. El tester define la severidad.

---

2. A qué escala y en qué ambiente probamos

Niveles de prueba

Cada nivel prueba algo **más grande** y lo compara contra algo **más cercano al usuario** [Fuente: Apuntes]:

1. **Unitarias**: **una pieza sola** contra su diseño detallado. Cada componente individual, después de construirlo, **independiente** de los demás, con acceso al código y apoyo del entorno (framework de unit test, depurador). Los errores se reparan apenas se encuentran, **sin registro oficial**. La cátedra muestra un ejemplo en **JUnit**: clase `Suma` con `setA(4)`, `setB(5)` y `assertEquals(9, suma.sumar())`.
2. **De integración**: **las piezas conectadas** contra el diseño. Verifica que partes que funcionan bien **aisladas** también funcionen **juntas**. Debe ser **incremental**:

    - **top-down**: de arriba hacia abajo, usando *stubs* para lo que falta abajo;
    - **bottom-up**: de abajo hacia arriba, usando *drivers*;
    - lo ideal es **combinarlos**.

    Hay que probar los **módulos críticos lo antes posible**, conectar de a poco y **minimizar los programas auxiliares** (stubs y drivers; su definición es Externa — Sommerville/Pressman).
3. **De sistema**: **todo armado** contra los requerimientos. Es la aplicación funcionando **como un todo** (construcción final). Verifica requerimientos **funcionales y no funcionales**: recuperación de fallas, seguridad, stress, performance. El ambiente debe parecerse **lo más posible a producción**.
4. **De aceptación**: **todo, en manos del usuario**, contra sus necesidades. Lo realiza el **usuario** para determinar si el sistema se ajusta a lo que necesita. La meta es **establecer confianza**; **encontrar defectos no es el foco principal**. Incluye:
    - **alfa**: el usuario prueba en un ambiente de **laboratorio** o del desarrollador;
    - **beta**: el usuario prueba en ambientes **reales** de trabajo.

Por qué los niveles están en ese orden: el modelo en V [Fuente: Apuntes]. Las etapas de desarrollo van de lo general a lo particular (requerimientos → diseño de alto nivel → diseño detallado → código). Cada una tiene su nivel de prueba **en espejo**, que vuelve de lo particular a lo general:

- unitarias ↔ diseño detallado;
- integración ↔ diseño;
- sistema ↔ requerimientos;
- aceptación ↔ necesidades del usuario.

La consecuencia práctica es Testing Temprano: el caso de aceptación se puede escribir apenas hay requerimientos, con **cero líneas de código**. Involucrar el testing temprano da visibilidad de cómo se va a probar y **disminuye el costo** de corregir [Fuente: Apuntes].

En Crovato:

- Probás que `rebajar()` calcule bien un 20% de descuento (**unitaria**).
- Probás que `publicarTienda()` genere un `catalogo.json` que la tienda sepa leer (**integración**).
- Con todo desplegado en un ambiente de prueba, comprás un set en oferta en 6 cuotas con la tarjeta de prueba de MercadoPago, y medís que el catálogo cargue en menos de 2 segundos (**sistema**).
- Al final, una clienta de confianza compra desde su casa y te dice si le resultó cómodo (**aceptación beta**).

Ambientes

Cada nivel necesita un lugar donde correr, y la cátedra lista 4 ambientes: **Desarrollo** → **Prueba** → **Pre-producción** → **Producción** [Fuente: Apuntes]. Cada uno se parece un poco más a la realidad, y en cada uno equivocarse cuesta un poco más. La idea de fondo sí está en la presentación: separar desarrollo de testing, y que el ambiente de sistema y de aceptación se parezca a producción.

El detalle de cada ambiente viene del resumen del compañero (no está en la presentación):

- en **desarrollo** se hacen las unitarias, y es flexible para experimentar;
- en **prueba** se hacen las de integración y sistema, y los desarrolladores no deberían tener acceso;
- **pre-producción** simula producción (carga, stress, seguridad), y ahí se hacen las de aceptación;
- **producción** es el uso real y **no es lugar para probar**.

Ejemplo real, de tu proyecto `tecnex`:

- En tu máquina corrés la API con Docker y una base `tecnex_development`; los mails no se envían, se guardan en archivos (**desarrollo**).
- Los tests corren contra otra base, `tecnex_test`, que se limpia en cada corrida, también en GitHub Actions (**prueba**).
- `tecnex-staging` en Fly.io corre con la **misma configuración que producción**, pero con su propia base y sin clientes (**pre-producción**; hoy está suspendido).
- `tecnex-production` manda mails de verdad y guarda las imágenes en la nube (**producción**).

En una palabra cada uno: **construir, probar, ensayar, usar**.

> **¿Cuál es el objetivo principal de la prueba de aceptación?**
> Establecer confianza en que el sistema sirve al usuario. Encontrar defectos no es el foco principal.

> **Integrás tu módulo con uno que todavía no existe y lo reemplazás por una versión simulada que devuelve datos fijos. ¿Cómo se llama eso y qué estrategia de integración es?**
> Es un *stub*, y se usa en la integración **top-down** (lo que falta está abajo). En bottom-up se usan *drivers* para simular lo que falta arriba.

---

3. Con qué probamos: el caso de prueba

El **caso de prueba** es “un set de condiciones o variables bajo las cuales un tester determinará si el software está funcionando correctamente o no”. Es **la unidad de la actividad de prueba** [Fuente: Apuntes]. Una buena definición permite **reproducir** defectos. El ejemplo de la cátedra: describirle a un compañero, paso a paso, cómo encontrar a Wally.

Para que alguien repita tu prueba sin preguntarte nada, necesita saber **qué** probás, **con qué** y **qué tiene que pasar**. Esas son las **tres partes** de un caso [Fuente: Apuntes]:

1. **Objetivo**: la característica a comprobar.
2. **Datos de entrada y de ambiente**: en condiciones preestablecidas.
3. **Comportamiento esperado**: la salida o acción esperada según los requerimientos.

La **condición de prueba** es la **reacción esperada** del sistema frente a un **estímulo** (formado por entradas). Cada condición se prueba en **al menos un** caso de prueba [Fuente: Apuntes].

Los casos tienen un objetivo, un criterio y una restricción, y esa tensión es el hecho 2 otra vez [Fuente: Apuntes]:

- el **objetivo** es descubrir errores (acá “errores” en sentido general: ver Error y Defecto);
- el **criterio** es hacerlo **en forma completa**;
- la **restricción** es hacerlo **con el mínimo esfuerzo y tiempo**.

Y una pista de dónde buscar: “los bugs se esconden en las esquinas y se congregan en los límites” (Boris Beizer).

De dónde se sacan los casos [Fuente: Apuntes]. Se preguntan a quien sabe cómo tiene que comportarse el sistema:

- **lo que pidió el cliente**: requerimientos, documentos del cliente, información de relevamiento;
- **lo que definió el equipo**: especificaciones de programación, código.

Solo lo que viene del cliente permite **validar**; el código solo permite **verificar** (Verificación y Validación).

En Crovato:

- **Objetivo**: que el pago en 6 cuotas cobre bien.
- **Datos**: un perfume de $60.000, 6 cuotas, MercadoPago en sandbox.
- **Esperado**: una preferencia por $60.000 en 6 cuotas.

Lo sacaste de RF-07, que te pidió el cliente, así que te sirve para validar.

> **¿Cuál es la diferencia entre la condición de prueba y los datos de entrada?**
> La condición es *qué* comportamiento querés comprobar (la reacción esperada ante un estímulo). Los datos son los valores concretos con los que armás ese estímulo en un caso. Una condición ("con monto por debajo del mínimo no ofrece cuotas") puede probarse con distintos datos ($500, $999).

---

4. Cómo elegir pocos casos que cubran mucho: caja negra y caja blanca

Como el tiempo y el presupuesto son limitados, los métodos sirven para pasar por **la mayor cantidad de funcionalidad con la menor cantidad de pruebas** [Fuente: Apuntes]. La primera pregunta es **si mirás el código o no**.

Caja negra: sin mirar el código

Se prueba la funcionalidad por **entradas y salidas**, sin mirar la implementación. Es una estrategia **dinámica** (se ejecuta) [Fuente: Apuntes]. Los casos salen de dos lugares:

De lo que dice la especificación:

- **Partición de equivalencias**. Tiene dos pasos:
  1. identificar las **clases de equivalencia válidas y no válidas**, por rango de valores continuos, valores discretos, selección simple o selección múltiple;
  2. identificar los casos de prueba.

  Si todos los valores de una clase se comportan igual, **alcanza con probar uno** por clase.
- **Análisis de valores límite**: es una variante de la anterior. En vez de cualquier valor de la clase, se eligen **los bordes**, porque es donde se esconden los bugs (Beizer).

De la experiencia:

- **Adivinanza de defectos** (conjetura de errores): ir a donde suele fallar el software. Por ejemplo: base de datos vacía o llena, sin permisos, datos inconsistentes, división por cero.
- **Testing exploratorio**: aprender el sistema mientras se lo prueba, diseñando las pruebas sobre la marcha.

Ejercicio de la cátedra (impuestos): "un empleado puede percibir hasta $4000 sin pagar impuestos; para los siguientes $1500 el impuesto es 10%; para los próximos $2000, 22%; cualquier monto superior, 40%" [Fuente: Apuntes — enunciado]. La solución de la cátedra está en imagen; **esta es una resolución propia** a partir del método:

Clase	Rango del sueldo	Válida/No	Valor representativo	Valores límite
C1	0 a 4000 → 0%	Válida	2000	0 · 4000
C2	4000,01 a 5500 → 10%	Válida	5000	4000,01 · 5500
C3	5500,01 a 7500 → 22%	Válida	6500	5500,01 · 7500
C4	> 7500 → 40%	Válida	10000	7500,01
C5	< 0 (negativo)	No válida	–100	–0,01
C6	no numérico	No válida	“abc”	—

- Con partición: **6 casos**, uno por clase.
- Con valores límite: se prueban los bordes de cada clase (unos 8 o 9 casos). Ahí es donde típicamente alguien escribió `<` en vez de `<=`.

Caja blanca: mirando el código

Se basa en la **estructura interna** del código, y permite **garantizar cobertura** (*test coverage*) [Fuente: Apuntes]. Los métodos se diferencian en **qué tan fino es lo que te asegurás de recorrer**. Cada uno exige un poco más que el anterior (definiciones: Externa — Myers, *The Art of Software Testing*, cap. 4):

Cobertura	Cada línea se ejecuta…	En el ejemplo `if (edad >= 18 && tieneDNI)`
Sentencias	cada **instrucción**, al menos una vez	2 casos: uno entra al `if`, otro al `else`
Decisión	cada **decisión** (`if`, `while`) da al menos una vez V y una vez F	acá coincide con sentencias
Condición	cada **condición individual** dentro de una decisión da V y F	(20, false) y (10, true): ¡los dos van al `else`, la decisión nunca da V!
Decisión/condición	las dos anteriores a la vez	existe por el problema de la fila anterior
Múltiple	**todas las combinaciones** de V/F de las condiciones	4: (V,V) (V,F) (F,V) (F,F)
Caminos básicos	cada **camino independiente**, usando un grafo de flujo. La **complejidad ciclomática** V(G) = decisiones + 1 es la cantidad mínima de casos (McCabe)	V(G) = 1 + 1 = 2 caminos

Limitaciones de caja blanca (resumen del compañero; coincide con Myers): la cantidad de caminos puede ser enorme. Además, probar todos los caminos **no garantiza** que el programa

cumpla la especificación, porque no detecta los **caminos faltantes** (lo que nunca se programó).

Qué método elegir

Cada método tiene fortalezas y debilidades: "**el mejor método es no usar un único método**" [Fuente: Apuntes]. Las **conclusiones de la cátedra sobre generación de casos** dicen lo mismo con más detalle [Fuente: Apuntes]:

- ninguna técnica es completa;
- las técnicas atacan problemas distintos;
- lo mejor es **combinarlas**;
- sin requerimientos, todo es más difícil;
- hay que tener en cuenta la conjetura de defectos;
- hay que ser sistemático y documentar las suposiciones.

En Crovato: probás el checkout sin abrir el código. Elegís un monto por clase y los bordes del mínimo para cuotas (**caja negra, especificación**), y además probás con el carrito vacío porque sabés que ahí suele fallar (**experiencia**). Después abrís `precios.ts` y ves `if (esSet && enOferta)`. Con un caso que entre al `if` y otro que no, cubrís **sentencias** y **decisión**. Para **condición** necesitás que `esSet` y `enOferta` den cada una verdadero y falso. Para **múltiple**, las 4 combinaciones.

> **¿Por qué cumplir la cobertura de condición no garantiza la de decisión?**
> Porque podés hacer que cada condición dé V y F sin que la decisión completa dé nunca verdadera. Con `A && B`, los casos (V, F) y (F, V) cubren las condiciones, pero los dos van al `else`.

> **Te piden "una técnica de diseño de casos basada en entradas válidas e inválidas". ¿Cuáles sirven?**
> Partición de equivalencias (clases válidas y no válidas) y su variante, análisis de valores límite.

---

5. Qué aspectos del sistema probamos: tipos de prueba

Los tipos responden a tres preguntas, en orden [Fuente: Apuntes]:

¿Vale la pena empezar a probar? → **Smoke test**. Es la **primera corrida** de los tests de sistema, y da cierto aseguramiento de que el software **no provoca una falla catastrófica**. El nombre viene de los ingenieros electrónicos: enchufar la placa y ver si sale humo. Evita arrancar el testing

formal sobre un build roto. El compañero agrega el **sanity test**: una corrida rápida después de una corrección, para ver que lo crítico sigue funcionando (no está en las presentaciones).

¿Hace lo que tiene que hacer? → **Testing funcional**. Se basa en las funciones y características y en su interoperabilidad. Puede estar **basado en requerimientos** o **basado en procesos de negocio** (un flujo completo).

¿Cómo lo hace? → **Testing no funcional**. La cátedra insiste: "¡NO HAY QUE OLVIDARLAS! Los RNF son tan importantes como los funcionales". El "cómo" tiene cuatro sentidos:

- **¿Aguanta?** → **performance** (tiempo de respuesta, concurrencia), **carga** (con el uso esperado), **stress** (más allá de lo esperado) y **volumen** (con muchos datos).
- **¿Es cómodo?** → **usabilidad** e **interfaces de usuario**. Las GUIs son mucho más complejas de probar que las interfaces de caracteres, porque el usuario tiene el control y hay muchas combinaciones.
- **¿Funciona en cualquier lado?** → **configuración** (drivers, redes, software de terceros, SO), **instalación** y **portabilidad**.
- **¿Dura en el tiempo?** → **fiabilidad** y **mantenimiento**.

En Crovato, llega una versión nueva de la tienda:

1. Abrís la página y agregás algo al carrito; si eso ya falla, no tiene sentido seguir (**smoke**).
2. Comprás un set en 6 cuotas de punta a punta (**funcional por proceso de negocio**).
3. Simulás un Black Friday con 500 compradores a la vez (**carga**) y con 2.000 (**stress**); cargás 10.000 productos (**volumen**); le pedís a tu tía que compre sola (**usabilidad**); la abrís en Safari y en un Android viejo (**configuración**); y la dejás andando una semana para ver si se cae (**fiabilidad**).

> **Llega un build nuevo y el equipo arranca a ejecutar los 200 casos del plan. A la hora descubren que el login no funciona y todo lo ejecutado no sirve. ¿Qué faltó?**
> Un smoke test: una primera corrida corta para confirmar que el build no tiene una falla catastrófica antes de empezar el testing formal.

---

6. En qué orden trabajamos y cuándo paramos

El proceso de testing

El proceso detallado de la cátedra es: Planificación → Identificación → Especificación → Ejecución → Análisis de fallas → (¿hay defectos? vuelve a ejecución) → Fin de las pruebas ("¿momento de dejar de probar?") [Fuente: Apuntes].

La versión resumida tiene 4 etapas: **Planificación y Control** → **Análisis y Diseño** → **Ejecución** → **Evaluación y Reportes**. Son las de cualquier trabajo (decidir, preparar, hacer y ver cómo salió), y cada una deja un entregable:

1. **Planificación y control** (decidir). Verificar que se entienden las metas y objetivos del cliente, de los stakeholders y del proyecto, y los **riesgos**. Se construye el **Plan de Prueba (Test Plan)**, que contiene:
   - los **riesgos y objetivos** del testing;
   - la **estrategia** de testing;
   - los **recursos**;
   - el **criterio de aceptación**.

   Controlar es revisar resultados, cobertura y criterio de aceptación, y tomar decisiones. El plan se hace **primero**, antes de diseñar y ejecutar.
2. **Identificación y especificación**, o análisis y diseño (preparar):
   - revisar la base de pruebas y verificar las especificaciones;
   - **evaluar la testeabilidad** de los requerimientos;
   - identificar los datos necesarios;
   - **diseñar y priorizar** los casos;
   - **diseñar el entorno** de prueba.
3. **Ejecución** (hacer):
   - priorizar los casos, crear los datos, **automatizar lo necesario** y armar las suites;
   - verificar el ambiente y ejecutar;
   - **registrar resultados y versiones**;
   - **comparar los resultados reales con los esperados**.
4. **Evaluación y reporte** (ver cómo salió):
   - evaluar los criterios de aceptación;
   - reportar a los stakeholders y consolidar la información;
   - verificar los entregables y que los defectos se hayan corregido;
   - **lecciones aprendidas**.

Entregables típicos (resumen del compañero), uno por etapa:

- plan de pruebas;
- casos de prueba;
- reporte de defectos o incidentes (qué pasó, qué falló, cómo reproducirlo);
- informe final (ciclos, defectos por ciclo, métodos usados).

En Crovato:

1. Apenas decidís que va a haber cuotas, escribís que el mayor riesgo es cobrar mal, y que vas a liberar con el 100% de los casos de pago pasando (**decidir**).

2. Leés RF-07 y ves que no dice si hay interés; así no se puede probar, entonces lo aclarás. Después escribís los casos CP-10 a CP-12 (**preparar**).
3. Con el checkout programado, los corrés en el sandbox y anotás que en 6 cuotas cobra $63.000 (**hacer**).
4. Al final ves que, una vez corregido, pasaron todos, y anotás que la próxima vez conviene pensar los casos de pago antes de programar (**ver cómo salió**).

Ciclos y regresión: qué pasa dentro de la ejecución

Un **ciclo de test** es la ejecución de **la totalidad** de los casos de prueba sobre **una misma versión** del sistema [Fuente: Apuntes]. Cuando termina un ciclo, se corrigen los defectos y llega una versión nueva. Ahí aparece la decisión de hacer o no regresión:

- **Sin regresión**: en el ciclo siguiente solo se vuelven a probar los casos que fallaron. Es más rápido, pero un arreglo puede romper algo que antes pasaba, y queda un **error escondido**. En el diagrama de la cátedra, el caso 3 pasaba; se arregla el caso 2 y aparece un "problema nuevo" en el 3 que nadie vuelve a mirar.
- **Con regresión**: se hace una **verificación total** de la nueva versión, corriendo todos los casos de nuevo, para **prevenir la introducción de nuevos defectos al corregir los detectados**. Ese problema se detecta y se arregla.

Dato del resumen del compañero (no está en las presentaciones): "al corregir un defecto se suelen introducir 2-3 defectos nuevos"; y "si aparece un defecto durante la corrida no se interrumpe el ciclo, salvo que sea invalidante".

Cuándo dejar de probar

Como probar todo es imposible (hecho 2), cuánto probar depende de la **evaluación del nivel de riesgo** y de los **costos**. El **criterio de aceptación**, definido en el plan, dice cuándo terminar una fase de testing [Fuente: Apuntes]. La presentación da cuatro formas de expresarlo:

- en términos de **costos**;
- **% de tests corridos sin fallas**;
- **fallas predichas que aún permanecen**;
- **no hay defectos de cierta severidad**.

La lista completa que circula entre los apuntes tiene ocho criterios. Son las respuestas a **cuatro preguntas que se hace el líder de testing antes de decir "terminamos"**:

1. **¿Me quedan recursos?** → *costo o momento específico*.
2. **¿Hice lo que había planeado?** → *ejecución exitosa del conjunto de pruebas diseñado*, *% de tests ejecutados sin fallas* y *cobertura estructural alcanzada*.
3. **¿Lo que queda roto es tolerable?** → *cantidad aceptable de fallas no críticas* e *inexistencia de defectos de determinada severidad*.

4. **¿Ya encontré lo que esperaba encontrar?** → *defectos encontrados similares a los estimados* y *fallas predichas que aún se mantienen*.

En Crovato: es jueves y el lanzamiento de las cuotas es el lunes.

1. **Recursos.** Primero mirás el calendario y la plata: si el lunes es la fecha comprometida o se terminó el presupuesto de testing, eso solo ya puede cortar (**costo o momento**). Es el criterio más pobre, porque no dice nada de la calidad, pero es el más usado.
2. **El plan.** Abrís el plan de pruebas: ibas a correr CP-01 a CP-40.
   - ¿Los corriste todos? (**ejecución del conjunto diseñado**)
   - De esos, ¿pasó el 95% que pedía el plan? (**% sin fallas**)
   - ¿Los tests recorrieron el 80% de `precios.ts`? (**cobertura estructural**)
3. **Lo que queda roto.** Cumplir el plan no significa que no quede nada roto, así que mirás los defectos abiertos. Quedan un texto sin tilde y un ícono desalineado, y el plan aceptaba hasta cinco cosméticos (**fallas no críticas aceptables**). Lo importante es que ninguno cobre mal (**sin defectos de cierta severidad**).
4. **Lo esperado.** Por versiones anteriores sabés que un cambio así trae unos 15 defectos, y encontraste 14: cuadra (**similares a los estimados**). Si hubieras encontrado 3, lo más probable es que los otros 12 sigan escondidos (**fallas predichas que permanecen**).

Hay fecha, el plan se cumplió, lo que queda es cosmético y encontraste lo que esperabas: se libera el lunes. Para reconstruirlo: **calendario** → **plan** → **defectos abiertos** → **experiencia**.

> **Escribir los pasos de un caso de prueba, ¿es ejecución?**
> No. Es diseño (análisis y diseño). Ejecutar es seguir esos pasos sobre el sistema real y comparar el resultado obtenido con el esperado.

> **Se corrigieron los 3 defectos del ciclo 1 y, para ahorrar tiempo, en el ciclo 2 solo se vuelven a probar esos 3 casos. ¿Qué riesgo hay?**
> Que la corrección haya roto algo que antes pasaba y nadie lo vea (error escondido). Para eso existe la regresión: correr todos los casos sobre la versión nueva.

> **El desarrollo pide liberar ya porque "los tests pasaron". ¿Eso alcanza para cerrar el testing?**
> No por sí solo. Hay que mirar el criterio de aceptación del plan: cobertura alcanzada, que no queden defectos de cierta severidad, que lo encontrado sea parecido a lo estimado y, si corresponde, la aprobación del cliente. La presión por liberar no es un criterio.

Referencia rápida (las listas en limpio)

No hay contenido nuevo acá: son las mismas listas de arriba, sin explicación.

- **Mitos**: empieza al terminar de codificar · es probar que funciona · testing = calidad de producto · testing = calidad de proceso · el tester es enemigo del programador.
- **7 principios (ISTQB)**: muestra presencia, no ausencia · exhaustivo imposible · temprano · agrupamiento de defectos · paradoja del pesticida · depende del contexto · falacia de la ausencia de errores.
- **Severidad**: bloqueante · crítico · mayor · menor · cosmético. **Prioridad**: urgencia · alta · media · baja.
- **Niveles**: unitarias · integración (top-down con stubs / bottom-up con drivers) · sistema · aceptación (alfa / beta).
- **Ambientes**: desarrollo · prueba · pre-producción · producción.
- **Partes del caso de prueba**: objetivo · datos de entrada y de ambiente · comportamiento esperado.
- **Caja negra**: partición de equivalencias · valores límite (especificación) · adivinanza de defectos · exploratorio (experiencia).
- **Caja blanca**: sentencias · decisión · condición · decisión/condición · múltiple · caminos básicos.
- **Tipos**: smoke · funcional (por requerimientos / por procesos de negocio) · no funcional (performance, carga, stress, usabilidad, mantenimiento, fiabilidad, portabilidad, volumen, configuración, instalación, interfaces de usuario).
- **Proceso**: planificación y control · análisis y diseño · ejecución · evaluación y reporte.
- **Plan de prueba**: riesgos y objetivos · estrategia · recursos · criterio de aceptación.
- **Criterios para dejar de probar**: costo o momento · ejecución del conjunto diseñado · % sin fallas · cobertura estructural · fallas no críticas aceptables · sin defectos de cierta severidad · defectos similares a los estimados · fallas predichas que permanecen.

Preguntas de repaso

1. **Error vs. defecto.** — Error: se detecta en la misma etapa donde se introdujo. Defecto: se escapó a una etapa posterior. Ver Error y Defecto.
2. **Severidad vs. prioridad.** — Severidad: gravedad técnica, la pone el tester (bloqueante… cosmético). Prioridad: urgencia de negocio, la define el cliente. Son independientes.
3. **¿Para qué sirve la regresión?** — Para detectar defectos introducidos al corregir otros. Sin regresión quedan errores escondidos en casos que antes pasaban.
4. **Partición de equivalencias vs. valores límite.** — La primera elige un representante por clase; la segunda elige los bordes de cada clase, donde se concentran los bugs.
5. **¿Por qué el testing no asegura la calidad?** — Porque detecta defectos sobre un producto ya construido; no evita que se introduzcan. Asegurar es prevenir durante el proceso. Ver

Calidad Embebida.

Testing ágil y TDD

> **U4 — Aseguramiento de Calidad de Proceso y de Producto** · Sección §14 del resumen completo. Índice y convención de fuentes en 00-indice. Este archivo supone leído 11-testing: acá solo está lo que cambia en ágil.

Cómo leer este archivo: igual que 11-testing, se lee de corrido. Las listas aparecen una sola vez dentro del texto; al final hay una referencia rápida y preguntas de práctica.

[Fuente general: Apuntes — 09 Agile Testing (presentación en imágenes, leída visualmente) y presentación TDD; bibliografía: Crispin & Gregory, *Agile Testing* y *More Agile Testing*]

La idea del tema: el testing deja de ser una fase

En cascada, el testing es una **fase** al final (requirements → design → implementation → verification). En ágil es un **ciclo continuo** dentro de cada iteración, que dura días y no meses o años [Fuente: Apuntes]. El chiste de la cátedra lo resume: "I don't always do agile, but when I do, I do it like waterfall". Hacer "mini-cascadas" dentro del Sprint, testeando el último día, **no** es testing ágil.

Todo el tema sale de esa diferencia. Si el testing es una **fase al final**:

- su objetivo es **encontrar** lo que quedó mal;
- lo hace un **equipo aparte**;
- necesita **documentos formales** para pasarse la información;
- prueba **desde la interfaz**, a mano (pirámide invertida);
- algo está "terminado" **cuando pasa esa fase**.

Si es una **actividad continua**, cada punto se da vuelta:

- su objetivo es **prevenir**;
- lo hace **todo el equipo**;
- los **tests automáticos son la documentación**;
- se apoya en **unitarias** (pirámide normal);
- las pruebas son **parte del Done**.

El resto del archivo desarrolla esos puntos:

1. Qué valora el testing ágil → el Manifiesto y los 9 principios (que dicen lo mismo).

2. Cómo se lleva a la práctica → las 6 prácticas, TDD, ATDD y la automatización.
3. Cómo se reparten las pruebas → cuadrantes y pirámide.
4. Quién lo hace y qué cuesta el cambio → el rol del tester y los desafíos culturales.

Dos ideas de este tema son transversales y tienen su propia nota: Testing Temprano (el "durante, no al final") y Calidad Embebida (el "prevenir, no encontrar" y "todo el equipo").

El ejemplo que recorre el archivo: la misma funcionalidad de 11-testing, "pagar en cuotas" en la tienda de Crovato, pero hecha en Sprints.

---

1. Qué valora el testing ágil

El Manifiesto del Testing

El Manifiesto dice "valoramos" y compara de a pares, como el Manifiesto Ágil [Fuente: Apuntes]. Cada valor responde a una pregunta sobre el testing, y siempre gana la opción que pone la calidad **antes y entre todos**:

Pregunta	Valoramos…	…sobre
¿Cuándo?	**pruebas durante** el desarrollo (*testing is an activity, not a phase*)	pruebas al final
¿Para qué?	**prevenir** defectos	encontrar defectos
¿Cómo?	**entender** lo que se está probando (*be a tester, not a checker*)	verificar la funcionalidad
¿Con qué actitud?	**construir un mejor sistema**	romper el sistema
¿Quién responde?	**responsabilidad de todo el equipo** por la calidad	el tester como único responsable (no hay un "quality gate" final custodiado por el tester)

Comparalo con 11-testing: allá el testing es "destructivo" y busca romper; acá busca **prevenir** y **construir**. No se contradicen: el testing tradicional describe qué hace una prueba (encontrar defectos), y el testing ágil cambia **para qué se organiza** el testing (que los defectos no lleguen a existir). Es Calidad Embebida.

En Crovato: antes probabas la tienda el día anterior a publicar, buscando qué se rompía, y si algo fallaba era "culpa del que probó". Ahora, mientras se programa el checkout, la tester se sienta con el programador a pensar qué pasa si MercadoPago rechaza la tarjeta (**durante**), y así el problema ni llega a programarse (**prevenir**). No se limita a tildar "paga en 6 cuotas: OK": entiende por qué un cliente elegiría 6 cuotas y qué esperaría ver (**entender**). Su objetivo es que la tienda venda mejor, no ganarle al programador (**construir**). Y si un bug llega a producción, es del equipo (**todos**).

Los 9 principios: el Manifiesto otra vez, con otras palabras

Acá está la repetición que confunde. Los **9 principios del testing en proyectos ágiles** [Fuente: Apuntes] **no son ideas nuevas**: la mayoría es un valor del Manifiesto dicho de otra forma. Esta tabla los ubica (la correspondencia es propia; los principios son de la cátedra):

Valor del Manifiesto	Principios que dicen lo mismo	Qué agrega el principio
Durante, no al final (Testing Temprano)	1. El testing **se mueve hacia adelante** en el proyecto · 2. Testing **no es una fase** · 9. **De probar al final a conducido por pruebas**	El 9 va un paso más allá: la prueba no solo acompaña, se escribe **antes** del código (TDD, ATDD)
Todo el equipo	3. **Todos hacen testing** · 8. Las pruebas son **parte del "Done"**	El 8 lo vuelve concreto: sin pruebas, la historia no está terminada
Entender, no tildar	5. Las pruebas **representan expectativas**	Las pruebas son la especificación de lo que se espera
Construir mejor / prevenir	6. Mantener el **código limpio**, corregir los defectos **rápido**	No acumular defectos
(no es un valor: es cómo sostenerlo ágil)	4. **Reducir la latencia del feedback** · 7. Reducir la **sobrecarga de documentación** de las pruebas	Enterarse rápido; las pruebas mismas son la documentación

Entonces, cuando leas "el testing se mueve hacia adelante", "testing no es una fase", "pruebas durante el desarrollo", "no se realizan únicamente al final" o "involucrar el testing lo más temprano posible", es **una sola idea**: Testing Temprano.

En Crovato:

- La historia "pagar en cuotas" empieza escribiendo sus pruebas de aceptación, antes del código (**1, 2 y 9**).
- El programador también escribe tests, no solo la tester (**3**).
- Esas pruebas dicen exactamente qué se espera: "6 cuotas de $10.000 para un perfume de $60.000" (**5**).
- La historia no se da por terminada hasta que pasan (**8**).
- Cada push corre los tests en minutos (**4**).
- Si algo falla, se arregla ese mismo día (**6**).
- No hay un documento de casos de prueba aparte, porque los tests son el documento (**7**).

> **¿“El testing se mueve hacia adelante” y “pruebas durante el desarrollo sobre pruebas al final” son dos ideas distintas?**
> No. Es la misma idea (Testing Temprano): una está en los principios y la otra en el Manifiesto. El principio 9 (“conducido por pruebas”) la lleva más lejos: la prueba se escribe antes del código.

---

2. Cómo se lleva a la práctica

Las 6 prácticas

Las prácticas concretas [Fuente: Apuntes] se ordenan por **quién ejecuta la prueba y cuándo se escribe**:

- **La máquina, después de cada cambio**:
  - (1) pruebas **de unidad e integración automatizadas**;
  - (2) pruebas **de regresión a nivel de sistema automatizadas**;
  - (6) **control de versión de las pruebas junto con el código**: los tests viven en el repo, al lado del código que prueban, para que la máquina siempre tenga las pruebas correctas.
- **Se escriben antes que el código**:
  - (4) **TDD** (Test Driven Development), a nivel unidad, lo hace el programador;
  - (5) **ATDD** (desarrollo conducido por pruebas de aceptación), a nivel aceptación, con el negocio.
- **Una persona, pensando**:
  - (3) **pruebas exploratorias**, que no se automatizan.

En Crovato: `setCombo.test.ts` vive en el repo al lado del código (**versionado**) y corre solo con cada cambio, junto con las pruebas de que el checkout sigue funcionando entero (**automatizadas**). Antes de programar el descuento de sets, escribiste el test que falla (**TDD**). Y antes de eso, acordaste con el dueño “un set de $50.000 con 20% se cobra $40.000” como prueba de aceptación (**ATDD**). Una vez por Sprint, alguien usa la tienda como un cliente curioso, probando cosas raras que ningún test tenía en cuenta (**exploratoria**).

TDD: la prueba escrita antes del código, a nivel unidad

[Fuente: Apuntes — TDD]

Qué es. Una técnica de Kent Beck (XP) que combina **Test First Development** + **Refactoring**, normalmente con pruebas unitarias. La cita que la justifica: “el acto de diseñar tests es uno de los mecanismos más efectivos para prevenir errores” (Beizer). Es el principio 9 llevado al código:

- **Tradicional**: diseñar → desarrollar → probar (QA **reactivo**).

- **TDD**: diseñar → **probar** → desarrollar (QA **proactivo**).

El ciclo Red-Green-Refactor: escribir un test → que compile → que **falle (Red)** → hacer el cambio mínimo para que pase (**Green**) → **quitar duplicación (Refactor)** → repetir. En cada fase escribís **una sola cosa, y la mínima**. Eso es lo que dicen **las 3 leyes de TDD** (Robert C. Martin):

- **Red**: escribís **test**, el mínimo para que falle.
  - Ley 1: no escribir código de producción hasta tener un test que falla.
  - Ley 2: no escribir más test del necesario para que falle.

  Que falle no es un trámite: confirma que el test **es capaz de detectar** que falta algo.
- **Green**: escribís **código**, el mínimo para que pase (ley 3: no escribir más código de producción que el necesario para que el test pase).
- **Refactor**: no escribís nada nuevo. **Mejorás** lo que hay sin cambiar el comportamiento.

Refactoring es una transformación que **preserva el comportamiento externo** (todas las pruebas siguen pasando) y **mejora la estructura interna**.

- **Por qué**: evitar el deterioro del diseño, simplificar, dar legibilidad, encontrar errores, incorporar lo aprendido.
- **Cómo**: con los tests en verde, encontrar código que “huele mal”, simplificarlo y correr los tests otra vez.

Los patrones se ordenan por la fase en la que estás:

- **Red bar** (¿qué test escribo?):
  - *starter test*: el primero, el caso trivial en que la operación no hace nada;
  - *one step test*: el siguiente, que enseñe una sola cosa nueva;
  - *another test*: si se te ocurre otra idea, la anotás y seguís;
  - *regression test*: ante un bug, primero el test que lo reproduce;
  - *explanation test* y *learning test*.
- **Green bar** (¿cómo lo hago pasar?):
  - ***fake it***: devolver un valor fijo;
  - ***triangulación***: un segundo test con otros datos obliga a generalizar;
  - ***obvious implementation***: si es obvio, escribirlo directamente;
  - *one to many*.
- **Otros**: *isolated test*, *test list*, *test first*, *assert first*, *evident data*.

Los beneficios se ordenan por quién gana:

- **El código**: queda robusto, predecible, limpio y tolerante al cambio, con menos duplicación y cobertura asegurada.
- **El diseño**: obliga a pensar la API y promueve **alta cohesión y bajo acoplamiento**.
- **El equipo**: mantener es más barato, con práctica se desarrolla más rápido, y se habilita el despliegue continuo.

Ejemplo mínimo (en el lenguaje de tu proyecto):

1. **Red**: `expect(precioConDescuento(1000, 10)).toBe(900)` falla porque la función no existe.
2. **Green**: `return 900` (*fake it*). Pasa.
3. **Triangulación**: agregás `expect(precioConDescuento(2000, 50)).toBe(1000)`. Falla, así que implementás `precio * (1 - pct/100)`.
4. **Refactor**: renombrás y extraés constantes; los tests siguen en verde.

En Crovato, con `totalCarrito()`:

1. Arrancás con "carrito vacío vale 0" (**starter test**), que falla porque la función no existe (**red**).
2. Escribís `return 0` (**fake it**, **green**).
3. Agregás "un perfume de $1.000 vale 1.000" (**one step**). Falla, y ya no podés devolver una constante (**triangulación**), así que escribís la suma.
4. Mientras tanto se te ocurre "¿y si la cantidad es 0?": lo anotás para después (**another test**).
5. Con todo en verde, renombrás variables y extraés una función (**refactor**).
6. Un mes después, un cliente reporta que los sets en oferta suman mal. Antes de tocar el código, escribís el test que reproduce el bug (**regression test**).

> **¿Fake it se usa en red o en green?**
> En green: es una forma de hacer pasar el test lo más rápido posible. Después la constante se elimina con triangulación (o en el refactor, quitando la duplicación con el test).

> **¿Qué no puede cambiar un refactor?**
> El comportamiento externo: todos los tests tienen que seguir pasando. Lo que cambia es la estructura interna.

ATDD: lo mismo, a nivel aceptación y con el negocio

[Fuente: Apuntes — diagrama] El ciclo de ATDD:

1. parte de una **característica con ejemplos**;
2. se escribe la **historia de usuario**;
3. se escriben las **pruebas de aceptación de alto nivel**, con el PO o el negocio (por ejemplo, los criterios de aceptación de la US: ver Criterios de Aceptación);
4. se exploran los ejemplos;
5. se programan y ejecutan las pruebas;
6. se **automatizan**;
7. se agregan más pruebas;
8. se acepta la historia.

En el medio se van corrigiendo los defectos.

La diferencia con TDD es el nivel: **TDD es a nivel unidad** (lo hace el developer); **ATDD es a nivel aceptación** (con el negocio). Conecta con los criterios de aceptación de las User Stories del 1er parcial.

Automatizar: por qué y cuánto

Casi todas las prácticas dependen de que las pruebas corran solas. Los **beneficios de automatizar** responden a "¿qué hace una máquina mejor que una persona?" [Fuente: Apuntes]:

- es más **rápida** (rápida ejecución);
- se puede **repetir** cuantas veces quieras (reusabilidad);
- llega a **más casos** (mayor cobertura y mayor alcance);
- **no se distrae** (mayor precisión);
- a la larga, **sale más barata** (reducción de costos).

Se automatiza lo mecánico y repetitivo (unitarias, integración, regresión); lo exploratorio sigue siendo manual. Y se automatiza **gradualmente**, empezando por lo más crítico o repetitivo.

---

3. Cómo se reparten las pruebas

Los cuadrantes del testing ágil

Los cuadrantes (Brian Marick, popularizados por Crispin & Gregory) [Fuente: Apuntes] clasifican **cualquier prueba** con dos preguntas:

- **¿Quién la entiende?** El negocio (arriba: Q2, Q3) o la tecnología (abajo: Q1, Q4).
- **¿Para qué sirve?** Para **guiar** mientras se construye (izquierda: Q1, Q2) o para **criticar** lo ya construido (derecha: Q3, Q4).

	Guían el desarrollo (soporte al equipo)	**Critican el producto**
Orientadas al negocio	**Q2** — *Automatizadas y manuales*: ejemplos, pruebas de aceptación de la historia, prototipos, simulaciones, pruebas de experiencia de usuario	**Q3** — *Manuales*: pruebas exploratorias, flujos de trabajo, usabilidad, **UAT**, alfa/beta, monitoreo y observabilidad
Orientadas a la tecnología	**Q1** — *Automatizadas*: pruebas unitarias, de componentes	**Q4** — *Herramientas*: rendimiento, carga, seguridad, recuperabilidad, otros atributos de calidad ("-ilidades")

La numeración va en forma de **U invertida**, empezando abajo a la izquierda: Q1 (abajo, guía) → Q2 (arriba, guía) → Q3 (arriba, critica) → Q4 (abajo, critica). **Cómo se ejecuta** cada una sale de

su lugar:

- lo de la izquierda se automatiza, para guiar rápido;
- Q3 es manual, porque hace falta una persona que juzgue;
- Q4 necesita herramientas, porque nadie simula 2.000 usuarios a mano.

Q4 son las no funcionales de 11-testing.

En Crovato:

- El test de `rebajar()` lo entiende solo un programador y te guía mientras programás (**Q1**).
- "Un set de $50.000 con 20% se cobra $40.000" lo entiende el dueño y también guía el desarrollo (**Q2**).
- Tu tía comprando sola en la tienda terminada, para ver si se entiende, juzga el producto desde el negocio (**Q3**).
- La simulación del Black Friday con una herramienta de carga juzga el producto desde la tecnología (**Q4**).

> **¿En qué cuadrante va una prueba de seguridad? ¿Y una prueba exploratoria? ¿Y la UAT?**
> Seguridad: Q4 (tecnología, critica el producto, con herramientas). Exploratoria y UAT: Q3 (negocio, critican el producto, manuales).

La pirámide del testing

La pirámide (Mike Cohn) [Fuente: Apuntes] dice **cuántas** pruebas de cada tipo conviene tener. La pregunta que la ordena es cuánto cuesta cada prueba y cuánto tarda en avisarte: cuanto más barata y rápida, más conviene tener.

Ágil (para prevenir bugs), de abajo hacia arriba:

- **base** ancha de **pruebas unitarias / de componentes**: las hace el programador, son el **80-90%**;
- **medio**: **pruebas de aceptación / capa de servicio-API / reglas de negocio / pruebas funcionales**, el **5-15%**;
- **punta** chica: **pruebas de flujo por interfaz gráfica (GUI)**, el **1-5%**;
- arriba, en una "nube": **pruebas manuales / exploratorias**.

Abajo están las automatizadas, rápidas y baratas; arriba, las lentas, frágiles y caras.

Tradicional (para buscar bugs): es la **misma pirámide dada vuelta**, el "cono de helado": **80-90% end-to-end/GUI**, 5-15% integración y 1-5% unitarias. Es lo que pasa cuando se prueba al final: se prueba todo desde la interfaz, a mano, que es lo más lento y caro de mantener.

En Crovato: la tienda tiene 40 tests de `precios.ts` que corren en 2 segundos (**base**), 6 tests que llaman al Worker como lo haría la tienda (**medio**), y 1 test que abre el navegador y hace una compra completa, que tarda un minuto y a veces falla porque cambió un botón (**punta**). Si fuera al revés, con 40 tests de navegador, tardarías 40 minutos por corrida y estarías arreglando tests rotos por cambios de diseño.

> **¿Por qué la pirámide tradicional está invertida?**
> Porque el testing tradicional prueba al final y desde la interfaz, a mano: mucho end-to-end/GUI y pocas unitarias. Es lento y caro de mantener.

---

4. Quién lo hace y qué cuesta el cambio

El rol del tester

[Fuente: Apuntes — diagrama] El tester es un **rol**, no necesariamente una persona, y trabaja **todo el Sprint**. Su función es hacer de **puente**, llevándole a cada uno lo que necesita:

- **Al Product Owner le lleva preguntas** sobre lo que se pide:
  - preparación y revisión de historias;
  - **definición de criterios de aceptación**;
  - mejor entendimiento del dominio;
  - identificar dependencias.
- **A los developers les lleva cómo se va a probar**:
  - definición de estrategias de prueba;
  - automatización de pruebas funcionales (ATDD);
  - feedback temprano sobre las historias;
  - detecta y notifica bugs;
  - sugiere mejoras de usabilidad.
- **Al System Team le lleva lo que hay que integrar**:
  - documentación de las historias a integrar;
  - plan, casos y datos de prueba de integración;
  - completitud del feature;
  - recibe bugs y los comunica.
- **Para todos, prepara el terreno y cierra**:
  - prepara y mantiene el **ambiente de pruebas**;
  - prepara los casos y datos de prueba de las historias del Sprint;
  - **valida el "Done"** y registra bugs;
  - hace la **aceptación de la historia con el PO**;
  - lleva las **métricas de calidad**.

En Crovato: la tester se sienta con vos (el PO) y te pregunta “¿las 6 cuotas tienen interés?”, y de ahí sale un criterio de aceptación (**PO**). Le cuenta al programador que va a probar con una tarjeta rechazada, así él lo tiene en cuenta desde el principio (**developers**). Cuando el checkout se integra con `controlstock`, prepara los datos para probar que el stock baje (**integración**). Al final del Sprint, verifica que todo cumpla el Done y te muestra la historia funcionando para que la aceptes (**cierre**).

Los desafíos culturales

[Fuente: Apuntes — mapa mental] Pasar de fase a actividad es un cambio de cultura. Para que funcione hay que mirar dónde cambia, qué lo frena, cómo hacerlo y quién lo tiene que apoyar desde arriba:

- **Dónde cambia: la cultura organizacional**. La filosofía de calidad, el ritmo sostenible, la relación con el cliente, el tamaño y cuánto se empodera al equipo.
- **Qué lo frena: las barreras al éxito**.
  - Pérdida de identidad del tester.
  - Roles adicionales.
  - Falta de capacitación.
  - No entender los conceptos ágiles.
  - Experiencias pasadas.
  - Diferencias culturales entre roles.
- **Cómo hacerlo: introducir el cambio**. “El cambio no es fácil”:
  - ser paciente y dejar que sientan el dolor;
  - construir credibilidad;
  - desarrollo profesional;
  - cuidado con la “mentalidad de policía de calidad”;
  - votar con los pies;
  - hablar de los miedos;
  - dar ownership al equipo;
  - celebrar los éxitos.
- **Quién apoya: las expectativas de la gerencia**. Los gerentes también tienen que cambiar, y hay que hablar el idioma del manager.

Ejemplo: en una agencia que hace tiendas como la de Crovato, la tester de siempre teme que, si “todos prueban”, ella sobre (**barrera: identidad**). No la convencés imponiéndolo. Esperás a que un bug en producción les duela a todos, y ahí mostrás cómo un test lo habría evitado (**sentir el dolor**, **credibilidad**). Cuando el equipo adopta los tests, lo celebran (**celebrar**). Y al gerente no le hablás de “pirámide de testing”, sino de “menos reclamos de clientes y menos horas de retrabajo” (**idioma del manager**).

Cierre: tradicional vs. ágil

Es la misma diferencia del principio, ahora con todo lo que se vio (síntesis propia de 11-testing y este archivo):

	Testing tradicional	Testing ágil
Cuándo	Fase al final	Actividad continua en cada iteración
Objetivo	Encontrar defectos (“romper”)	Prevenir defectos (“construir mejor”)
Quién	Tester / equipo de QA independiente	Todo el equipo; tester como rol
Documentación	Plan, casos, informes formales	Mínima; las pruebas automatizadas son la especificación
Pirámide	Invertida (mucho GUI manual)	Normal (mucha unitaria automatizada)
Aceptación	Usuario + gerente de testing + líder	Usuario + equipo, típicamente en la Sprint Review
“Terminado”	Cuando pasa la fase de testing	Las pruebas son parte de la DoD

Pistas para distinguirlos: “al final”, “después de codificar”, “estricto”, “separación estricta dev/testing”, “documentación formal exclusiva” → tradicional. “Continuo”, “colaboración”, “automatización”, “todo el equipo”, “CI” → ágil.

> **“Ejecución una vez terminado el desarrollo” y “planes de prueba detallados antes de ejecutar”, ¿de qué enfoque son?**
> Del tradicional.

Referencia rápida (las listas en limpio)

No hay contenido nuevo acá: son las mismas listas de arriba, sin explicación.

- **Manifiesto**: durante > al final · prevenir > encontrar · entender > verificar · construir mejor > romper · todo el equipo > solo el tester.
- **9 principios**: se mueve hacia adelante · no es una fase · todos hacen testing · reducir la latencia del feedback · representan expectativas · código limpio y defectos corregidos rápido · menos documentación · parte del Done · de probar al final a conducido por pruebas.
- **6 prácticas**: unidad e integración automatizadas · regresión de sistema automatizada · exploratorias · TDD · ATDD · pruebas versionadas con el código.
- **TDD**: Red → Green → Refactor · 3 leyes de Martin · patrones red bar / green bar / otros.

- **Beneficios de automatizar**: rápida ejecución · reusabilidad · mayor cobertura · mayor precisión · mayor alcance · reducción de costos.
- **Cuadrantes**: Q1 unitarias (tecnología, guía, automatizadas) · Q2 aceptación (negocio, guía, automatizadas y manuales) · Q3 exploratorias/UAT (negocio, critica, manuales) · Q4 carga/seguridad (tecnología, critica, herramientas).
- **Pirámide ágil**: unitarias 80-90% · aceptación/API 5-15% · GUI 1-5% · nube manual/exploratoria.
- **Desafíos culturales**: cultura organizacional · barreras al éxito · introducir el cambio · expectativas de la gerencia.

Preguntas de repaso

1. **¿Por qué la pirámide ágil es al revés que la tradicional?** — Porque ágil busca prevenir con feedback rápido: muchas pruebas unitarias automatizadas (baratas, rápidas) y pocas de GUI (lentas, frágiles). El tradicional prueba al final, desde la interfaz y a mano.
2. **¿En qué cuadrante va una prueba de carga? ¿Y la UAT?** — Carga: Q4 (tecnología, critica el producto). UAT: Q3 (negocio, critica el producto, manual).
3. **¿Cuáles son los pasos del ciclo de TDD?** — Red (test que falla) → Green (código mínimo para que pase) → Refactor (quitar duplicación sin cambiar comportamiento).
4. **Un equipo “ágil” programa durante 9 días del Sprint y prueba el día 10. ¿Es testing ágil?** — No: es una mini-cascada. El testing sigue siendo una fase al final, solo que más corta.

Guía para diseñar un tablero Kanban

> Esta guía explica cómo pasar de la descripción de un contexto (un equipo, su forma de trabajar, sus restricciones) a un tablero Kanban completo: qué tiene que tener, el procedimiento paso a paso, los errores típicos y una plantilla con un control final. La teoría completa de Kanban está en 03-kanban.

---

1. Qué tiene que tener el tablero

Un tablero bien diseñado para un contexto tiene **cuatro partes**:

1. **El tablero completo**: columnas, subcolumnas y WIP.
2. **Los tipos de trabajo**, **justificados**.
3. **Al menos 5 políticas**, cada una **con su propósito**.
4. Que todo **salga del contexto** y **respete las prácticas de Kanban**.

Para revisar si está bien, sirven tres preguntas:

- **¿Salió del contexto?** Cada característica del contexto tiene que verse en el tablero, y no tiene que haber columnas que el contexto no justifique.
- **¿Está completo?** Columnas, WIP, tipos de trabajo, clases de servicio y políticas.
- **¿Está justificado y se ve en el tablero?** Cada decisión tiene un porqué, y las políticas están escritas sobre el tablero, no solo en un párrafo aparte.

---

2. Lo mínimo de Kanban que se usa

Todo esto se explica en 03-kanban; acá está solo lo que se aplica en el tablero.

Columnas = el proceso. Las etapas por las que pasa el trabajo, de izquierda a derecha: entrada → actividades → terminado. Cada actividad se parte en **En progreso (EP) | Hecho (H)**. La subcolumna “Hecho” es un **buffer**: ahí espera la tarjeta hasta que la columna siguiente la tira.

WIP = máximo de tarjetas por columna, escrito arriba [Fuente: Apuntes]. Se justifica con la cantidad de personas: un WIP cercano a la cantidad de personas evita que alguien haga dos cosas a la vez. “Al 100% de capacidad se tiene un rendimiento mínimo.” Ver Limitar el WIP.

Pull, no push. Nadie asigna trabajo. Cuando una columna está por debajo de su WIP, quien tiene lugar tira la siguiente tarjeta del “Hecho” anterior. Si la columna siguiente está llena, no se

empuja: se ayuda a destrabarla.

Tipos de trabajo = carriles horizontales (filas). Son **clases de ítem**, no etapas [Fuente: Apuntes]:

- **requerimientos**: casos de uso, user stories, features;
- **defectos**: de producción, de mantenimiento;
- **mantenimiento**: refactorización, actualización de infraestructura;
- **solicitudes**: de cambio, sugerencias de mejora.

A cada carril se le puede **asignar capacidad según la demanda**; el ejemplo de la cátedra es 60% casos de uso, 30% mantenimiento, 10% defectos.

Clases de servicio = color de tarjeta, cada una con su regla [Fuente: Apuntes]:

Clase	Color	Regla de la cátedra	Cuándo usarla: ¿qué pasa si espera?
Expreso	Blanco	WIP = 1; el resto se pone en espera; **puede exceder el WIP**; **la capacidad no se reserva**; si hace falta, entrega especial a producción	Pierdo ya mismo
Fecha fija	Rosa	Queda en cola hasta que convenga; respeta el WIP; entregas programadas cuidando la fecha; si la fecha está en riesgo, **se promueve a Expreso**	Pierdo pasada una fecha
Estándar	Amarillo	Respeta el WIP; se prioriza por valor de negocio; FIFO si no hay Expreso ni Fecha fija	Pierdo de a poco
Intangible	Verde	Igual que Estándar; valor de negocio no inmediato pero importante a futuro (refactor, deuda técnica)	Hoy no pierdo, mañana sí

La columna “cuándo usarla” es una forma propia de ordenarlas; las reglas son de la cátedra.

Políticas = reglas explícitas. Son acordadas, visibles, **pocas, sencillas, bien definidas, aplicables siempre y modificables** [Fuente: Apuntes].

---

3. El procedimiento paso a paso

Son los 7 pasos de la cátedra para aplicar Kanban [Fuente: Apuntes], en el orden en que conviene hacerlos. Entre paréntesis va el número de cada paso en la lista de la cátedra.

Paso A — Leer el contexto y asignarle un lugar a cada característica

Subrayá cada característica del contexto y decidí **dónde va**: columna, carril, clase de servicio o política. Escribí la tabla **"contexto → tablero"**: muestra tu interpretación.

Señales típicas y adónde van:

Si el contexto dice…	…en el tablero va
"hay que determinar si es pertinente", "triage", "se prioriza según impacto"	una **columna de filtro** al principio (Pertinencia, Triage)
"revisión de pares", "revisiones cruzadas antes del testing"	una **columna de Revisión** antes de Testing + política "el revisor no es el autor"
"el PO valida / aprueba antes de producción", "UAT a cargo del PO"	una **columna de Validación/UAT del PO** antes de producción + política de aprobación
"CI/CD", "validaciones automáticas"	una **columna de Pipeline CI** + política de integración frecuente; pipeline roto = Expreso
"documentación como parte del DoD"	una **columna de Documentación** o un criterio de salida escrito sobre el tablero
"trazabilidad y visibilidad"	ID y datos en cada tarjeta, nada sale del tablero sin registro (por ejemplo, un área de "No pertinentes" con el motivo)
"prioridad dinámica", "incidentes críticos"	**clases de servicio** (Expreso, Fecha fija, Estándar) + política de repriorización
"flujo mixto técnico y creativo", "tareas de distinto tipo"	**carriles** separados por tipo de trabajo
"entregas por módulo / por lote"	una columna o política que agrupa tarjetas por módulo
"flujo continuo según capacidad"	**WIP en cada columna** + política de pull
"N personas con roles X e Y"	el **WIP** de cada columna se justifica con esas personas

Al terminar, controlá dos cosas: **ninguna característica quedó sin lugar**, y **no agregaste nada que el contexto no justifique**. Por ejemplo, una columna "Build" que el contexto no menciona no se justifica.

Paso B — Columnas: el proceso (paso 1 de la cátedra, “Proceso”)

Modelá el proceso en el orden en que pasa el trabajo: **Entrada/Cola** → **actividades** → **Hecho/En producción**. Partí cada actividad en **EP | H**.

El ejemplo de la cátedra: *Cola de Producto → Análisis (EP / H) → Desarrollo (EP / H) → Listo para Build → En Testing (En progreso / Listo para despliegue) → En Producción* [Fuente: Apuntes — 10 Lean Kanban]. Ojo: “Listo para despliegue” no es una columna aparte, es la subcolumna “Hecho” de En Testing. Las columnas de espera (Cola, Listo para Build) y la final (En Producción) no se parten en EP / H, porque ahí nadie trabaja. Usalo como base y agregá lo que pide el contexto (paso A).

Paso C — WIP arriba de cada columna (paso 3 de la cátedra, “Límites de WIP”)

Ejemplo de la cátedra: 5-4-4-4-2, total 19 = Cola 5 · Análisis 4 · Desarrollo 4 · Listo para Build 4 · En Testing 2 (contando En progreso + Listo para despliegue). Le pone WIP también a la cola y a la columna de espera; En Producción no lleva.

Cómo elegir el número:

- personas que hacen esa actividad, más 1 o 2 de buffer para el “Hecho”;
- una columna de una sola persona: WIP 2 (1 tarjeta en progreso y 1 tarjeta terminada en Hecho esperando que la tiren). Lo que espera es la **tarjeta**, no la persona: si las dos están ocupadas, esa persona no empieza otra y ayuda donde está el cuello de botella;
- una entrada que no se puede rechazar (incidentes): sin WIP en la entrada, pero con WIP en el triage.

Escribí **el porqué** de cada número y el total.

Paso D — Tipos de trabajo: carriles (paso 2 de la cátedra, “Trabajo”)

1. Elegí los tipos **a partir del contexto**. Si el contexto los nombra (por ejemplo, “requerimientos, defectos y mantenimiento”), usá esos.
2. **Definí cada uno**: qué abarca.
3. **Justificá** por qué existe en este contexto.
4. Asignale un **% de capacidad** y aclará que se ajusta midiendo la demanda.

Los tipos **no son las columnas**: es el error más común. Y el % de capacidad va **acá**, no como política.

Paso E — Clases de servicio con color (paso 6 de la cátedra)

Usá **solo las que el contexto necesita**, y para cada una decí **qué ítems entran ahí en este contexto**. Si usás Fecha fija o Expreso como política, explicá **la regla completa** (parte 2). Escribir solo “se permite hasta 2 ítems de fecha fija” no alcanza.

Paso F — Al menos 5 políticas (paso 4 de la cátedra; los pasos 5 y 7 entran acá)

Escribí **6 o 7**, por si alguna no cuenta. Cada una tiene que decir:

- **la regla concreta** y verificable;
- **quién** la aplica, si corresponde;
- **el propósito**: qué problema evita o qué característica del contexto cumple.

Temas para cubrir, de la lista de la cátedra:

1. **Reposición**: cuándo, cuánto y quién carga trabajo en la entrada.
2. **Criterios de "hecho" de cada actividad** (criterios de tracción): qué tiene que cumplir una tarjeta para pasar a la columna siguiente.
3. **WIP y pull**: qué se hace cuando una columna está llena.
4. **Clases de servicio**: la regla de Expreso y la de Fecha fija aplicadas al contexto.
5. **Cadencia** (paso 7 de la cátedra): reunión diaria, revisión del tablero, releases.
6. **Cuellos de botella** (paso 5 de la cátedra): mover gente hacia donde se traba el trabajo.

Buena política: "*Para pasar de Revisión a Testing, la revisión debe estar aprobada por un integrante distinto del autor, con los hallazgos registrados en la tarjeta.* — Propósito: que ningún ítem llegue a testing con defectos que una revisión detecta más barato, y dejar trazabilidad."

Mala política: "*Se realizarán revisiones técnicas cruzadas.*" Repite el contexto, no dice nada verificable y no tiene propósito.

Paso G — Escribir las políticas sobre el tablero

Escribí cada política **debajo de la columna a la que aplica** (aunque sea abreviada) y desarrollala abajo con su propósito. Ponerlas solo en un párrafo aparte no alcanza.

---

4. Errores típicos

Error	Por qué está mal	Paso
Poner **las columnas** como tipos de trabajo	Las columnas son etapas del proceso; los tipos son carriles	D
Una política que es una **intención** ("generar un flujo multidisciplinario")	No es una regla verificable	F
Políticas **solo en texto**, copiando el contexto	Tienen que verse en el tablero y decir algo concreto	G

Error	Por qué está mal	Paso
Tablero **genérico**, sin tipos ni políticas	No sale del contexto; un párrafo de reflexión no reemplaza las partes que faltan	A, D, F
% de capacidad por tipo **sin definir ni justificar** los tipos	Cada tipo necesita definición y porqué	D
Columnas que **no salen del contexto**	Todo lo que está en el tablero tiene que estar justificado	A
Algo que el contexto pone en el DoD (por ejemplo, documentación) **no se ve** en el tablero	Tiene que ser una columna o un criterio de salida visible	A, G
Reglas **arbitrarias** que contradicen el contexto	Una política tiene que servir al contexto, no ir contra él	F
El **% de capacidad** escrito como política	Va con los tipos de trabajo, no es una política	D
“Los ítems no pertinentes se remueven”	No dice quién decide, con qué criterio ni adónde van	F
“Se permite hasta 2 ítems de fecha fija”, sin más	Regla de clase de servicio **incompleta**	E
WIP **sin justificar**	El WIP se justifica con la cantidad de personas por actividad	C
Tarjetas de ejemplo que son **tareas**	Las tarjetas son ítems de trabajo (historias, defectos), no tareas	D

Un buen tablero tiene:

- columnas tomadas del contexto (por ejemplo, “Analizar pertinencia” y “Validación”), cada una con EP / H;
- WIP en cada columna;
- cada tipo de trabajo definido;
- un propósito para cada política.

---

5. Plantilla y control final

Plantilla

1. **Contexto → tablero**

Característica del contexto	Dónde va (columna / carril / clase / política)

2. **Tablero**

	Entrada	**___ (EP / H)**	**___ (EP / H)**	**___ (EP / H)**	**___ (EP / H)**	**Terminado**
WIP						—
Tipo 1 (__%)						
Tipo 2 (__%)						
Tipo 3 (__%)						

Debajo de cada columna, una línea con la política que aplica.

3. **Justificación del WIP**: personas por actividad + buffer. Total = __.
4. **Tipos de trabajo**: cada uno con definición, por qué existe en este contexto y su %.
5. **Clases de servicio**: cuáles, con color, y qué ítems del contexto entran en cada una.
6. **Políticas** (6 o 7): regla concreta + quién + **Propósito**.

Control final:

- ☐ Cada característica del contexto tiene un lugar en el tablero.
- ☐ No hay columnas que el contexto no justifique.
- ☐ Todas las actividades tienen EP / H y WIP, y el WIP está justificado con las personas.
- ☐ Los tipos de trabajo son carriles (no columnas), están definidos, justificados y con %.
- ☐ El % de capacidad está con los tipos, no como política.
- ☐ Las clases de servicio tienen su regla completa (sobre todo Fecha fija y Expreso).
- ☐ Hay al menos 5 políticas, cada una con regla concreta y propósito.
- ☐ Las políticas están también escritas sobre el tablero.
- ☐ Hay una política de pull/WIP y una de cuellos de botella o cadencia.

Definición

La calidad **se construye mientras se trabaja**; no se agrega al final. “La calidad no se inyecta ni se compra: debe estar embebida desde el inicio” [Fuente: Apuntes — 12 PPQA].

De ahí salen tres consecuencias. Todos los temas repiten alguna de ellas con otras palabras:

1. **Prevenir es mejor que detectar.** Asegurar la calidad es prevenir (durante el proceso); controlarla es detectar (sobre el producto ya hecho). Por eso **el testing controla la calidad, no la asegura**, y **aumentar las pruebas no aumenta la calidad**: el testing la revela, no la construye [Fuente: Apuntes — 09 Testing; 12 PPQA].
2. **Es responsabilidad de todos**, no de un tester o de un grupo de calidad que “la revisa al final” [Fuente: Apuntes — principios de calidad; Manifiesto del Testing].
3. **Es más barato**: trabajar con calidad cuesta menos que hacerlo sin calidad, porque corregir tarde cuesta ~10x por etapa y el retrabajo evitable es el 40-50% del desarrollo [Fuente: Apuntes — Puntos claves PPQA; 13 Revisiones Técnicas].

Cómo la llama cada tema

Tema	Cómo aparece	Consecuencia
06-calidad	Principio 1 “no se inyecta ni se compra, debe estar embebida”; principio 9 “el aumento de las pruebas no aumenta la calidad”	1
06-calidad	Principio 2 “es un esfuerzo de todos”	2
06-calidad	“Trabajar con calidad es más barato”; costo 10x por etapa	3
06-calidad · 07-aseguramiento-de-calidad	“El testing controla la calidad del producto; no la asegura”	1
11-testing	“El testing NO puede asegurar ni calidad en el software ni software de calidad”; mitos “testing = calidad de producto / de proceso”	1
12-testing-agil-y-tdd	Manifiesto: “**prevenir** defectos sobre encontrarlos”, “construir un mejor sistema sobre romperlo”	1
12-testing-agil-y-tdd	Manifiesto: “responsabilidad de **todo el equipo** sobre el tester como único responsable”; principio ágil 3 “todos hacen testing”	2

Tema	Cómo aparece	Consecuencia
12-testing-agil-y-tdd	TDD: QA **proactivo** (diseñar → probar → desarrollar) frente al QA reactivo tradicional	1
10-revisiones-tecnicas	Principios de V&V: “la prevención es mejor que la cura”, “evitar es más efectivo que eliminar”	1
02-lean	Principio “embeber la integridad conceptual”: construir calidad desde el principio, no inspeccionarla después	1
05-practicas-continuas	Las prácticas continuas como “aseguramiento de calidad embebido”: el pipeline reemplaza parte del control formal (CCB, auditorías de configuración)	1
05-practicas-continuas	SCM en ágil: “es responsabilidad de todo el equipo”, con tareas de SCM embebidas en las demás tareas del Sprint [Fuente: Apuntes — 02 SCM]	2
10-revisiones-tecnicas	Costo de corregir de 1 a ~100 según la etapa (Gilb & Graham); retrabajo evitable del 40-50%	3

Cómo se relaciona con Testing Temprano: testing temprano dice *cuándo* probar (cuanto antes). Calidad embebida dice *dónde está la calidad* (en cómo se construye, no en cuánto se prueba). Se refuerzan, pero no son lo mismo: podés probar muy temprano y aun así no tener calidad si el requerimiento está mal entendido.

Ejemplo / aplicación real

En Crovato, los bugs de cuotas venían de una ERS ambigua (no decía si había interés). Duplicar los casos de prueba no lo habría arreglado; sí lo arregla aclarar el requerimiento y revisarlo entre todos antes de programar. El testing sigue siendo necesario, pero para **confirmar** que se construyó bien, no para “darle calidad” a la tienda.

Referenciado en

```
LIST
FROM <span class="wl">Calidad Embebida</span>
```

x ## Definición

Los criterios de aceptación son las condiciones que definen los límites de una User Story: le dicen al equipo qué necesita el Product Owner para que la historia entregue valor (los requerimientos funcionales mínimos), le dan al equipo una visión compartida de la historia, ayudan a desarrolladores y testers a derivar las pruebas, y le indican al equipo cuándo dejar de agregar funcionalidad a una historia [Fuente: Apuntes].

Un buen criterio de aceptación [Fuente: Apuntes]:

- **Define una intención, no una solución** (ej.: “el usuario debe elegir al menos una cuenta para operar”, no cómo se implementa esa elección).
- Es **independiente de la implementación**.
- Es **relativamente de alto nivel**: no hace falta que describa cada detalle.

Las **pruebas de aceptación** son el paso siguiente: convierten esos criterios en tests concretos (manuales o automatizados) que demuestran que la historia fue desarrollada correctamente [Fuente: Apuntes; Fuente: Externa — Cohn, M., *User Stories Applied*, cap. 6]. El proceso tiene dos pasos: primero se anotan notas/recordatorios de tests apenas surgen en la conversación (por ejemplo, al dorso de la tarjeta si se usan tarjetas físicas), y después esas notas se convierten en tests completos [Fuente: Externa — Cohn, cap. 6]. Siempre que se pueda, conviene escribir los tests **antes** de programar la historia: eso obliga a explicitar temprano supuestos que de otro modo el programador podría pasar por alto (por ejemplo, qué pasa si falta un dato opcional) [Fuente: Externa — Cohn, cap. 6]. Quien especifica los tests es el cliente/Product Owner, eventualmente con ayuda de un programador o tester [Fuente: Externa — Cohn, cap. 6].

Ejemplo / aplicación real

Historia + criterios de aceptación + pruebas derivadas, para un GPS [Fuente: Apuntes]:

> **Buscar Destino por Dirección** Como Conductor quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme.
>
> **Criterios de Aceptación:**
>
> - La altura de la calle es un número.
> - La búsqueda no puede demorar más de 30 segundos.
>
> **Pruebas de Usuario:**
>
> - Probar buscar un destino en un país y ciudad existentes, de una calle existente y la altura existente (pasa).
> - Probar buscar un destino en un país y ciudad existentes, de una calle inexistente (falla).

> - Probar buscar un destino en un país y ciudad existentes, de una calle existente y la altura inexistente (falla).
> - Probar buscar un destino en un país inexistente (falla).
> - Probar buscar un destino en un país y ciudad existentes, de una calle existente y demora más de 30 segundos (falla).

Nótese que cada criterio de aceptación (“es un número”, “no más de 30 segundos”) se traduce directamente en al menos una prueba concreta que confirma o refuta ese criterio — esa es la relación Confirmation de las 3 C’s (ver User Story).

Referenciado en

```
LIST
FROM <span class="wl">Criterios de Aceptación</span>
```

Definición

La diferencia es **dónde se encuentra la falla respecto de dónde se introdujo** [Fuente: Apuntes — 09 Testing de Software y 13 Revisiones Técnicas]:

- **Error**: se encuentra **en la misma etapa** en que se introdujo. En revisiones: falla originada en el producto de trabajo que se está inspeccionando.
- **Defecto**: un error **que no se detectó y pasó a una etapa posterior**. En revisiones: falla originada en un producto de trabajo **predecesor**.

Falla es el término general que usa la cátedra para ambos: un error en un **producto de trabajo**, es decir, en cualquier salida del ciclo de vida (ERS, diseño, código, plan de prueba…) [Fuente: Apuntes — 13 Revisiones Técnicas].

Qué encuentra el testing: cuando el testing de sistema encuentra un bug, este viene de etapas anteriores (requerimientos, diseño, código), así que en sentido estricto son **defectos**. Las diapositivas a veces dicen "descubrir errores" (por ejemplo, el objetivo de los casos de prueba); ahí "error" está usado en sentido general, no con la definición técnica.

Cómo la llama cada tema

Tema	Cómo aparece
11-testing	Definición error/defecto; "proceso destructivo de encontrar **defectos**"; severidad (bloqueante → cosmético) y prioridad
10-revisiones-tecnicas	Error vs. defecto según el producto de trabajo inspeccionado; gravedad **mayor / menor / cosmética**
02-lean	Desperdicio "Defectos": bugs que pasan a etapas posteriores y generan retrabajo
04-metricas	Métrica "defectos" (una de las 4 básicas); densidad de defectos = defectos / tamaño

Ojo con las escalas de gravedad: testing usa 5 niveles de **severidad** (bloqueante, crítico, mayor, menor, cosmético) más 4 de **prioridad**; revisiones usa 3 (mayor, menor, cosmética). No se contradicen: son dos clasificaciones de la cátedra para dos técnicas distintas.

Ejemplo / aplicación real

En Crovato, la ERS dice "pagar en cuotas" sin aclarar si tienen interés. Si en la revisión de la ERS alguien lo marca, es un **error** (se encontró en la misma etapa). Si nadie lo nota, se programa sin

interés y el testing de sistema encuentra que el cobro no coincide con lo que quería el cliente: ese mismo problema ahora es un **defecto**, y arreglarlo cuesta reescribir código y pruebas.

Referenciado en

```
LIST
FROM <span class="wl">Error y Defecto</span>
```

Definición

Quien controla un trabajo no debería ser **quien lo hizo**, ni **depender de quien lo hizo**. Hay dos razones, y la cátedra usa las dos:

- **Ver**: algunos errores se le pasan más fácil al autor que a otros, porque construir y controlar requieren mentalidades distintas [Fuente: Apuntes — 13 Revisiones Técnicas; Myers, citado en 09 Testing].
- **Decir**: quien depende del controlado tiene incentivos para callar o suavizar lo que encontró. Sería **juez y parte** [Fuente: Apuntes — 12 PPQA].

La independencia tiene grados. En una revisión alcanza con que sean **pares** distintos del autor. En una auditoría hace falta alguien **independiente del proyecto**.

Cómo la llama cada tema

Tema	Cómo aparece	Grado
10-revisiones-tecnicas	"Algunos errores se le pasan más fácil **al autor** que a otros"; en la inspección el lector no es el autor	Pares
11-testing	Myers: un programador debería **evitar probar su propio código** (salvo unitarias); una unidad de programación no debería probar sus propios desarrollos	Otra persona u otro equipo
11-testing	Ambiente de prueba: los desarrolladores no deberían tener acceso (resumen del compañero)	Otro ambiente
07-aseguramiento-de-calidad	El GAC **no reporta al gerente de proyecto**; no más de una posición entre el GAC y la gerencia de primer nivel; reporta a alguien interesado en la calidad	Fuera de la cadena del proyecto
09-auditorias	Auditoría = "**evaluación independiente**" (IEEE 1028); el auditor es externo al proyecto; tabla revisión (pares, defectos) vs. auditoría (independiente, cumplimiento)	Fuera del proyecto

Un matiz con ágil (la relación es propia): el testing tradicional pone la independencia en la estructura (un equipo de QA aparte). El testing ágil valora la "responsabilidad de **todo el equipo**" (12-testing-agil-y-tdd, Calidad Embebida). No elimina la independencia: el tester sigue siendo un

rol, y las revisiones las hacen pares distintos del autor. Lo que cambia es que nadie es “el único responsable”.

Ejemplo / aplicación real

En Crovato, si vos escribís `precios.ts` y vos mismo lo revisás, vas a leer lo que quisiste escribir, no lo que escribiste (**ver**). Y si la persona que controla la calidad de la tienda le reporta al encargado del proyecto, que está apurado por publicar, es probable que un problema grave llegue como “detalle menor” (**decir**).

Referenciado en

```
LIST
FROM <span class="wl">Independencia</span>
```

Definición

WIP (*Work In Progress*) es el trabajo empezado y no terminado. **Limitarlo** es fijar un **máximo explícito de ítems por columna o estado**, de modo que nadie empiece algo nuevo hasta que haya lugar [Fuente: Apuntes — 10 Lean Kanban].

La razón va contra la intuición: "**al 100% de capacidad se tiene un rendimiento mínimo**". Como en una autopista en hora pico, si todos están 100% ocupados no hay holgura y el flujo se traba. "Los sistemas eficaces se centran en el **flujo del trabajo**, no en tener ocupados a los trabajadores" [Fuente: Apuntes].

En la práctica funciona con **pull, no push** ("Pull, no push!!!" en la cátedra). Nadie asigna trabajo: cuando alguien libera capacidad, **tira** la siguiente tarjeta. Y cuando una columna se llena, el equipo tiene que ayudar a destrabarla: "limitar el WIP **provoca conversación y mejora**" [Fuente: Apuntes].

Cómo la llama cada tema

Tema	Cómo aparece
03-kanban	Práctica 2, **limitar el WIP**, con pull; paso de la cátedra "límites de WIP por columna" (ej. 5-4-4-4-2)
02-lean	Desperdicio **cambio de tareas** (multitasking: "cada cambio de tarea tiene un costo de 'volver a entrar'") y desperdicio **trabajo parcialmente terminado** (el inventario del software). Los dos se atacan limitando el WIP
02-lean	"Minimizar el WIP" es de Lean/Kanban, **no** del Manifiesto Ágil
04-metricas	Lead, cycle y touch time muestran el efecto del WIP: con mucho WIP los ítems esperan, y la eficiencia (touch / tiempo transcurrido) baja

Ojo: "dejar de empezar, empezar a terminar" es una frase conocida de la comunidad Kanban, pero no está en las presentaciones de la cátedra.

Ejemplo / aplicación real

En Crovato tenés abiertas a la vez las cuotas, el rediseño del carrito y el cambio de envíos. Saltás de una a otra (**multitasking**) y ninguna llega a publicarse (**trabajo a medias**). Si ponés WIP = 1 en "Desarrollo", terminás las cuotas antes de tocar el carrito. Cada ítem tarda menos en salir (baja el cycle time), aunque haya momentos en que "no estás haciendo nada nuevo".

Referenciado en

```
LIST
FROM <span class="wl">Limitar el WIP</span>
```

Definición

Mejorar la forma de trabajar **de a poco y todo el tiempo**: mirar cómo salió, cambiar algo, ver si funcionó, y repetir. No es un proyecto que termina; es un ciclo.

En ICSW aparece en casi todos los temas con otro nombre. La idea de fondo es la misma; lo que cambia es **qué se mejora** y **cada cuánto** (esta distinción es propia, armada a partir de los temas):

- **El trabajo del equipo**, en ciclos cortos (días o semanas): Lean, Kanban, retrospectivas de Scrum y Nexus.
- **El proceso de la organización**, en ciclos largos (meses o años): el proceso basado en calidad de PPQA, IDEAL y CMMI nivel 5.

Cómo la llama cada tema

Tema	Cómo aparece	Qué se mejora
02-lean	Principio "**amplificar el aprendizaje**": ciclos cortos con feedback rápido, dejar que la solución evolucione (el compañero lo titula "Amplificar el aprendizaje – (Mejora continua)")	El trabajo del equipo
03-kanban	Principio "**acordar la búsqueda de mejora a través del cambio evolutivo**"; práctica 6 "**mejorar colaborativamente, evolucionar experimentalmente**" (si el experimento funciona se mantiene, si no se vuelve atrás); práctica 5 "ciclos de feedback"	El flujo del equipo
01-nexus	**Nexus Sprint Retrospective**, complementada "de abajo hacia arriba" por las retros de cada equipo	El trabajo de todos los equipos juntos
Scrum	Sprint Retrospective	El trabajo del equipo
07-aseguramiento-de-calidad	**Proceso basado en calidad**: definir → desarrollar → evaluar → si la calidad no está OK, **mejorar el proceso** y repetir; si está OK, estandarizar	El proceso de la organización

Tema	Cómo aparece	Qué se mejora
08-modelos-y-estandares	**IDEAL**: ciclo circular (iniciar, diagnosticar, establecer, actuar, aprender, y vuelve a empezar)	El proceso de la organización
08-modelos-y-estandares	**CMMI nivel 5, Optimizado**: “foco en la mejora continua” de procesos (CAR, OPM). También el nivel 5 de SPICE	El proceso de la organización

Para distinguir enfoques: “mejora continua del proceso” es algo que comparten Lean y el Manifiesto Ágil. Es de las pocas ideas que comparten casi todos los enfoques, así que rara vez sirve para distinguir uno de otro.

Ejemplo / aplicación real

En Crovato, cada viernes mirás qué salió mal en la semana y cambiás una sola cosa: por ejemplo, revisar los PR antes de empezar trabajo nuevo. La semana siguiente ves si funcionó (**ciclo corto, como Kanban o una retro**). Una vez por año, en cambio, revisás todo el proceso de la tienda (cómo se especifica, se prueba y se publica) contra lo que pide un modelo, y armás un plan de cambio (**ciclo largo, como IDEAL**).

Referenciado en

```
LIST
FROM <span class="wl">Mejora Continua</span>
```

Definición

Probar **lo antes posible**, en vez de dejar el testing para el final. La razón es siempre la misma: **cuanto más tarde se encuentra un defecto, más caro es corregirlo** (el costo crece ~10x por etapa) [Fuente: Apuntes — 09 Testing de Software; 13 Revisiones Técnicas, datos de Gilb & Graham]. Además, pensar cómo se va a probar algo da **visibilidad** temprana de qué se espera de él [Fuente: Apuntes].

La idea tiene **tres intensidades**, y cada tema la toma en una de ellas. No son ideas distintas: es la misma idea llevada cada vez más lejos.

1. **No al final**: el testing no es una etapa que empieza al terminar de codificar; acompaña todo el desarrollo.
2. **Desde los requerimientos**: los casos de prueba se pueden diseñar apenas hay requerimientos, con cero líneas de código (modelo en V).
3. **Antes del código**: la prueba se escribe primero y guía lo que se programa (TDD, ATDD).

Cómo la llama cada tema

Cuando leas cualquiera de estas frases, es esta misma idea:

Tema	Cómo aparece	Intensidad
11-testing	Mito “el testing es una etapa que empieza al terminar de codificar” (es falso)	1
11-testing	Principio 3 del ISTQB: “testing temprano”	1-2
11-testing	Modelo en V: cada etapa de desarrollo tiene su nivel de prueba en espejo; los casos de aceptación se escriben con cero líneas de código	2
12-testing-agil-y-tdd	Manifiesto: “pruebas **durante** el desarrollo sobre pruebas al final” (*testing is an activity, not a phase*)	1
12-testing-agil-y-tdd	Principios ágiles 1 “el testing se mueve hacia adelante” y 2 “testing no es una fase”	1-2
12-testing-agil-y-tdd	Principio ágil 9 “de probar al final a conducido por pruebas”; TDD y ATDD	3

Tema	Cómo aparece	Intensidad
10-revisiones-tecnicas	V&V “conviene comenzar temprano”; las inspecciones (Fagan, 1976) buscan “detectar defectos lo más cerca posible de donde se generan”; se pueden inspeccionar versiones incompletas	1-2
05-practicas-continuas	CI: integrar al menos una vez por día y que cada integración corra los tests; evitar una “fase de integración” al final	1
01-nexus	El NIT promueve la CI, y la Nexus Daily detecta problemas de integración durante el Sprint, no al final	1
02-lean	“Si no es posible no inspeccionar, inspeccioná después de pasos pequeños” (embeber la integridad conceptual)	1

Lo que **no** es esta idea, aunque suene parecido: “prevenir sobre encontrar” y “la calidad no se inyecta” van un paso más allá. No dicen *cuándo* probar, sino que la calidad se construye y el testing solo la revela. Eso está en Calidad Embebida.

Ejemplo / aplicación real

En la tienda de Crovato, para la funcionalidad “pagar en cuotas”:

- **Intensidad 1**: en vez de probar todo el día antes de publicar, cada push corre los tests (CI).
- **Intensidad 2**: al escribir el requerimiento “pagar en 6 cuotas”, escribís también el caso “un perfume de $60.000 en 6 cuotas genera una preferencia por $60.000”. Al hacerlo descubrís que el requerimiento no dice si hay interés, y lo aclarás antes de programar.
- **Intensidad 3**: antes de escribir `calcularCuotas()`, escribís el test que falla (TDD).

Referenciado en

```
LIST
FROM <span class="wl">Testing Temprano</span>
```

Definición

[Fuente: Apuntes — 13 Revisiones Técnicas]

- **Verificación**: ¿estamos construyendo **el producto correctamente**? Compara la **implementación** contra la **definición del problema** (lo especificado).
- **Validación**: ¿estamos construyendo **el producto correcto**? Compara el producto contra el **mundo real**, la necesidad del usuario.

Forma corta de recordarlo: verificar = "¿lo hice como lo especificamos?"; validar = "¿es lo que el usuario necesitaba?". Se puede verificar perfecto y fallar la validación: una especificación mal entendida, implementada al pie de la letra.

V&V es un proceso de **ciclo de vida completo**: empieza con revisiones de requerimientos, sigue con revisiones de diseño e inspecciones de código, y termina en la prueba. Debe planificarse y conviene comenzar temprano [Fuente: Apuntes]. Tiene **dos aproximaciones complementarias** [Fuente: Apuntes]:

- **Estática** (sin ejecutar): revisiones técnicas, inspecciones, walkthroughs.
- **Dinámica** (ejecutando): testing.

Cómo la llama cada tema

Tema	Cómo aparece
10-revisiones-tecnicas	Definición de V&V; principios de V&V; revisiones = V&V estático
11-testing	Testing = V&V dinámico. Los casos derivados de **requerimientos** permiten verificar **y validar**; los derivados del **código** solo permiten verificar. La prueba de **aceptación** es la que más valida (contra las necesidades del usuario)
11-testing	Falacia de la ausencia de errores: sin defectos (verificado) pero no sirve al usuario (no validado)
06-calidad	Los tres círculos: verificar acerca la calidad realizada a la programada; validar acerca la realizada a la necesaria
07-aseguramiento-de-calidad	Las revisiones de calidad como "principal método de validación de la calidad de un proceso o producto"
08-modelos-y-estandares	CMMI nivel 3: áreas de proceso **VER** (Verificación) y **VAL** (Validación), por separado, en la categoría Ingeniería

Tema	Cómo aparece
09-auditorias	La auditoría de proyecto verifica que los requerimientos de la ERS se validen en el Plan de V&V
05-practicas-continuas	El pipeline verifica automáticamente cada cambio; las pruebas de aceptación del pipeline van contra lo que pidió el usuario

Ejemplo / aplicación real

En Crovato, comprobar que 6 cuotas de un perfume de $60.000 generan una preferencia por $60.000, como dice RF-07, es **verificar**. Que una clienta real compre desde su casa y diga si pagar en cuotas le resultó claro es **validar**. Si RF-07 hubiera omitido el interés que el dueño quería cobrar, la verificación pasaría y la validación fallaría.

Referenciado en

```
LIST
FROM <span class="wl">Verificación y Validación</span>
```